Programming Languages and Formal Methods for GPUs

DANIEL SAINATI, University of Pennsylvania, USA
CYNTHIA RICHEY, University of Pennsylvania, USA
PAUL BIBERSTEIN, University of Pennsylvania, USA
YUKA IKARASHI, Cornell University, USA
BENJAMIN C. PIERCE, University of Pennsylvania, USA

Programming modern GPUs is challenging due to the diversity and rapid evolution of their architectural landscape and the need to manage complex parallel and asynchronous hardware to achieve optimal performance. Many new languages have been designed to address these challenges, employing a wide variety of programming models, but there is little consensus on the best approach. Formal methods for GPUs arrived later, but the difficulty of writing efficient kernels has begun to generate significant interest in verification and static analysis in this space.

In this survey, we aim to familiarize programming languages and formal methods researchers both with prior work and with the unique needs and priorities of the GPU community, which differ in critical and often surprising ways from those of the broader computing ecosystem. First, we give background on GPU hardware and the SIMT programming model and outline their major themes and challenges. Next, we genealogize existing programming languages research in the area, tracing the development of key PL ideas alongside the hardware and highlighting the cutting edge. Finally, we present a taxonomy of formal methods for GPUs, classifying work by methodology and use case and identifying prominent gaps in the literature.

1 Introduction

GPUs are everywhere: they power applications ranging from video games to physics simulations to LLMs, but despite their ubiquity and importance, they are challenging to program effectively. Their dominant programming model—Single-Instruction, Multiple-Threads, or SIMT—requires reasoning about hundreds or thousands of parallel and concurrent threads and is extremely prone to data races and deadlocks [340]. These errors are notoriously difficult to detect and debug: they are frequently nondeterministic, may only appear for certain program configurations, and often require deep knowledge of GPU hardware to fix [99, 360]. The constant evolution of this hardware also means identical programs may have entirely different behavior across machines [232].

Despite these challenges, writing a functionally correct GPU program, or *kernel*, is only the beginning of the story—making it fast is even harder. Because GPUs are primarily used for performance-critical applications, developers must often spend significant effort optimizing their programs. GPUs feature few hardware optimizations [276] and a very thin software abstraction layer [229], so the difference in performance between an optimized and unoptimized program can often be multiple orders of magnitude [278]. Moreover, the optimization search space for kernels is vast, with few universally applicable techniques, so achieving so-called "speed-of-light" performance often requires expert knowledge and heroic effort [127].

As if all this were not challenging enough, recent changes in GPU hardware design have upended the SIMT programming model [233, 235, 237], breaking its fundamental assumptions, like its per-thread view of program semantics [17], and throwing the ecosystem into disarray. GPUs' thin abstractions make these changes extremely visible to developers, who are often forced to reimplement their kernels from scratch—often with entirely new algorithms [294]—to run efficiently on new hardware.

How do programming languages and formal methods fit into this story? From GPUs' earliest days, programming languages have coevolved with hardware [206, 229], meaning that innovations in language design can have tangible impact on the broader ecosystem [331]. Currently, GPUs are at an inflection point, with previously dominant programming models struggling to adapt to new hardware [312, 355] but demand for efficient kernels greater than ever. Some have attempted to bridge

the gap by empowering old paradigms with new features [22, 61, 119, 217, 302], while others have taken inspiration from outside the world of GPUs [140, 357]. Still others argue that no one abstraction can work for all machines and programmers instead require direct control over compilation if they wish to keep up with hardware evolution [153, 154]. Which approach will win out—or whether an entirely different strategy will emerge—is currently unclear.

In formal methods, the situation is similar. Many old approaches have been invalidated by hardware changes, but the field has been slow to adapt; no tools exist that support the newest hardware features, so none can analyze kernels as they are truly written in practice. As kernels grow both more valuable and harder to write, so too grows the need for effective static analysis, compounding further as developers turn to LLMs to optimize [25, 346] and generate [63] their kernels.

GPU programming is thus fertile ground for the application of PL techniques, both to make correct and efficient kernels easier to write and to give developers more confidence in their correctness and efficiency. For the PL community's work to have an impact, however, researchers must be familiar both with prior research and the priorities and concerns of GPU programmers. In this survey, we address this need via the following contributions:

- a historical overview of GPUs and the SIMT programming model, identifying three major themes: an incessant demand for improved performance, a vast and complex optimization space, and diverse and rapidly changing hardware (Section 2),
- a genealogy of programming models and languages for GPUs, describing their coevolution with hardware features and highlighting the cutting edge (Section 3), and
- a taxonomy of formal methods for analyzing GPU programs, identifying under-served areas and key opportunities (Section 4).

2 Overview of GPUs and the SIMT Programming Model

Before we can discuss languages and formal methods for GPUs, it will be useful to develop a base of knowledge about how to program them. In this section, we provide a short history of GPUs and their development, describing their basic SIMT programming model and how they evolved from specialized graphics hardware to general-purpose accelerators (Sections 2.1 and 2.2). We will learn by example how to optimize kernels (Section 2.3) and discuss the main themes of the GPU problem space (Section 2.4). Finally, we will discuss the common challenges that GPU programmers currently face (Section 2.5) and explore how recent hardware developments have upended existing paradigms, presenting an exciting opportunity for new programming languages and static analyses (Section 2.6).

2.1 Early Graphics Processing Units

The 1980s and 1990s saw an explosion of dedicated graphics processors, originally for use in workstations—high-performance computers used for tasks like computer-aided design (CAD) and 3D modeling—and later in personal computers and video game consoles. These early devices were specialized to individual stages of the *graphics pipeline*, or the process of transforming a 3D model into a 2D image for display on a screen, but needed to rely on the CPU to connect those stages and perform any remaining computations. When the NVIDIA GeForce 256 [298] launched in 1999, it was the first mass-market chip capable of running the entire graphics pipeline on a single piece of hardware; accordingly, it was marketed as the first *graphics processing unit*, or "GPU".

The GeForce 256 was a major step forward in terms of its hardware capabilities, but, like the many graphics processors preceding it, it was not *programmable*; users were forced to interact with the hardware exclusively through its provided API, which exposed a limited set of operations. The first steps towards programmability came with Sony's Emotion Engine [183] and the GeForce 3 [205], which allowed users to customize operations in some or all stages of the graphics pipeline.

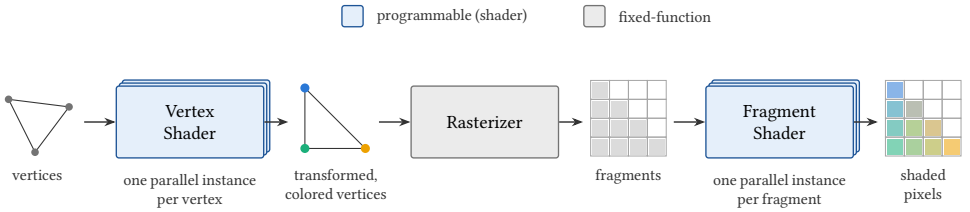


Fig. 1. A simplified graphics pipeline of the early programmable-shader era. A programmable *vertex shader* transforms each vertex and assigns it a color; a fixed-function rasterizer converts a triangle into the *fragments* it covers; and a programmable *fragment shader* computes each pixel's final color from the interpolated vertex colors. Every vertex and fragment is processed independently, so each shader stage is inherently data-parallel.

Users could write their own *shaders*, or small programs specifying how to process a single element of a particular kind (e.g., a vertex in a graphical model, or a pixel in an image), which could then be composed into a full pipeline to perform an entire graphics computation; Figure 1 depicts a representative pipeline of this era. Images and other data were *streamed* through this pipeline, with the GPU executing each individual component shader over many elements simultaneously. This model, known as Single-Instruction, Multiple Data (SIMD) [107], had appeared previously on other hardware and was a natural fit for the parallel nature of graphics workloads.

The additional flexibility offered by hardware allowed users to more easily write custom shaders using specialized *shading languages*, but it also opened the door to other uses of GPUs beyond graphics. In 2001, Larsen and McAllister [187], building on prior work by Hoff et al. [145], described a matrix multiplication procedure on the GeForce 3 that "tricked" the hardware into processing the input matrices by representing them as images. Before long, GPUs were being used in a variety of non-graphics domains, including robotics [190], numerical and physical simulation [40, 134, 201], and boolean satisfiability [330]. As general-purpose GPU (GPGPU) programming continued to proliferate, specialized processors tailored for graphics workloads no longer sufficed for the full range of applications; demand grew for hardware that could support both graphics (which still dominated) and these emerging use cases.

2.2 GPUs and the SIMT Programming Model

In 2006, NVIDIA released the Tesla microarchitecture [206], with AMD's TeraScale [148] following shortly after. These devices implemented a "unified shader model", wherein each component of the graphics pipeline became equivalent in capability. In practice, this meant that GPUs became a homogeneous collection of directly programmable parallel processors, no longer tethered to the basic structure of graphics workloads. Every part of the pipeline could run on the same hardware, dramatically increasing its flexibility. This change proved successful; while the hardware has continued to evolve (as we will see in Section 2.6), modern GPUs still share this same fundamental design [6, 235, 237].

Despite this success, however, the unified shader model, as realized by Tesla and its descendants, presents a very thin abstraction over the hardware, making GPUs extremely challenging to program. Programmers are forced to reckon directly with factors like the performance characteristics of RAM and manual cache management in order to write efficient code. To understand these challenges firsthand, in this section we explore the hardware and programming model of GPUs at a particular point in time: NVIDIA's Pascal [231] microarchitecture. This period in the evolution of GPUs is a good starting point for those seeking to understand the basics of GPU programming; it is complex enough to reveal many of the subtleties of its *Single-Instruction, Multiple-Threads* (SIMT) programming model, but does not yet include any of the newer features that would later complicate and ultimately undermine it.

2.2.1 The Streaming Multiprocessor. An NVIDIA Pascal GPU consists of a number of *streaming multiprocessors* (SMs), hardware units roughly analogous to CPU cores that execute independently

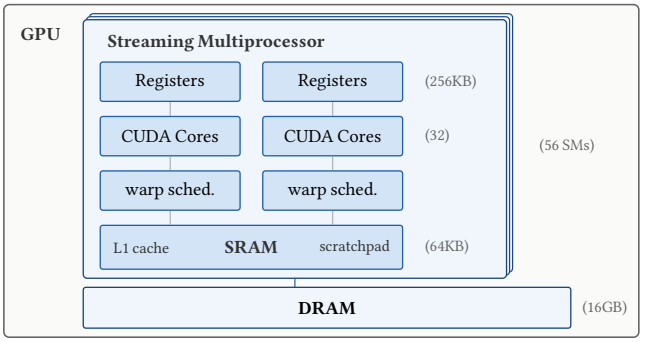


Fig. 2. Simplified SIMT hardware organization, annotated with NVIDIA Pascal P100-scale capacities. A GPU contains many streaming multiprocessors (SMs); each SM has two warp schedulers, two groups of CUDA cores, a large partitioned register file, an L1 cache, and programmer-managed shared memory (scratchpad). All SMs share the GPU's off-chip DRAM.

and in parallel; the number of SMs on a GPU varies between architectures but has trended upwards with time. Each SM, in turn, consists of a number of CUDA Cores (basic compute units for processing integer or floating-point operations) and *warp schedulers*, which decide at the hardware level which computations to assign to which Cores at each clock cycle. Each warp scheduler is responsible for a group of 32 CUDA Cores, and Pascal's SMs feature two warp schedulers (for a total of 64 Cores), while SMs on newer architectures like Hopper [235] and Blackwell [237] have four. Each warp scheduler has an associated program counter—meaning that CUDA Cores administered by the same warp scheduler always execute the same instruction at the same time—and a register file, which stores data private to each CUDA Core; one Core cannot access the parts of the register file that belong to another. Some literature uses the term "warp scheduler" to refer to both the actual scheduler and the Cores for which it is responsible, but in this discussion we will distinguish between the two.

In addition to its CUDA Cores, warp schedulers, and register file, an SM also contains fast SRAM that is accessible by all CUDA Cores, but not by any other SM. As on the CPU, SRAM can be used as an automatically populated L1 cache, but in an important departure from CPUs, it can also be used as manually managed scratchpad memory, giving the programmer control over when data is loaded into and evicted from it. Beyond SRAM is DRAM, the GPU's main memory, which is accessible by all SMs. DRAM is significantly larger than SRAM but is not located on the same physical die as the SMs; requests to it are extremely high-latency relative to registers or SRAM.

Pascal's DRAM and SRAMs have particular access behaviors designed to maximize data movement speed in common GPU applications. For instance, reads from and writes to DRAM occur at a 128-byte granularity; that is, when requesting data from DRAM, an entire 128-byte chunk of data will be returned by the hardware, starting at the nearest 128-aligned address containing the request. When multiple CUDA Cores under the same warp scheduler issue simultaneous requests to DRAM, these requests are grouped together by the hardware in a process called *memory coalescing* to minimize the number of chunks that must be loaded. In the most common case, the 128-byte chunk size allows each of the 32 CUDA Cores attached to a warp scheduler to simultaneously load one 4-byte word[1] or float in a single request to DRAM; typically (but not always) this occurs when each Core requests a consecutive word. The coalescing process extends to larger data types as well; if, for example, each Core were to request a vector of four floats (for a total request size of 512 bytes across all 32 Cores), the hardware would attempt to coalesce those requests into four chunks and thus four DRAM loads. Accesses that can be perfectly packed into the minimal number of chunks for the requested amount

[1]GPUs are primarily designed for 32-bit, not 64-bit, computation.

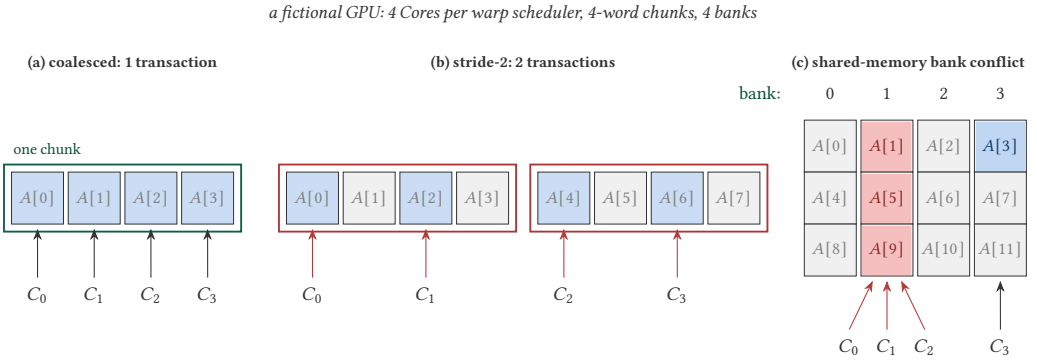


Fig. 3. Access patterns on a fictional GPU with 4 Cores per warp scheduler, 4-word chunks, and 4 shared-memory banks (real NVIDIA GPUs use 32 of each). In (a), consecutive loads coalesce into one transaction; in (b), stride-2 loads touch two chunks and fetch twice the data they need; in (c), word $A[i]$ lives in bank i mod 4, so Cores C_0, C_1, and C_2, which access $A[1]$, $A[5]$, and $A[9]$, serialize on bank 1 while C_3 reads $A[3]$ in parallel.

of unique data (i.e., 1 chunk for 128 bytes, 4 chunks for 512 bytes, etc.) are called *coalesced* [240], while accesses that span more chunks are called *uncoalesced*.

SRAM, on the other hand, is divided into 32 *memory banks*, with its address space striped across these banks with a width of four bytes. Consecutive 4-byte words map to consecutive banks, so address `0x00` would be contained in bank 0, address `0x04` in bank 1, address `0x08` in bank 2, and so on. Every 128 bytes, the bank numbers wrap around; address `0x7C` (i.e., 124) maps to bank 31, but `0x80` (i.e., 128) maps to bank 0. This structure is designed to allow the 32 CUDA Cores associated with a warp scheduler to each read consecutive words from SRAM in parallel; each memory bank acts in parallel with all the others, but accesses to different addresses within the same bank (e.g., `0x00` and `0x80`) must be handled sequentially. Two CUDA Cores attempting to simultaneously request different words from the same SRAM memory bank results in a *bank conflict* [254], where the two requests must be ordered and processed sequentially, slowing down data movement. Figure 3 illustrates coalesced, uncoalesced, and bank-conflicting access patterns.

2.2.2 The SIMT Programming Model. Understanding the hardware is important for performance engineering, but many of these details are abstracted away when actually writing code for GPUs. In particular, programmers primarily interact with GPUs via the Single-Instruction, Multiple-Threads (SIMT) programming model. This model first appeared in CUDA [229] alongside the Tesla microarchitecture, but has since been adopted by most major GPU manufacturers. The terminology used to describe the SIMT model differs across its various implementations; in this survey we will use the names associated with CUDA, but AMD's HIP [8], Apple's Metal [16], and Khronos Group's OpenCL [309] use different terms for many concepts. These terminological differences are summarized in Figure 4.

In the SIMT model, programmers control the behavior of their programs, or *kernels*, by specifying the behavior of individual *threads*. The thread is the lowest level of the GPU's *execution hierarchy*, representing the most basic compute resource available on the GPU. Threads each have their own ID and *local memory*, which is inaccessible to other threads; local memory sits at the bottom of the *memory hierarchy*, mirroring the execution hierarchy. Threads operate independently in general, but execute in lockstep with others in the same *warp*—a group of 32 consecutive threads, where the first thread in the warp has an ID that is a multiple of 32.

Programmers often assign the same instructions to most or all of the threads executing a kernel, leveraging the fact that each thread sees different local data to perform computations over large amounts of data in parallel. One common intuition for this sort of *streaming parallelism* is to think of

CUDA [229] — NVIDIA	**HIP** [8] — AMD	**Metal** [16] — Apple	**OpenCL** [309] — Cross-platform
Thread	Work-item	Thread	Work-item
Warp	Wavefront	SIMD group	Sub-group
(Thread) Block	Work-group	Threadgroup	Work-group
Grid	Grid	Grid	NDRange (N-Dimensional Range)
Local Memory	Local Memory	Thread Memory	Private Memory
Shared Memory	Shared Memory	Threadgroup Memory	Local Memory
Global Memory	Global Memory	Device Memory	Global Memory

Fig. 4. Terminological differences between CUDA, HIP, Metal, and OpenCL. The size of a warp equivalent varies across manufacturers; a CUDA warp, for example, is always 32 threads, while a Metal SIMD group is 16 threads and an AMD wavefront is 64 work-items.

a thread's ID as an index into a "parallel `for` loop" wrapping a program; when executing a statement like `out[threadIdx.x] = in[threadIdx.x]`, each thread copies a different cell of `in` into `out`.

Threads can also make control decisions based on their data or IDs, leading to situations in which different threads execute entirely different instructions. When two such threads belong to different warps, they can proceed in parallel, but threads within the same warp must execute synchronously; if two threads in the same warp take different branches, those threads cannot execute different instructions at the same time. Instead, execution is *masked*, or *predicated* [149], whereby threads are selectively enabled or disabled based on whether they pass or fail the guard of a conditional. In the example in Figure 5, only the first 16 threads in a warp will pass the conditional of the `if` statement. The other 16 (those with IDs 16 through 31) will be disabled while the success branch executes. Once that branch is finished, the situation reverses: the first 16 threads are disabled while the others are enabled, and the failure branch of the conditional proceeds. This pattern, wherein threads in the same warp are assigned different instructions—leading to unnecessarily sequential execution of otherwise parallel code—is called *warp divergence* and is a common source of performance degradation in GPU kernels.

```
1 if (threadIdx.x < 16) {
2     x++;
3 } else {
4     x--;
5 }
```

Fig. 5. Divergent control flow based on thread ID.

Ascending one level up the execution hierarchy, the threads and warps executing a kernel are organized into *blocks*. The number of threads per block is configurable at each invocation, or *launch*, of a kernel (up to a hardware-dependent maximum), but these sizes are usually multiples of 32 so that the threads within each block can be evenly distributed into warps. Threads in the same block but not the same warp execute entirely independently (i.e., in parallel), but they are guaranteed to have the same view of *shared memory*—the level of the memory hierarchy matching the block's place in the execution hierarchy. Shared memory, or SHMEM, has the same contents and address space for all the threads in a block and is the primary medium for intra-block communication—to avoid data races, the programming model also includes barrier instructions to synchronize all the threads in a block after a shared memory operation[2]. Barriers must be executed by every thread in a block, or the kernel will experience *barrier divergence*, which is undefined behavior.

Different blocks, however, have separate shared memory and must instead communicate through *global memory*, which sits at the top of the memory hierarchy and is common to the entire collection of blocks executing a kernel, also known as the *grid*. Like the number of threads per block, the number of blocks in the grid is configurable at run time when the kernel is launched by the CPU, or *host*.

2.2.3 Latency Hiding and Occupancy. Having laid out the programming model and structure of the hardware, we can now consider how the one maps onto the other. When executing a kernel, instead of naïvely assigning threads, warps, and blocks one-to-one to CUDA Cores, warp schedulers, and

[2]Use of barriers rather than locks means that much of the existing research on lock-based synchronization does not translate to GPUs [203, 368].

SMs, each CUDA Core actually hosts *multiple* threads, each warp scheduler multiple warps, and each SM multiple blocks; selecting which logical warp is active on the CUDA Cores at each clock cycle is the primary function of the warp scheduler. When a warp would stall on a high-latency operation like a DRAM load, the warp scheduler can dynamically decide to pause that warp—saving the state of all its threads to the register file—and load the threads of a different warp onto that scheduler's CUDA Cores. That new warp can execute until it too encounters a long instruction, and a paused warp can be reloaded from registers onto the CUDA Cores to pick up where it left off. In this way, the latency associated with slow operations is *hidden* by scheduling other computations on the same hardware.

This dynamic scheduling capability is one of the GPU's most important hardware optimizations and allows the grid to contain more blocks than the GPU has SMs and individual blocks to contain more warps than SMs have warp schedulers. The SIMT model (in particular, the mandate that warps within the same block have the same view of shared memory) simply requires the hardware to ensure that warps in the same block are always scheduled on the same SM. The *occupancy* [249] of an SM refers to the ratio of active warps on that SM to the maximum number of warps it could schedule concurrently; the greater the occupancy of a GPU's SMs for a given kernel, the better it is at hiding latency. Thus, to maximize occupancy, GPU programmers frequently *oversubscribe* the hardware, launching kernels with far more blocks per grid and threads per block than the hardware can execute simultaneously. This minimizes the frequency with which a warp is stalled on an expensive operation but no other warps are eligible to be scheduled in its place.

2.3 A Sample of GPU Programming and Kernel Optimization

Having discussed the SIMT model abstractly, we now turn our focus to implementing and optimizing kernels within it. To optimize a kernel, one must first understand how its speed is limited, or *bound*: a kernel is *compute-bound* if it is bottlenecked by how quickly the GPU can perform arithmetic operations, and *memory-bound* if it is bottlenecked by how quickly the GPU can move data to and from (global) memory. The roofline model [350] concretizes this notion by plotting a kernel's performance against its *arithmetic intensity*, or its ratio of arithmetic operations to memory operations. The "roofline," as depicted in Figure 6, is formed by intersecting lines representing the maximum memory and arithmetic bandwidths of the GPU; kernels to the left of the intersection point are memory-bound, while kernels to its right are compute-bound.

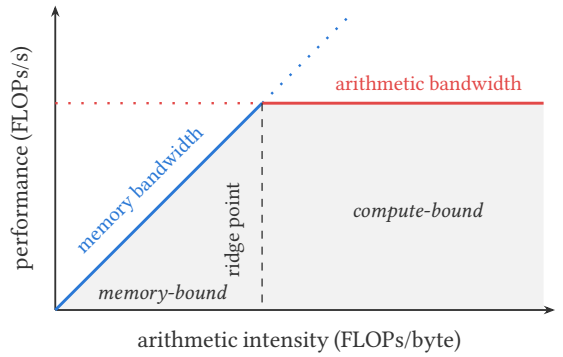


Fig. 6. The roofline model [350]. A kernel's attainable performance is capped by the lower of two ceilings; kernels to the left of the ridge point are memory-bound, while those to its right are compute-bound.

In practice, kernels rarely reach the roofline, but the model remains useful because it tells us where our optimization effort is best spent. The first kernel we will optimize in this section will be a matrix transpose, an operation that switches the row and column indices of a matrix. Transpose performs very little arithmetic, so it is a memory-bound kernel—as a result, our optimization effort will focus on improving memory access patterns.

2.3.1 Optimizing Transpose. The transpose algorithm is easy to understand and specify: the element at row i and column j is written to row j and column i; Figure 7 depicts a straightforward implementation of this algorithm in CUDA. Before examining its performance, recall the SIMT programming

model from Section 2.2.2; CUDA surfaces many of its ideas. The body of the `__global__` function describes the work of one thread, and launching a kernel runs that function across every thread in the grid. Identifiers like `threadIdx.x` and `blockIdx.x` associate each thread and block with a unique ID, which they use to determine which part of the input they should process. The static constant `SIZE` refers to the dimensions of the matrix being transposed (for simplicity in this example we assume that it is square), which is further divided into square *tiles* with dimensions `TILE_DIM`. Tiling kernels this way is a common design pattern and a form of recursive task decomposition, which improves data locality and modularity by structuring code to mirror the compute hierarchy. In this case, each block transposes one tile of the input matrix independently of other blocks.

Depending on a kernel's launch configuration and tile size, a block may not have enough threads to process every element of its tile at once. The loop on line 11 handles this: the first `BLOCK_ROWS` rows of the tile are transposed in parallel, followed by the next `BLOCK_ROWS`, and so on until the end of the tile. Each thread thus transposes several elements of its column, spaced `BLOCK_ROWS` apart.

```
1  __global__ void transposeNaive(float *odata, const float *idata) {
2    const int block_x = blockIdx.x % (SIZE / TILE_DIM);
3    const int block_y = blockIdx.x / (SIZE / TILE_DIM);
4
5    const int thread_x = threadIdx.x % TILE_DIM;
6    const int thread_y = threadIdx.x / TILE_DIM;
7
8    const int col = block_x * TILE_DIM + thread_x;
9    const int row = block_y * TILE_DIM + thread_y;
10
11   for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS)
12    // reads are coalesced, writes are not
13     odata[col * SIZE + row + j] =
14       idata[(row + j) * SIZE + col];
15 }
```

Fig. 7. A naïve matrix transpose in CUDA.

Now, consider the memory access patterns of our kernel. The easiest way to do this is to choose constants for the various values—say `SIZE = 1024`, `TILE_DIM = 32`, and `BLOCK_ROWS = 8`—and examine the addresses that each thread reads from and writes to. For the first three threads in block 0, the read addresses are 0, 1, and 2, but the write addresses are 0, 1024, and 2048! This points to a coalescing problem, which we previously discussed in Section 2.2.1: the reads are coalesced, but the writes are *strided* by 1024—that is, the index into `odata` differs by 1024 between consecutive threads. As a result, a warp's 32 reads all fit within a single 128-byte chunk—one trip to DRAM—but writes require 32 separate trips, one for each thread. This uncoalesced write results in an unacceptable performance penalty, and—worse—the problem is inherent to the data movement involved: one might attempt to invert the order of reads and writes so that consecutive threads write to consecutive addresses in `odata`, but this will result in an uncoalesced *read* pattern instead. The only solution is to somehow eliminate the strided access pattern entirely.

As it turns out, each block's shared memory can be used for exactly this purpose. Although it is much smaller than global memory, shared memory is quicker to access, and accesses do not need to be chunked. Here we see another benefit of decomposing our input matrix into `32 * 32` tiles: we can load our tile into shared memory using a coalesced read, do the strided work on-chip, and eventually perform a (coalesced) write back to global memory. The code in Figure 8 implements this process: each block reads its tile from `idata` into `tile` using a coalesced read, storing it in its original orientation. The transpose happens when the tile is read back out again on line 23: it is written as `tile[(thread_y + j) * TILE_DIM + thread_x]` but read as `tile[thread_x`

```
1  __global__ void transposeCoalesced(float *odata, const float *idata) {
2    __shared__ float tile[TILE_DIM * TILE_DIM];
3
4    const int block_x = blockIdx.x % (SIZE / TILE_DIM);
5    const int block_y = blockIdx.x / (SIZE / TILE_DIM);
6
7    const int thread_x = threadIdx.x % TILE_DIM;
8    const int thread_y = threadIdx.x / TILE_DIM;
9
10   int col = block_x * TILE_DIM + thread_x;
11   int row = block_y * TILE_DIM + thread_y;
12
13   for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS)
14     tile[(thread_y + j) * TILE_DIM + thread_x] =
15       idata[(row + j) * SIZE + col];
16
17   __syncthreads();
18
19   col = block_y * TILE_DIM + thread_x; // transpose block offset
20   row = block_x * TILE_DIM + thread_y;
21
22   for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS)
23     odata[(row + j) * SIZE + col] =
24       tile[thread_x * TILE_DIM + thread_y + j];
25 }
```

Fig. 8. A coalesced matrix transpose in CUDA.

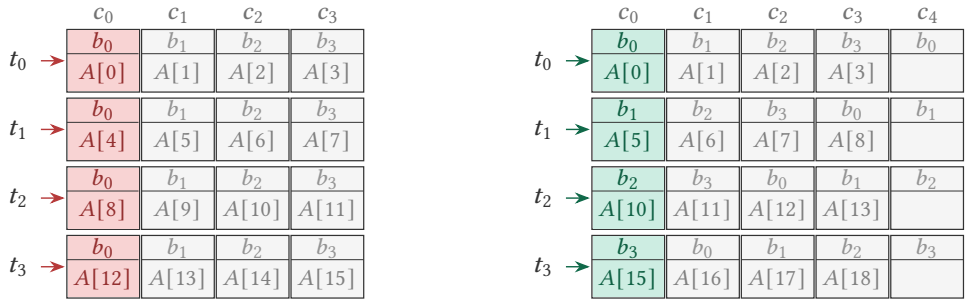


Fig. 9. Bank conflicts occur when threads access data in the same memory bank. In this example, we consider a `4 * 4` tile of a matrix, stored in `A`, and use a warp size of four threads for simplicity. On the left, naïvely filling `A` with the tile's entries causes values in the same column to be stored in the same bank, so threads accessing such values must serialize. On the right, adding a column of padding to `A` ensures that there is always one more element in each row than threads in a warp. Threads thus access different banks and can run in parallel.

`* TILE_DIM + thread_y + j]`, swapping the positions of the rows and columns. Finally, the result is written to `odata` in the same coalesced order that it was read from `idata`, completing the transpose for that tile. Every global access is coalesced and the strided steps are confined to shared memory, thereby reducing overall global access and increasing arithmetic intensity [166]. This general design pattern is common irrespective of the specific tile size we used: in practice, tile sizes are often chosen to fit precisely into on-chip memory.

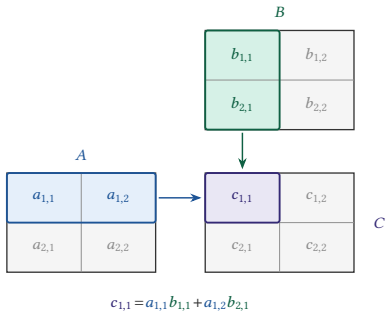
Unfortunately, this implementation still is not as efficient as one might anticipate due to memory bank conflicts. Recall that shared memory is divided into 32 *banks*, each of which can be accessed in parallel; consecutive words are assigned to consecutive banks. When our tile is initially written to shared memory, each thread writes to consecutive elements of the tile, so no bank conflicts occur. However, on line 23, each thread accesses an element staggered 32 places from the previous thread, so all warp threads access the same bank. All 32 threads in the warp must thus access their data serially, mitigating the benefits of using shared memory.

```
1  __global__ void transpose(float *odata, const float *idata) {
2    __shared__ float tile[TILE_DIM * (TILE_DIM + 1)];
3
4    const int block_x = blockIdx.x % (SIZE / TILE_DIM);
5    const int block_y = blockIdx.x / (SIZE / TILE_DIM);
6
7    const int thread_x = threadIdx.x % TILE_DIM;
8    const int thread_y = threadIdx.x / TILE_DIM;
9
10   int col = block_x * TILE_DIM + thread_x;
11   int row = block_y * TILE_DIM + thread_y;
12
13   for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS)
14     tile[(thread_y + j) * (TILE_DIM + 1) + thread_x] =
15       idata[(row + j) * SIZE + col];
16
17   __syncthreads();
18
19   col = block_y * TILE_DIM + thread_x; // transpose block offset
20   row = block_x * TILE_DIM + thread_y;
21
22   for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS)
23     odata[(row + j) * SIZE + col] =
24       tile[thread_x * (TILE_DIM + 1) + thread_y + j];
25 }
```

Fig. 10. A coalesced matrix transpose with no bank conflicts in CUDA.

Luckily, this issue can be resolved in a fairly simple way: we can simply add one extra element of "padding" to each row of our tile. If we increase our row size from 32 to 33, then the access pattern on line 23 will be staggered by 33 instead of 32: thread 0 accesses element 0, thread 1 accesses element 33, thread 2 accesses element 66, etc. Since 33 is coprime with 32, so thread 0 accesses bank 0, thread 1 accesses bank 1 (since 33 modulo 32 is 1), and so on. This technique is visually illustrated in Figure 9, and the resulting kernel appears in Figure 10.

2.3.2 Optimizing Matrix Multiplication. We have discussed optimizing matrix transpose in detail, but this kernel is quite simple in comparison to many others commonly run on GPUs. To get a sense of what these more complex kernels are like, we'll discuss the process of optimizing matrix multiplication, or "matmul", arguably the most fundamental operation in modern GPU programming. A thorough treatment is not feasible here, as high-performance matmul implementations, such as




```
__global__ void gemm(float *C,
                     const float *A,
                     const float *B,
                     float alpha, float beta) {
  const int block_x = blockIdx.x % (SIZE / TILE_DIM);
  const int block_y = blockIdx.x / (SIZE / TILE_DIM);
  const int thread_x = threadIdx.x % TILE_DIM;
  const int thread_y = threadIdx.x / TILE_DIM;

  const int col = block_x * TILE_DIM + thread_x;
  const int row = block_y * TILE_DIM + thread_y;

  if (row < SIZE && col < SIZE) {
    float sum = 0.0f;
    for (int k = 0; k < SIZE; k++) {
      sum += A[row * SIZE + k] * B[k * SIZE + col];
    }
    const int i = row * SIZE + col;
    C[i] = alpha * sum + beta * C[i];
  }
}
```

Fig. 11. Left: a graphic showing how rows and columns of the input matrices combine to produce the output matrix. Right: a naïve GEMM in CUDA. Note that a simple matmul is a special case of GEMM where $\alpha = 1, \beta = 0$.

those found in cuBLAS [242], may span hundreds of lines of code and rely on complicated techniques; instead, our goal is to continue building intuition for some of the main techniques in GPU optimization.

Like transpose, matrix multiplication is a simple algorithm: it takes two input matrices and produces a single output matrix, the elements of which are the dot product of each row of the first input with each column of the second. In graphics, matrix multiplication may be used to rotate a 3D object; in machine learning, the second matrix often holds weights associated with a neural network, leading some input values to have a greater influence on the output than others. General matrix multiplication (GEMM)[3] is a standard form of matrix multiplication that computes $C = \alpha AB + \beta C$, where A, B, and C are matrices and α and β are scalars. Figure 11 shows the process of producing an element of the output matrix, along with a naïve implementation of GEMM.

Unsurprisingly, the GEMM implementation in Figure 11 is inefficient, achieving barely 1% of cuBLAS's performance [39]. Some of its shortcomings resemble those in our naïve transpose, but matmul presents an additional opportunity for data reuse. Namely, in an $n \times n$ matmul, each input value is used in n different dot products, whereas each value in transpose is single-use. In this case, then, tiling the input matrices—shown in Figure 12—improves performance by increasing the likelihood that reused data are served from cache rather than global DRAM.

```
__global__ void tiledGemm(float *C,
                          const float *A,
                          const float *B,
                          float alpha, float beta) {
  const int block_x = blockIdx.x % (SIZE / TILE_DIM);
  const int block_y = blockIdx.x / (SIZE / TILE_DIM);

  const int thread_x = threadIdx.x % TILE_DIM;
  const int thread_y = threadIdx.x / TILE_DIM;

  const int col = block_x * TILE_DIM + thread_x;
  const int row = block_y * TILE_DIM + thread_y;

  float sum = 0.0f;
  for (int tile_k = 0; tile_k < SIZE; tile_k += TILE_DIM)
    for (int k = tile_k; k < tile_k + TILE_DIM; k++)
      sum += A[row * SIZE + k] * B[k * SIZE + col];

  const int i = row * SIZE + col;
  C[i] = alpha * sum + beta * C[i];
}
```

Fig. 12. A tiled GEMM implementation in CUDA (assuming `SIZE` is divisible by `TILE_DIM`).

While tiling is likely to improve cache locality in practice, it is not guaranteed; storing the tiles in shared memory, as we did in transpose, provides certainty. A simple approach is to first load a pair of tiles—one from each matrix—into shared memory, then perform the relevant arithmetic. However, these operations cannot begin until both tiles have loaded fully, and all arithmetic must complete before the next pair of tiles are loaded. Since GPUs use different hardware units for computation and data movement, this approach leaves performance on the table.

We can regain this lost performance via *double buffering*, a technique in which two pairs of tiles—four in total—are maintained in shared memory. After the first pair of tiles is loaded, the compute

[3]When performed over single-precision floating-point numbers, GEMM is often also called SGEMM.

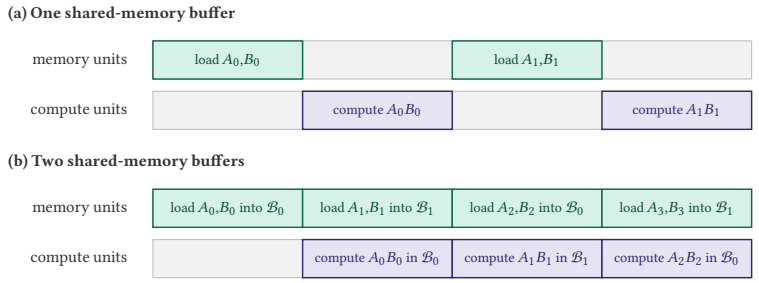


Fig. 13. Top: one shared-memory buffer means that either memory units or compute units are idle. Bottom: double buffering ensures that the hardware is always busy. $\mathcal{B}_0$ indicates buffer 0; $\mathcal{B}_1$ is buffer 1.

units begin calculating the dot product for that pair; simultaneously, the memory units load the next pair of tiles. By the time the first dot product is complete, the second load will have finished, and the compute units can begin work on that while the memory units again load the next tile. This process, illustrated in Figure 13, ensures that the hardware is always busy.

Double buffering is but one of many complex ways to optimize matrix multiplication, and we will not explore all possible optimizations here. We direct interested readers to Boehm [39]'s matmul optimization worklog and Hijma et al. [143]'s survey for further study.

2.4 Main Themes of GPU Programming

Up to this point we have been discussing the SIMT model and how one might write kernels in it, but SIMT has not always been the dominant mode of GPU programming, and it is unlikely to remain so forever. Accordingly, we now broaden our view beyond SIMT and identify three main themes of GPU programming that transcend the details of its hardware and software model.

Theme I: Slow Code is Wrong

Performance is an important concern in any domain, but it often takes a back seat to correctness. High-level languages and programming patterns, for example, frequently rely on the assumption that making one's code easier to read, write, or analyze is worth a non-trivial performance cost. Whether this trade-off is worth it depends on the application; one might prefer a language with explicit memory management for implementing an operating system, for example, but would likely use a garbage-collected language for a website. GPU applications, however, almost universally decline this trade—**performance is a first-order concern, in many cases on par with correctness.**

One reason for this is that, despite their conceptual similarity, an individual SM is significantly less powerful than a CPU core. When designing hardware to process massively data-parallel workloads, one tends to see the best performance-improvement-to-cost ratio from increasing the number of parallel processors on a chip, rather than making existing processors faster [276]. For this reason, many hardware optimizations like branch prediction, memory pre-fetching, and out-of-order execution that we take for granted on the CPU are entirely absent on GPUs, the chip space and silicon they would require instead being used to house more cores. As a result, optimizing a kernel can frequently improve its throughput by multiple orders of magnitude [278]; as we noted in Section 2.3.2, an unoptimized GEMM, for example, barely achieves 1% of the throughput of an optimized one [39].

Another reason is that GPUs are expensive specialty equipment, are finely tuned for specific, performance-sensitive use cases like graphics, deep learning, and scientific computing, and are unlikely to be used for other purposes. In other words, developers that don't care about performance are unlikely to target a GPU in the first place. Many GPUs are also rented by their users [13, 114], who pay per unit of time or compute; slow code thereby incurs a direct monetary cost to its author or their employer, further disincentivizing GPU usage for performance-insensitive tasks.

GPU programmers thus care deeply about performance and are rarely willing to trade it for other benefits like abstraction or ease of analysis—this care has shaped the languages and programming paradigms that predominated throughout the GPU's history. While *mass-market* [311] DSLs like PyTorch [261] and TensorFlow [2] can be popular in specific cases, low-level languages like CUDA remain the default choice for general-purpose kernel implementation [22]. GPGPU languages that do see wide use typically sacrifice things like expressiveness [76, 331] or simplicity [229, 278] rather than performance. Numerous performance engineering worklogs [39, 316], educational materials [87, 130, 132, 133], and microbenchmarking research papers [3, 161, 163, 199, 209] also evince the premium that GPU programmers place on performance.

Theme II: Specification is Easy, Optimization is Hard

As we saw in Section 2.3, kernels like matrix transpose or multiply have straightforward naïve descriptions but quickly balloon in complexity as we start to optimize them. These are instances of a more general pattern: **most kernels are simple to describe and specify but complex to implement efficiently.** As an illustrative example, a popular tutorial [39] on optimizing GEMM uses only 14 lines of CUDA for the naïve kernel, but 213 lines for its most optimized version. NVIDIA's cuBLAS [242] library for linear algebra operations—the standard benchmark for GEMM performance—is over 100MB in size and contains over 5000 implementations of GEMM, the best of which is selected at run time based on the characteristics of the matrices being multiplied [291].

As we discuss in greater detail in Sections 3 and 4, a common theme in GPU programming research is building better tools for managing this complexity, balancing the desire for elegant abstractions with the need for efficiency. Low-level languages like CUDA give programmers a great deal of control over how programs are mapped to hardware, but in turn place a commensurate burden on programmers to perform that mapping efficiently. Kernel performance in these languages is more often limited by poor implementation than the maximum throughput of hardware [61]—that is, most kernels do not reach the roofline.

Higher-level approaches, meanwhile, must contend with the fact that the GPU optimization space is vast, and few optimizations are applicable in all circumstances. A simple program transformation like loop unrolling, for example, can have dramatically different effects on performance depending on the kernel in question [26], as it decreases the number of instructions necessary to execute a loop but increases *register pressure*—the number of registers in use by each thread. This can result in lower maximum occupancy on SMs, since each thread now requires more space devoted to storing its state [338]. Whether this is a worthwhile trade depends on the performance characteristics of the kernel in question: compute-bound kernels benefit from faster loop execution and will not mind the increased register pressure, while memory-bound kernels will suffer from reduced occupancy.

Loop unrolling is but one example of an optimization whose usefulness varies from kernel to kernel; in general, general-purpose optimizing compilers for GPUs struggle to find and apply the appropriate optimizations in all circumstances [127]. Instead, GPU compilers often limit themselves to a particular domain with well-understood performance characteristics—like graphics [259, 272], tensor algebra [66, 174, 341, 356], or deep learning [62, 185, 286]—or leave much of the decision-making about optimization in the hands of the programmer [128, 153, 277, 357]. Those that do try to optimize general-purpose code frequently rely on *autotuning* [347] or *iterative compilation* [38] to search for the best results [126, 169, 278]. These optimization strategies—popularized by the ATLAS linear algebra compiler [68]—try various optimizations with different parameters, evaluating the resulting programs to find the ideal set of optimizations to apply for best performance. While useful, these strategies are limited by which optimizations the compiler knows about and so are usually paired with other methods to identify potentially useful optimizations [127].

This theme is more than just challenging optimization, however: the other side of the trade-off is that kernels' simple logic often makes them quite amenable to static analysis. For example, many kernels (up to 60% [203] or 70% [191]) feature an implicit separation between control and data, wherein a kernel's control flow and the addresses of its memory accesses are entirely independent of its input. Throughout this survey we will refer to this property as "control-data independence", and it is a fairly common simplifying assumption when writing, analyzing, and optimizing GPU programs that one can reliably distinguish values carrying control (also called *potential* [226] or *integrity* [191] in some literature) information from those that carry input data [153]. We can see control-data independence in practice in the examples in Section 2.3: all the access expressions into the input or output matrices or the shared memory buffers depend only on static constants and thread indices, making them more amenable to tracking by type systems or compiler analyses. However, the validity of this assumption is highly domain-dependent; in tensor algebra and graphics it is applicable very often, while in applications like chemical process modeling it is broken frequently [28].

Theme III: Hardware is Rapidly, Visibly Evolving

Another consequence of GPUs' thin abstraction layer is that programmers are significantly more exposed to hardware changes than they would be on a CPU. The specifics of CPU hardware change frequently as technology improves, but thanks to the hardware optimizations CPUs perform, they can present programmers with robust abstractions and a programming model that has not fundamentally changed since the 1970s [276]. By contrast, every generation of NVIDIA GPU since the Volta [233] microarchitecture has changed its programming model drastically. We will discuss the precise details of these changes in Section 2.6 and focus here on their impact instead.

The introduction of Tensor Cores [248] in Volta and asynchronous data movement in the Ampere [234] and Hopper [235] microarchitectures completely changed how performant GPU kernels were written. Kernels that were efficient on one generation became useless on the next, with migration commonly requiring dramatic restructuring of code. As an extreme example, Flash Attention [83, 84, 294, 363]—a popular machine learning kernel—required an entirely new algorithm to achieve good performance on Hopper [294], resulting in over a year of lag time between Hopper's release and the development of a usable attention kernel [302]. The disruption caused by hardware evolution affects low- and high-level languages both; kernels that exercise direct control over hardware features must be rewritten for new architectures, while those written in more hardware-agnostic languages must wait for the compiler, library, and tooling ecosystem to catch up before they can be used. **One of the central themes of modern GPU language research has thus been the question of who among the compiler and the programmer should be responsible for the performance of code**: relying too much on the compiler makes languages brittle as hardware evolves [355], while leaning too much on the programmer makes them challenging and labor-intensive to use [119, 153].

Along with differences between generations of GPUs from the same manufacturer, programmers are also exposed to variance in design across manufacturers. While AMD's TeraScale [148] and NVIDIA's Tesla [206] designs were originally quite similar, for example, their more recent chips have diverged dramatically; AMD's newest chip generations [6, 7] feature fewer fixed-function units than NVIDIA's but come with an L3 cache layer for programmers to manage that NVIDIA chips do not have. AMD's Matrix Cores [301] and Intel's XMX [155] engines likewise began quite similarly to NVIDIA's Tensor Cores, but their programming models on current hardware have become distinct.

Architectural variation also has consequences for tools that analyze the behavior of GPU kernels. It is common for such tools to make simplifying assumptions about the programs they target in order to make their analyses more tractable, but these assumptions are frequently invalidated by the release of new microarchitectures [33, 34, 361] or are only applicable to certain manufacturers. Static analysis tools are thus often forced to specialize on a handful of particular hardware targets.

2.5 Major Challenges of GPU Programming

Having discussed the chief priorities and concerns of GPU programmers, we now turn our attention to the specific difficulties they face, identifying two classes of challenge: *implementation challenges* and *engineering challenges*. Implementation challenges are small-scope difficulties encountered when writing or reasoning about code, like debugging errors or optimizing underperforming kernels. Engineering challenges are larger in scope, encompassing questions like how to balance performance with portability or how to adapt kernels and pipelines to newly released hardware.

2.5.1 Implementation Challenges. Yang et al. [360] and van den Haak et al. [340] analyzed posts on Stack Overflow [305] by topic and content, identifying common challenges faced by GPU programmers; due to the nature of the data set, most of these challenges are small-scope. Many of these—like poorly documented APIs, unexpected run-time errors, or confusion about data type conversion—overlap with issues common on the CPU, suggesting that the usual approaches there, like expressive type systems or improved tooling, would also find success in the GPU space. However, other classes of problems are unique to GPUs and have little precedent in other domains [99].

Debugging correctness errors in GPU kernels is especially challenging, for instance, because error reporting and handling is unusually complex. Kernels execute asynchronously with respect to the host that launched them, and so handling errors from the host requires blocking until the kernel finishes—expensive and typically not an option in production [131]. Identifying the source of errors thus becomes more challenging when they occur in production, and static bug-finding methods are commensurately more valuable. Common bugs include the usual memory concerns like out-of-bounds or uninitialized access—made more challenging by the multi-level memory hierarchy—along with more GPU-specific concerns like undefined behavior arising from barrier or warp divergence. As an example, if we had forgotten the `__syncthreads()` on line 16 of Figure 8, it would be possible for a thread to read uninitialized data out of `tile` on line 22 before the corresponding thread had written the correct data to it on line 14. To make matters worse, the precise behavior of GPU hardware is often underspecified, resulting in frequent confusion about details like floating-point rounding modes [115, 202, 339], performance characteristics [161, 177, 199], and memory consistency [10, 303, 333]; extensive effort is often required to nail down GPU specifications.

Beyond correctness, developers frequently have difficulty identifying both why their kernels underperform and how to fix performance issues once identified [360]; in particular, they often struggle with coalescing global memory accesses, eliminating bank conflicts in shared memory accesses, efficiently buffering data in shared memory, and avoiding warp divergence, as well as configuration questions like how many threads or blocks to launch a kernel with, how to divide work among threads (i.e., tiling), or how many resources to devote to different tasks in a pipelined kernel.

In addition to such broadly applicable optimizations, developers must also grapple with optimization techniques that are particular to certain kernels or domains. *Swizzling*, for example, involves mapping data and computation onto hardware resources using approaches other than the usual "consecutive threads get consecutive data". So, for example, while a typical kernel would distribute 64 bytes over a warp by assigning the first thread in the warp the first 2 bytes, the second thread the next 2, and so on, a swizzled kernel might assign bytes 0 and 32 to the first thread, bytes 1 and 33 to the second, and so forth. For certain kernels in domains like cryptography [30] and linear algebra [54], an appropriate choice of swizzle can improve data locality or result in superior shared-memory access patterns compared to a naïve mapping.

Another important optimization technique is *warp specialization*, or writing kernels where different warps are assigned different tasks. Warp specialization has its roots in tools like CudaDMA [27], an API for improving kernels' memory bandwidth, and Singe [28], a domain-specific compiler for combustion chemistry. Chemistry kernels are unusual because they frequently lack the usual

control-data independence property; the "control flow" of a chemical reaction often depends on the quantity and configuration of the reagents involved, and so the input data to a chemical simulation can flow into branch guards and array accesses. Additionally, data about reaction rates is required to compute the results of reactions, but there is far too much such data to store in shared memory; it must be loaded on-demand as each stage of the reaction occurs. Singe's solution to this problem is to specialize the warps undertaking its computations: producer warps are responsible for loading reaction rate data from global memory and putting it into shared before it is required by the consumer warps that actually perform the reaction computations, reducing the amount of time that these warps must spend waiting on global memory loads. Producer-consumer specialization has proven effective for a variety of kernels—achieving up to a 4X performance improvement over usual data-parallel approaches in some cases [295]—and has only grown in importance with the evolution of GPU hardware, as we will see in Section 2.6.

At a higher level of granularity than individual kernels, developers also must optimize the performance of entire pipelines, in which multiple kernels are run in sequence to apply multiple transformations to data. One particular optimization of interest is *kernel fusion*, in which multiple sequential kernels are combined into a single kernel with the same behavior but improved data locality, lower loop overhead, and fewer expensive data transfers between host and GPU. Kernel fusion has its origins in research on stream fusion [81], further cementing the connection between GPUs and stream programming. Fusion is complex, however, with many possible implementations, and developers must contend not only with how to best fuse their kernels but also with whether to fuse them at all; like most GPU optimizations, fusion can both improve or degrade performance depending on the kernels in question. In particular, fusing high-occupancy kernels with low-occupancy ones can limit the latency hiding ability of the resulting fused kernel, lowering overall performance. Consider an example where two kernels—call them k_1 and k_2—that compute large outputs are fused with another (k_3) that reduces over the results of k_1 and k_2 to produce a scalar. Both k_1 and k_2 must be computed before k_3 can run, but without fusion, the result of k_1 can be stored off-chip while k_2 is being evaluated, allowing both kernels to achieve high occupancy. If all three are fused, however, k_1's result must be held in memory while k_2 is running, potentially reducing occupancy by increasing memory usage [106]. Whether avoiding CPU-GPU data transfer outweighs the reduced occupancy depends on the input, so heuristic-based optimizers can struggle to know whether to fuse or not.

Another concern worth highlighting is management of the boundary between the GPU and its host CPU—in particular, efficient and correct movement of data between them. There are two main ways to perform this data movement: using a unified address space between the two machines (such as CUDA's Unified Memory [256]), which is simpler but can have worse performance [177], or manually sending data back and forth. In the former case, programmers must reckon with the fact that the CPU and GPU have different memory models and must therefore take care to use design patterns that respect a compound consistency model that unifies the machines [365]. In the latter case, most languages require programmers to manually track the provenance of pointers; attempting to dereference a pointer to CPU memory from the GPU is a common source of bugs. Developers must also respect the implicit assumptions made by kernels about how they will be launched; improperly configured kernels can suffer from poor performance or run-time errors, but few languages feature ways to manage configuration requirements [184].

Finally, programmers also often struggle with translating CPU into GPU code or understanding why such translations do not result in the expected performance gains. These difficulties manifest at many granularities, from low-level concerns like implementing a specific serial loop nest as a parallel loop in the SIMT model to high-level questions about how to parallelize entire algorithms. Many of these challenges are highly domain-specific, as the various application areas of GPU programming all have different performance characteristics—no one-size-fits-all solution exists.

2.5.2 Engineering Challenges. Beyond questions of implementation, developers must also contend with larger software engineering concerns. One common problem is verifying that the behavior of an optimized kernel is correct and matches its original naïve implementation—as we saw in Section 2.3, these two versions of the same kernel can look quite different. Formal methods tools can help with this task, but as we shall see in Section 4, these can be difficult to use in their own right, and few support all the advanced GPU features that are typically needed to achieve optimal performance.

Another important question is how to strike the right balance of portability and performance for a kernel or pipeline. Because of GPUs' thin abstraction layer, these two factors are frequently at odds; optimizing a kernel typically requires specializing it to a particular architecture. Basic properties of the programming model—like the number of threads in a warp, for example—differ across brands of GPUs, so naïvely transporting code between them results in subpar performance. Portability is not only a performance concern either; different GPUs often disagree on critical details like floating-point rounding modes and the precision of numerical operations, so identical kernels can yield subtly different results on different architectures [115].

Manufacturer differences are not the only dimension of portability programmers must consider; they must also be prepared to migrate kernels forward or backward between different generations of GPUs. As discussed in Theme III, GPU hardware is constantly evolving, and a kernel that is efficient on one chip generation may be unacceptably slow on another. Sometimes these changes are unavoidable, as in the case of the Flash Attention kernel that required an entirely new algorithm for the release of Hopper [294], but in other cases kernels can be migrated; the question is how to do so effectively, and whether the migration can be done automatically.

As GPUs increasingly handle data more sensitive than graphics workloads (zero-knowledge proofs, for example [211]), an emerging new challenge has been the security and privacy of GPU computation. GPUs' unique architectures expose them to different kinds of vulnerabilities than are covered by traditional research [342]; for example, GPU processes are not isolated from one another and can peek into each other's memory [269]. Yandrofski et al. [359], meanwhile, describe a procedure for designing adversarial kernels that starve others for resources when concurrently executing on a single device. This topic is too wide to cover in detail here—we direct interested readers to Mittal et al. [221] and Wang and Oswald [345]'s surveys for further information—but we note that programming languages and formal-methods-based approaches to these problems are scarce [138].

By far the largest challenge in recent years, however, has been the shift towards deep learning as the predominant mode of GPU programming. For most of the history of GPUs, graphics was the primary use case, and the hardware and programming model reflect this; as we have discussed, SMs were designed to efficiently implement shaders, and the SIMT model itself descends from stream processing [45, 228]. But, while these design decisions were a natural fit for programming the graphics pipeline, they may not necessarily be appropriate for workloads in other domains. Moreover, as GPUs have evolved to favor deep learning as a primary application, their hardware has changed in ways that have eroded the most fundamental assumptions of the SIMT model, leaving developers scrambling to reconcile their old programming models with the reality of new hardware [61, 140, 312, 325]. The poster child of this erosion is the Tensor Core.

2.6 Tensor Cores: Beyond SIMT

The release of the Volta [233] microarchitecture brought two changes to GPU hardware that would dramatically impact the SIMT programming model. The first, Independent Thread Scheduling [246], removed the hardware restriction that required all CUDA Cores managed by the same warp scheduler to execute synchronously. In SIMT terms, this means that threads within the same warp no longer need to execute in lockstep; divergence within a warp still degrades performance [351], but the cost is not nearly as large as it was prior to Volta. To allow the restoration of warp-synchronous behavior after

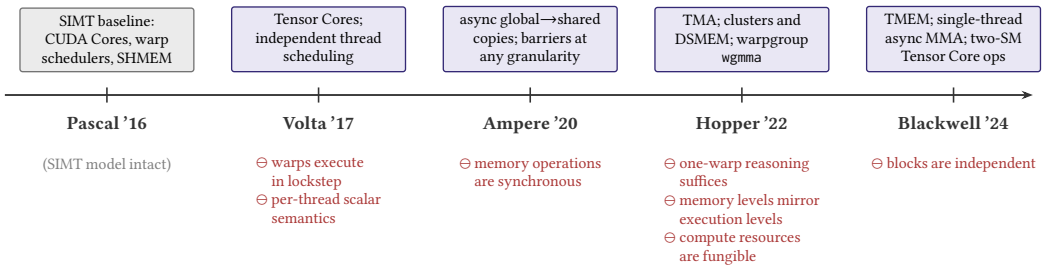


Fig. 14. Five generations of NVIDIA microarchitectures. Above the timeline, the major hardware capabilities each generation introduced; below, the SIMT assumptions that those capabilities *removed* (marked ⊖, in red).

divergent branching, Volta also added new barrier primitives to synchronize all the threads within a warp. Kernels written before Volta had not needed such synchronization, however, and became vulnerable to data races and barrier divergence on new hardware—a major backwards-incompatible change, with many more soon to follow, as Figure 14 summarizes.

Volta's second, more influential change was the introduction of *Tensor Cores* [248], which broke two fundamental assumptions of the SIMT model: that the behavior of a kernel can be specified at a per-thread granularity and that each thread maps to a CUDA Core on hardware. Tensor Cores are fixed-function units for fast matrix multiplication based on systolic arrays [182]; every warp scheduler on Volta GPUs has an associated pair of Tensor Cores [17], which can perform matrix multiplications in parallel with its associated CUDA Cores. The programming model for Tensor Cores requires them to be invoked *collectively* by groups of threads (quarter-warps—or *quad-pairs*—on Volta [35], but this would change with later architectures [251, 252]); if a Tensor Core is invoked with an incorrect number of threads, execution can hang or produce invalid output. With Tensor Cores, programmers must now reason at the level of groups of threads executing collectively, and threads within these groups can be concretely running on either a CUDA Core or a Tensor Core.

Unlike CUDA Cores, which operate on scalar values, Tensor Cores (as their name suggests) operate on tensors, requiring a fundamentally different programming model. Tensor Core inputs are small—8×4 and 4×8 on Volta [35] and up to 256×32 and 32×256 on later GPUs [161, 252]—but kernels often operate over matrices many times larger (or with more dimensions). Thus, tiling, previously one optimization among many, became a mandatory prerequisite for using the Cores; we illustrate the approach visually in Figure 15. Additionally, Tensor Cores expect their inputs to be laid out in specific ways—corresponding to low-level details of their underlying hardware—that are typically different from how tensors are arranged for most applications; rather than being contiguous in memory, inputs must be distributed across the threads invoking the Tensor Core [35]. Multiplying matrices thus became a process of swizzling them into tiles acceptable to the Cores, which then handled the results. Tiles, rather than scalars, had become the new fundamental units of computation.

Further compounding their challenging new programming model, the only way to directly program Tensor Cores on release was via the unstable `wmma` (Warp Matrix Multiply Accumulate) preview library [17]; APIs like CUTLASS [322] were developed later with simpler interfaces, but these do not (and cannot, in CUDA [22]) enforce any of the Tensor Cores' implicit requirements on cooperative execution, so their ergonomics remain lacking. Despite their complexity, however, the performance of Tensor Cores could not be ignored, and other manufacturers like AMD [301] and Intel [155] soon followed suit with their own systolic units to compete with NVIDIA's. Today, as much as 90% of a modern GPU's throughput capacity lies in these units [61], so programmers wishing to produce performant tensor kernels must learn to wield them effectively.

Another sea change for the SIMT model came with the release of the Ampere [234] microarchitecture. Ampere changed the details of Tensor Cores slightly from Volta, combining the previous two

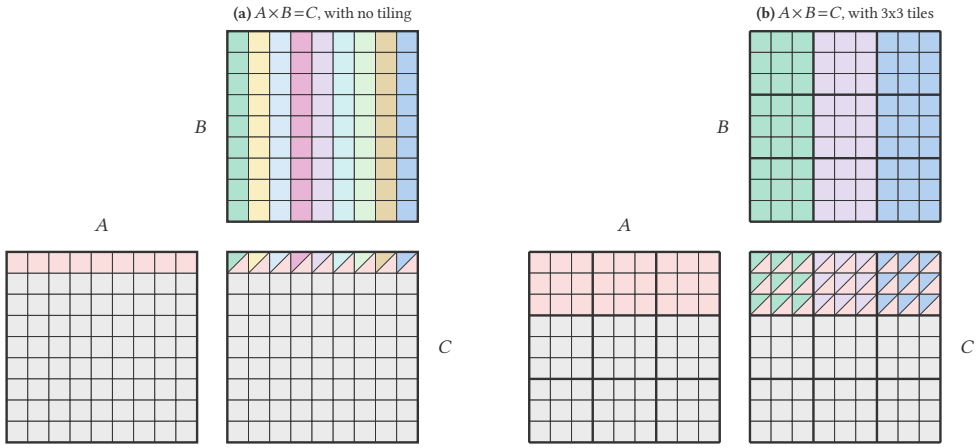


Fig. 15. Two approaches to matrix multiplication. In (a), threads compute C one scalar cell at a time; each cell is the sum of products of a cell-row in A and a cell-column in B. In (b), warps cooperate to compute C a tile at a time using Tensor Cores; one tile of C is the sum of products of a tile-row of A and a tile-column of B.

Tensor Cores per warp scheduler into one, larger Core, but the much more significant change was the introduction of hardware support for asynchronous copies between global and shared memory. Instead of requiring a full warp to request coalesced chunks of data from DRAM, blocking further execution of that warp until the data arrived, individual threads could now issue asynchronous requests for chunks from global memory and continue with other work until the request completed. In effect, the latency-hiding approach to mitigating the slow speed of memory operations that had predominated on earlier architectures was no longer sufficient; for maximum performance, one needed to make use of asynchronous data movement as well [199]. To better support asynchronous programming, Ampere also added synchronization barriers at arbitrary granularity, rather than just at the level of warps and blocks.

Asynchronous programming grew in importance as Tensor Core designs continued to evolve, allowing them to process more, larger matrices faster. By the fourth generation of Tensor Cores, they had become so fast that the primary bottleneck for performance of tensor kernels was no longer the speed of the Cores themselves, but rather the ability of the rest of the SM to keep them fed. So, with the Hopper [235] generation of microarchitectures, NVIDIA equipped each SM with a *Tensor Memory Accelerator* (TMA) [255], a fixed-function hardware unit designed to satisfy the Tensor Cores' insatiable appetite for data. The TMA expands upon the asynchronous data movement introduced in Ampere by issuing more, larger asynchronous transfers between global and shared, thereby freeing up the CUDA Cores to do other work and accelerating the flow of data into the Tensor Cores [235, 357]. In a possibly surprising knock-on effect, issuing fewer, larger requests via the TMA also results in less addressing math, thereby reducing register pressure [171] and further improving efficiency by allowing more blocks to be scheduled on each SM.

At the same time, Hopper decoupled the implicit relationship between the execution and memory hierarchies' levels by giving programmers the ability to organize blocks into *clusters* via the introduction of *distributed shared memory* (DSMEM), a way for separate SMs to access each other's SRAM. DSMEM obviated the need for inter-block communication to pass through global memory, and functionally added a new level to the SIMT execution hierarchy (clusters of blocks) that lay between the block and the grid but lacked its own dedicated level of the memory hierarchy.

Hopper also evolved the programming model of Tensor Cores again by changing the granularity of their execution. On Ampere, Tensor Core operations are invoked by individual warps via a `wmma`

[253] instruction, while on Hopper, to support larger matrix sizes, they are invoked by a *warpgroup*—a collection of four cooperatively executing consecutive warps. Volta had previously raised the number of warp schedulers per SM from two to four, so a warpgroup-level `wgmma` (Warpgroup Matrix Multiply Accumulate) [251] effectively treats the four physical Tensor Cores on each SM as a single, extra-large logical Core [357]. This invalidated another, subtle property of the SIMT model: when reasoning about the correctness (rather than the performance) of kernels, it had hitherto been possible to make the simplifying assumption that all thread blocks consisted of just one warp [33, 34, 361]. In the presence of warpgroup-level operations, however, such an assumption no longer holds; analysis of kernel behavior post-Hopper must contend with the full complexity of inter-warp interactions.

The increasing heterogeneity of SMs on Hopper encouraged the paradigm of warp-specialized programming, in particular producer-consumer pipelines wherein one producer warp would manage the TMA while consumer warps issued work to Tensor Cores. As we discussed in Section 2.5.1, warp specialization predated Hopper, but the presence of so many heterogeneous hardware units operating in tandem on Hopper's SMs made it practically mandatory for maximum performance [357]. It was so important, in fact, that support for warp specialization was introduced at the hardware level [61], violating yet another SIMT assumption that all compute resources are fungible.

NVIDIA continued to heterogenize the SM in its Blackwell [237] microarchitecture by introducing another kind of memory, *Tensor Memory* (TMEM), designed to give Tensor Cores a low-latency location in which to accumulate output [161]. Each Blackwell SM shares one bank of Tensor Memory, introducing a new form of memory outside the usual SIMT memory hierarchy and a separate set of instructions for interfacing with it, issued cooperatively by a warp [252]. TMEM further encourages a pipelined programming style; full utilization of SMs on Blackwell requires performing a matrix multiplication on one tile via its Tensor Cores, while simultaneously loading in the next tile via the TMA and writing the previous one to output from Tensor Memory [316].

The performance characteristics of Tensor Memory are not yet fully understood, and whether TMEM is faster than the traditional memory hierarchy depends both on the size of the matrices being multiplied and whether the multiplication is part of a large pipeline or is just a "single-shot" operation. Microbenchmarks indicate that TMEM achieves peak throughput when processing matrices of size 64×64 [161], despite the fact that Blackwell's Tensor Cores themselves support operations with outputs of size 256×256 [252]; efficient kernels targeting Blackwell must thus vary the degree to which they rely on TMEM depending on their specific dimensions and data usage characteristics, resulting in different code from case to case.

Blackwell also once again changed the programming model of Tensor Cores, moving from a bulk-synchronous warpgroup-cooperative mode to a single-thread semantics where Tensor Core operations are issued asynchronously by one thread in a block and results are accumulated into Tensor Memory. TMEM is shared among all the threads in a block, however, and the programming model for Blackwell's Tensor Cores likewise requires operations to be processed at block granularity; multiple warps or warpgroups within the same block cannot issue separate Tensor Core instructions as on previous architectures. Despite Tensor Cores no longer requiring precisely four warps in order to execute, the need for warp-specialized code (i.e., multiple warps within a block splitting work heterogeneously) to achieve maximum throughput means that kernel analysis tools and compilers cannot return to their one-warp-per-block simplifying assumptions. At the same time, it became possible for two SMs to collaborate to perform a multi-SM Tensor Core operation, requiring two threads in adjacent blocks in a cluster to synchronously issue the operation [252]—it had not previously been possible for threads in different blocks to cooperate in this way. These dramatic changes to the programming model, along with the fact that Hopper GPUs still see extensive use alongside Blackwells [316], mean that kernels that wish to be portable between architectures require radically different implementations or compilation procedures for each target.

At the end of this sequence of architectural changes—two years into the lifecycle of Blackwell—the programming model for GPUs looks vastly different than it did pre-Volta. Many of SIMT's most basic assumptions have been broken, and some [61, 140, 312, 325] have argued that the SIMT paradigm is fundamentally unsuited to the new, heterogeneous, asynchronous reality of GPU programming. Accordingly, we must wonder: what programming model will arise to replace it?

3 A Genealogy of GPU Programming Languages

To design the languages that will support the future of GPUs, it is helpful to understand the languages that were the foundation of their past. In this section, we present a genealogy of programming paradigms for GPUs, tracing the evolution of their software model in tandem with the hardware. Figure 16 depicts this genealogy—starting from early shading DSLs that predate GPUs as we understand them today and continuing through to languages for modern hardware—while later figures in this section present zoomed-in views of its overall graph. We identify languages as inheriting from one another when a later language uses or develops ideas originally implemented by an earlier one; in many cases this relationship is well known (as in the case of Brook [46] and CUDA [229], for example [45, 228]), while in others we have identified conceptual connections that have not been previously documented (e.g., the Real-Time Shading Language [272] and user-schedulable languages).

To make this survey tractable, we limit discussion in this section according to two criteria. First, languages' original or primary compilation targets should be GPUs. Many high-performance or scientific computing DSLs like OpenMP [82], OpenACC [349], Legion [29], Chapel [49], Realm [9], Sequoia [101], and MATLAB [327] can be compiled to GPUs but were originally designed for other architectures—full exposition of these languages is beyond the scope of this survey. Likewise, general-purpose languages like Java [118, 136, 137, 157, 192, 270, 358, 364] and Lime [19, 20, 91] that have been extended to target GPUs also fall outside the scope of this survey. Second, languages should be novel or make influential contributions to the GPU ecosystem. Many DSLs exist besides those we describe here—including, but certainly not limited to, Opt [89] for least-squares optimization, OptiML [311] and Latte [335] for machine learning, StencilGen [282] for general stencil computations, and Spark [108], Forma [281], Gator [112], and CoolerSpace [60] for graphics programming—but we focus on those of particular interest to a programming languages audience.

3.1 Shading Languages

Our genealogy begins with two early graphics frameworks—**shade trees** [76] and **image synthesizers** [267]—from which all modern shading languages descend [108]. These languages predate the modern GPU, but they embody a dichotomy between abstraction and control that will reappear in various guises throughout our discussion of its programming languages.

The shade trees framework allows users to describe the structure of their shader declaratively in terms of a data-flow graph, or shade tree. Compound shaders, such as lighting or atmosphere shaders, are described by multiple individual shade trees, which are joined (or "grafted") together by the framework's compiler into a combined tree. Shade trees' abstraction is simple and modular; shaders don't need to know each other's implementation details to be grafted together. The trade-off, however, is expressiveness—shade trees do not support any kind of non-linear control flow or mutable state.

Image synthesizers, meanwhile, are a procedural framework; shaders are imperative programs and can handle the kinds of complex state and control that shade trees cannot. Consequently, however, they impose less structure on shading programs and must rely on weaker abstractions—primarily function signatures at procedure boundaries. Image synthesizers thus lack shade trees' modularity, and building composite shaders from smaller ones typically requires interprocedural reasoning.

Shade trees and image synthesizers delimit a conceptual spectrum between abstraction and expressiveness that shaped the development of many early graphics languages; languages in this era

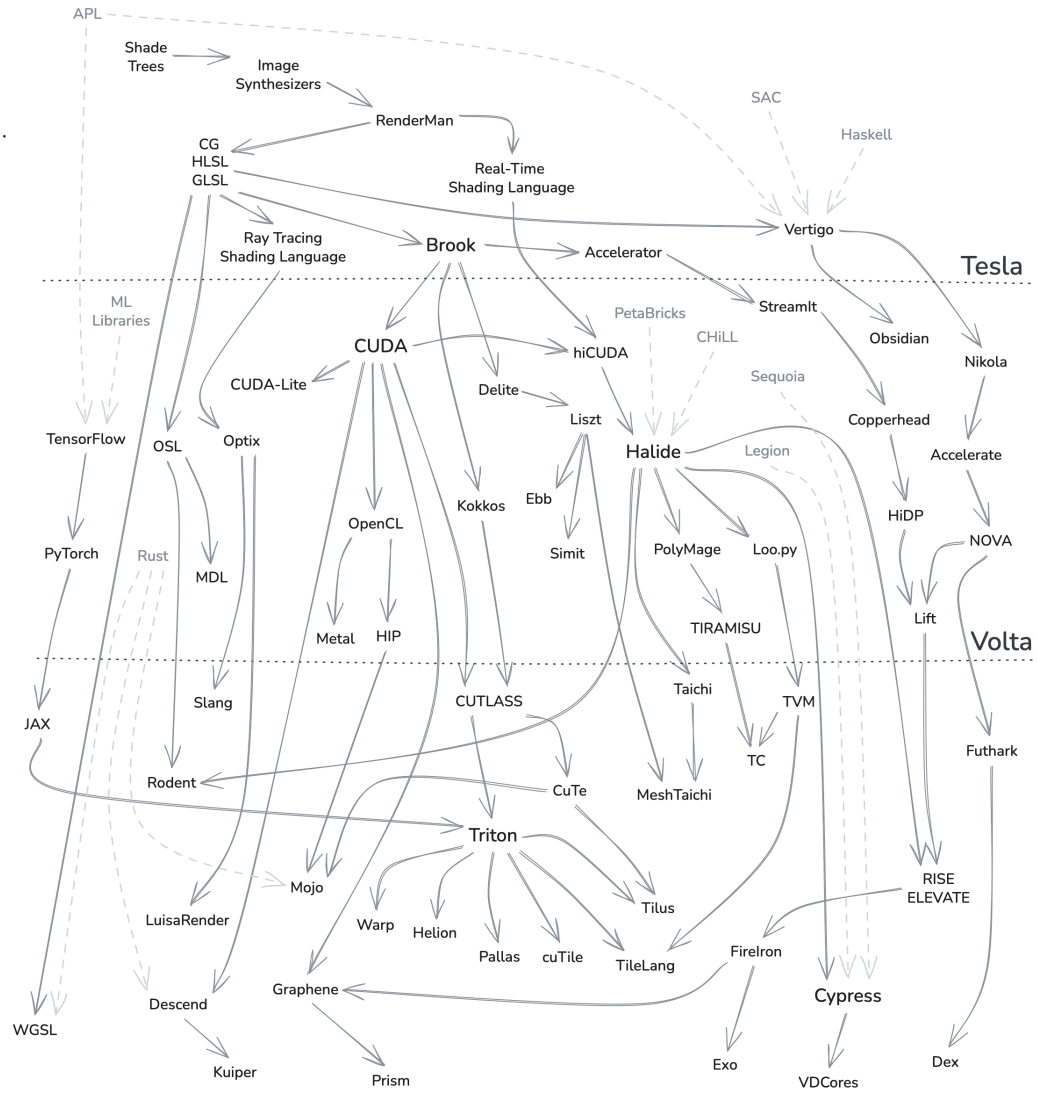


Fig. 16. A genealogy of programming languages for the GPU. Languages are connected by arrows when one inherits its ideas, theory, or implementation from another. Dashed arrows indicate connections from a language not originally designed for GPUs. The languages are (roughly) temporally sorted, with the horizontal dotted lines indicating the release of major new architecture designs. Throughout the rest of this section we present a number of sub-genealogies, highlighting particular language families in more detail.

frequently described themselves in relation to these two poles of design. Pixar's **RenderMan** [129] language, for example, positioned itself towards the center of this spectrum; RenderMan was based on C, and was therefore procedural like image synthesizers, but also imposed additional requirements on program structure to recover some of shade trees' modularity. In particular, RenderMan shaders are specified as classes, in the object-oriented sense, and common graphics objects like surfaces and lights have distinct types and operations. Shader composition occurs via specialized control operators, guaranteeing a uniform abstraction for building modular shaders.

RenderMan's compiler targeted a SIMD array-processing architecture but gave programmers the illusion of a sequential semantics, automatically parallelized by the compiler. Instead of the parallel nature of the hardware, RenderMan exposes the structure of the graphics pipeline by allowing programmers to directly describe how often values change during computation. Variables in RenderMan are decorated with *computation rates*: `Uniform` variables are constant during the lifetime of a shader (although they can differ between multiple instances of the same shader, and so are not necessarily constant for an entire program), while `Varying` variables change over the lifetime of a shader instance. These rates allow RenderMan to compute the values of expressions that rely only on `Uniform` variables when shaders are instantiated, rather than recomputing them on each application of the shader. Programmers had previously performed "scheduling optimizations" like this by hand, so reifying these details of the shading process as variable annotations represents an early example of graphics languages leveraging domain-specific information to produce faster code.

The **Real-Time Shading Language** (RTSL) [272] built upon the idea of computation rates by exposing even more of the structure of the graphics pipeline. In RTSL, developers write *pipeline shaders*, specifying high-level behavior declaratively with respect to the entire graphics pipeline, rather than individual components of it. Values in a shader can be annotated with *computation frequencies*, which are similar to RenderMan's computation rates but have finer granularity: instead of two rates for uniform and varying computations, frequencies directly describe where in the pipeline an operation occurs. The `perlight` annotation on line 2 of Figure 17 indicates that `NdotL` should be computed once for each light source, for example. RTSL's compiler uses these frequencies to decompose and map the full-pipeline shader into compositions of individual stages that could actually execute on a graphics pipeline, neatly separating the concerns of specifying programs' behavior and translating that behavior to hardware. This idea would prove enormously influential and would be further developed by a new line of research into *user-schedulable languages*, which we discuss in more detail in Section 3.6.

```
1 surface float4 lightmodel_diffuse (float4 ka, float4 kd) {
2     perlight float NdotL = max(0, dot(N,L));
3     return ka * Ca + integrate(kd * NdotL * Cl);
4 }
```

Fig. 17. A simple light model in RTSL, from Proudfoot et al. [272].

While RTSL sat between shade trees and RenderMan on the abstraction-control spectrum, other languages chose to move further in the direction of image synthesizers in order to support more applications beyond graphics. In contrast to RTSL's whole-pipeline approach, **Cg** [215], **GLSL** [287], and **HLSL** [220] adopted a *shader-per-stage* paradigm, wherein programmers would write individual shaders for each programmable stage of the graphics pipeline[4]. The primary benefit of this approach is expressiveness; these are fully featured programming languages that can express computations outside the graphics domain. The trade-off is that certain graphics operations (such as tessellating an image, for example) are not easily separable into distinct pipeline stages. For such cross-cutting operations, shaders must either implement redundant functionality across their stages or divide the implementation of these operations across the pipeline stages in ways that do not correspond to natural divisions of their logic [108].

In addition to graphics languages, Cg, GLSL, and HLSL also identified themselves as *streaming languages* and their programs as transformations over streams of input data. Their streaming paradigm served as inspiration for two different approaches to general-purpose GPU programming: Vertigo [97], which we discuss in Section 3.4, and Brook [46], which we discuss in Section 3.2.

Even after these branching approaches to GPGPU programming, research into shading languages continued, particularly on languages for rendering 3D images. One common technique for rendering is *ray tracing* [348], which computes the colors of a scene by simulating the paths of rays of light

[4]These languages are extremely similar; they are all derived from C, share a common history, and for our purposes can be considered the same [108].

emitted from an imaginary camera or eye. Unlike many other graphics operations, ray tracing does not always map cleanly onto the usual SIMT programming model; the paths taken by light rays are often affected by the materials and surfaces they encounter as they travel, so the control flow of a ray-tracing kernel is highly sensitive to its input data. Angled surfaces can also cause rays to take wildly different paths from their neighbors, making them hard to efficiently schedule on SIMD hardware, where neighboring threads must execute in lockstep. Accordingly, many consumer-grade variants of modern GPUs (i.e., those found in personal computers rather than industrial datacenters) come equipped with dedicated Ray Tracing Cores to accelerate rendering workloads [170].

An early ray-tracing language was the **Ray Tracing Shading Language** (confusingly also abbreviated RTSL) [260], which built on GLSL, adding Java-style classes and references along with a number of ray-tracing-specific features, like a dedicated construct to iterate over all the light sources in a scene. It also expanded upon RenderMan's computation rates, adding a handful of new variable qualifiers, and its compiler targeted early SIMD GPUs using an algorithm to automatically group rays with similar paths into *packets*, which could be executed together. Previously, ray-tracing programs performed packetization manually, forcing developers to reason about the parallel behavior of many rays, but RTSL allowed kernels to be written as if for a single ray, parallelizing automatically.

This single-ray model proved successful, and was adopted by many subsequent ray-tracing languages. The **OptiX** [259] DSL, for instance, built on RTSL and made the ray-tracing pipeline fully programmable using a multi-stage model, wherein developers would write a kernel for each of the possible events an individual ray could experience during its lifetime, such as encountering an object or the scene's background. OptiX then parallelizes these kernels over all the rays and light sources in a scene and invokes the appropriate ones as each ray traverses the scene. The GPU's structure and hierarchy, along with the specific algorithm used to compute scene traversal, are opaque to the user and are instead managed by OptiX's runtime. OptiX is compiled to PTX [250], CUDA's low-level bytecode, to run on GPUs. The **LuisaRender** [370] DSL later updated OptiX's programming model and design to make it more portable and to make use of new hardware like Ray Tracing Cores.

In competition with GLSL, HLSL also modernized itself with features like ray-tracing support [219] and a successor DSL called **Slang** [139], which added new language features like generics, classes, and interfaces that RTSL and OptiX had previously brought to GLSL. Others, however, elected to diverge from the existing approaches to shading DSL design by abstracting away the details of how rendering is performed, thereby simplifying their programming model. Sony's Open Shading Language (**OSL**) [117], for example, implicitly performs ray tracing at run time, but users do not interact with this process directly; the language makes no distinction between light sources and other kinds of surfaces, for example, and does not expose the structure of the graphics pipeline. Instead, shaders are connected to each other in a graph and are evaluated lazily as data is demanded from them by nodes downstream in the graph. OSL's simple programming model has made it the industry standard for VFX and animation tasks.

Additionally, unlike other shaders whose final output is usually a set of colors describing the scene as viewed from a particular angle, OSL shaders compute *radiance closures*, which describe symbolically how the elements of a scene interact with light and give rendering programs information about each scene. This idea was adopted by other languages for physically based rendering, including the Material Definition Language (**MDL**) [168], a DSL specifically for creating closure-like structures that describe how different materials interact with light. MDL programs are pure and declarative so as to be more easily parallelized on GPU hardware.

Some rendering DSLs aimed for portability by obscuring hardware details; the WebGPU Shading Language (**WGSL**) [352], for instance, aims to bring GLSL to the web, allowing shading programs to run both on GPUs and in a web browser and incorporating many design principles from Rust [328]. The **Rodent** [265] DSL, meanwhile, used partial evaluation techniques along with ideas from

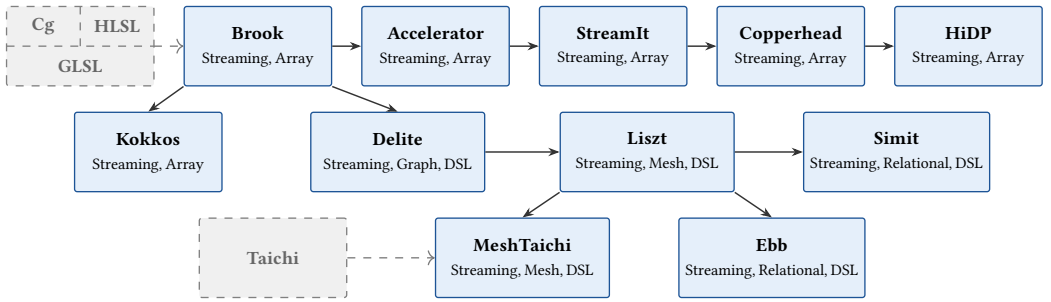


Fig. 18. Descendants of Brook [46]. Dashed boxes and lines indicate connections to or from languages outside this immediate slice of the genealogy in Figure 16.

user-schedulable languages like Halide [277, 278] (discussed in Section 3.6) to present users with a declarative programming model for their renderers. Users first specify the particular scene to be rendered, abstracting the details of which rendering procedures are used under the hood. Then, Rodent applies the first Futamura projection [111], partially evaluating its interpreter on the provided scene to yield a specialized renderer for that particular scene; to execute the renderer on a given hardware target, one need only supply implementations of each rendering procedure required by the scene. Zheng et al. [370] note that, while elegant, this compilation process is quite lengthy.

3.2 Programming with Parallel Patterns

One of the first languages explicitly designed for GPGPU programming (as opposed to being "abused" for that purpose, as many early shading languages were) was **Brook** [46], whose origins were in the general stream programming space but was adapted to GPUs via compilation to shaders; to execute on GPUs, Brook source code was lowered to Cg, leveraging their conceptual similarities. Brook relies on the insight that common structures in graphics programming (e.g., images) can naturally be expressed as functions (e.g., maps from coordinates to color values) and programs manipulating them as transformations on and compositions of those functions. However, Brook does not take this idea as far as the functional languages we discuss in Section 3.4; Brook programs are still imperative in style, resembling an extension of C. Many GPU languages descend from Brook, and we chart their genealogy in Figure 18.

```
1 kernel void saxpy (float a, float4 x<>, float4 y<>, out float4 r<>) {
2     r = a * x + y;
3 }
```

Fig. 19. A basic vector multiplication and addition in Brook, from Buck et al. [46].

Brook programs are expressed in terms of three key ideas: *streams*, *reductions*, and *kernels*. Streams are simply Brook's term for collections of data, operating similarly to arrays in a conventional programming model. Reductions function like folds (in the functional programming sense) and aggregate a stream into a single scalar result. Kernels are stream transformers that accept any number of input streams and produce any number of outputs. Crucially, kernels are written on a per-element basis—that is, kernels implicitly loop over their input streams and are evaluated point-wise. In the example in Figure 19, the input `a` is a scalar, while `x` and `y` are streams. However, in the body of the `saxpy` kernel, the `x` and `y` refer to elements of their respective streams; the kernel will be invoked once per pair of input elements to compute each component of the output. During compilation, each Brook kernel is mapped to a single shader in Cg.

Following Brook's success, **StreamIt** [329]—a contemporary of Brook also originally designed for other hardware—was also adapted to target GPUs using integer linear programming (ILP) [336]. StreamIt views programs as graphs, with nodes representing stream functions and edges representing data dependencies between them. Constraints like "each function maps to one SM" and "functions

that depend on each other execute sequentially" are encoded as ILP inequalities, and the system's solution denotes a mapping from the stream graph to GPU hardware.

The **Accelerator** [318] language, meanwhile, presented a higher-level programming model than Brook's; no hardware details are exposed to programmers, who specify the behavior of their programs as single functions, rather than multiple kernels as in Brook. Accelerator automatically breaks up functions into kernels during compilation, which happens in a just-in-time, rather than ahead-of-time, fashion, further increasing portability. Accelerator's key data type is the parallel array, equivalent in concept to Brook's streams; in the example in Figure 20, the `array` input is explicitly parallelized on line 3, causing the computations on lines 8 and 14 to be processed element-wise but expressed as bulk operations on a single value.

Copperhead [53] simplified the stream programming model with the aim of increasing productivity; Copperhead is embedded in Python [273], and computations are expressed as compositions of classic data-parallel stream combinators like `map` and `reduce` operating over arrays of input. **HiDP** [367] took a similar approach, but also exposed the hierarchical, parallel structure of post-Tesla GPUs without being specialized to any one architecture. Programmers can indicate via annotation at which hierarchy level they would like their combinators to execute, but HiDP's compiler backend is responsible for concretely mapping these combinators to hardware, performing combinator fusion where possible to improve performance.

```
1  static float[,] Blur(float[,] array, float[] kernel) {
2      float[,] result;
3      DFPA parallelArray = new DFPA(array);
4
5      FPA resultX = new FPA(0f, parallelArray.Shape);
6      for (int i = 0; i < kernel.Length; i++) {
7          int[] shiftDir = new int[] { 0, i};
8          resultX += PA.Shift(parallelArray, shiftDir) * kernel[i];
9      }
10
11     FPA resultY = new FPA(0f, parallelArray.Shape);
12     for (int i = 0; i < kernel.Length; i++) {
13         int[] shiftDir = new int[] { i, 0 };
14         resultY += PA.Shift(resultX, shiftDir) * kernel[i];
15     }
16
17     PA.ToArray(resultY, out result);
18     parallelArray.Dispose();
19     return result;
20 }
```

Fig. 20. A 2D matrix convolution in Accelerator, from Tarditi et al. [318].

The **Kokkos** [93, 94] library, meanwhile, adapted the data-parallel model to target GPUs (with a particular focus on heterogeneous CPU-GPU systems) as well as other high-performance hardware. Kokkos is embedded in C++'s template system and has an explicit `parallel_for` construct for launching computations across many threads. It also features a unique separation of "execution spaces" and "memory spaces", which allows users to differentiate programmatically where code is executed from where data is stored and models, among other things, how code may execute on a GPU but access data stored on the host. Kokkos was also influential in the development of polymorphic data layouts, which we discuss in Appendix B.

While languages like Brook and Copperhead presented an element-wise programming model over arrays and streams, others realized that the same approach extended to other structures when operations over their elements were not data-dependent. A number of languages generalized the streaming approach by allowing users to specify the behavior of their programs via actions over *stencils*: local regions of a data structure that only expose an element and its immediate neighbors at a time. Parallel maps over streams can be viewed as a very simple stencil over arrays or lists where the stencil consists only of the element being mapped, while a stencil over a graph would allow users to specify operations per-edge and per-node. Stencil programs are parallelized over the components of the underlying data structure, and draw heavily from *parallel patterns* or *algorithmic skeletons* [71]—a technique pioneered in the late 80s to express program behavior without relying on one single implementation—and thus are often highly portable across data structures and architectures.

As many problem domains are tightly coupled with specific data structures, the parallel-patterns approach became particularly popular for designing DSLs. The **Delite** [57, 310] framework, for example, used parallel patterns to allow programmers to implement their own DSLs. Delite is embedded in Scala [372] and has a deferred execution model inherited by every DSL implemented

within it; programs are compiled to a dependency graph of operations that the compiler is free to reorder and optimize. Nodes in the graph that do not depend on each other are implicitly parallelized. Users can create their own DSLs by defining new operations (i.e., nodes), which can be automatically compiled to CUDA or associated with handwritten microkernels as desired.

A number of DSLs were implemented in Delite, including OptiML [311] and **Liszt** [56], the latter of which was later expanded into an independent mesh-based DSL for partial differential equation (PDE) solvers [88]. A *mesh* is a data structure common in graphics applications that describes shapes as collections of vertices and edges (and faces, in the case of 3D shapes) but that can also be applied to other domains; PDEs in particular can be embedded in a mesh, enabling graphics techniques to solve them. Operations on meshes are typically uniform over their components and only have data dependencies between neighboring elements, so kernels have abundant opportunities for parallelism. However, mesh elements cannot be accessed arbitrarily; they must instead be obtained from their neighbors via topological relationships encoded into the mesh's structure, imposing a particular order upon any iterations over the mesh. Liszt's stencil-based design reflects these concerns; the language exposes no details of its execution order, and operations like the `for`-comprehension (not a `for` loop!) in Figure 21 are stenciled over the mesh, expose only local information about each element, and are implicitly parallelized by the compiler.

```
1 for (v <- vertices(mesh)) {
2   if (ID(v) == 1)
3     Temperature(v) = 1000.0f
4   else
5     Temperature(v) = 0.0f
6 }
```

Fig. 21. A `for`-comprehension stencil over the vertices of a mesh in Liszt, from DeVito et al. [88].

While Liszt dealt with dense meshes, **MeshTaichi** [362] handled sparse meshes by extending Taichi (discussed in Section 3.6), using its ideas to express kernels over sparse meshes as if they were dense. However, during its evolution from Taichi, many of that language's user-scheduling capabilities were removed, making the result more portable but offering users less control.

In the domain of physical simulation, multiple different data structures can be useful depending on the level of abstraction at which one wishes to view the simulated system. Reflecting this, the **Simit** [175] simulation DSL gave users a programming model parallelized over graphs for local reasoning but also included constructs that allowed programmers to shift between this model and one based on vectors or matrices to view simulations globally. Meanwhile, the contemporaneous **Ebb** [32] DSL had a multi-layer model: its lower layer allowed programmers to describe arbitrary data structures upon which to base their kernels, encoding the underlying topology of these structures as relations (in the sense of databases) to make definitions extensible and customizable. The upper layer, on the other hand, abstracted over the details of these relations and allowed programmers to specify kernels element-wise, in a stencil-like fashion.

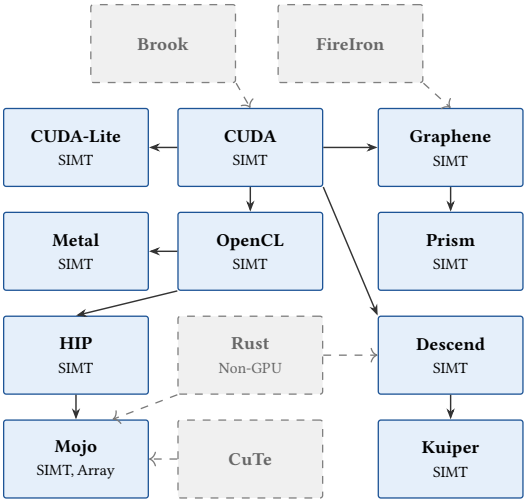


Fig. 22. Descendants of CUDA [229].

3.3 SIMT Languages

The modern GPU landscape owes an enormous amount to Brook; it popularized the use of the term "kernel" to refer to a GPU program, for example. More tangibly, though, the ideas underlying Brook led directly to the development of CUDA, the Tesla microarchitecture, and the SIMT programming model [45, 228], which extended Brook's model by giving programmers more control over how

kernels are distributed over the GPU's compute hierarchy; instead of implicitly parallelizing a kernel point-wise over an input stream, programmers specify exactly how to allocate the parts of a kernel's workload to its participating threads and blocks. We have already discussed CUDA and SIMT at length in Section 2, so in this section we focus on those languages that adapted or extended CUDA in an attempt to evolve the SIMT model, showing their genealogy in Figure 22.

One of the first issues identified with SIMT programming was its complexity, so some early research attempted to simplify the model. **CUDA-Lite** [337], for example, attempted to flatten the memory hierarchy by only exposing programmers to global memory, relying on the compiler to rewrite kernels to use shared memory where necessary for performance. In practice, however, CUDA-Lite's compiler was unable to do this transformation automatically, so programmers were expected to insert annotations on memory accesses or control flow describing where in the hierarchy they wanted data to live to guide compilation. Some examples of these annotations can be seen in Figure 23; the annotations on lines 3 and 4 indicate to the compiler that arrays a and b live in global memory, while the annotation on line 14 specifies the iteration bounds of the loop.

```
1  void kernel (float *a, float *b) {
2     __annotation (L"__global__ TPB 1");
3     __annotation (L"garray a 2 4 ASIZE ASIZE");
4     __annotation (L"garray b 2 4 ASIZE ASIZE");
5     int thi = threadIdx.x;
6     int bki = blockIdx.x;
7     float t = (float) thi + bki;
8     int i;
9
10    __annotation (L"BoundChk");
11    if (bki * TPB + thi >= ASIZE)
12       return;
13    for (i = 0; i < ASIZE; i++) {
14       __annotation (L"loop i 0 ASIZE 1");
15       b[(bki*TPB+thi)*ASIZE + i] =
16       a[(bki*TPB+thi)*ASIZE + i] * t;
17    }
18 }
```

Fig. 23. A matrix-scaling kernel in Cuda-Lite, from Ueng et al. [337].

Other languages attempted to grapple with SIMT's complexity by providing programmers with more guardrails and safety guarantees. **Mojo** [222] uses Rust's [328] ownership model to rule out memory leaks and use-after-free errors and supports multiple execution models, including SIMT for targeting GPUs. Mojo also incorporates ideas from the literature on user-schedulable languages (discussed in Section 3.6) along with an algebraic layout system (discussed in Appendix B) inspired by CuTe [244]. **Descend** [184], meanwhile, builds further on ownership to guarantee full data-race freedom; Descend's key idea is *views*—array access operators that are guaranteed to be memory-safe by construction. By defining reads and writes by composition of views, rather than arbitrary indexing, users can be sure that their accesses are memory-safe. Descend also extends Rust's borrow checker to account for the GPU hierarchy—capturing how ownership of a resource by a block can result in each of that block's threads owning a different portion of that resource—and further reflects that hierarchy in the form of type-level *execution resources*, through which programmers assign computations to threads and blocks to ensure non-overlapping ownership of memory.

Kuiper [217] is a dependently typed SIMT language embedded in F* [315], extending its type system with separation logic [285] to allow programmers to prove properties about Kuiper kernels within the language itself. Kuiper equips its separation logic with *located resources*, which describe how each thread's individual behavior affects the entire kernel; a resource's location describes where in the compute hierarchy it lives and can be used in predicates to assert ownership of that resource by a given thread or block. Other predicates encode notions of visibility for different memory regions and allow Kuiper to describe how, for example, global memory is visible to all threads in a grid but shared memory is only visible to threads in the same block. The opportunity exists to further develop the ideas underlying Descend and Kuiper, as neither has yet formalized its metatheory.

In response to the changes to GPUs we discussed in Section 2.6, work has emerged attempting to evolve the SIMT model to better reflect the reality of new hardware. One such attempt was **Graphene** [123], a SIMT IR with three main features of interest to compilers. The first is a representation format for tensors that allows users to express them as combinations of shapes and element types. Shapes contain a dimension and a stride, which describe how the tensor is laid out in memory, while the element type describes what is stored at each index; this can itself be another shape, allowing tensors

to be defined hierarchically. These shapes are examples of *tensor layouts*—a language feature more commonly seen in tile-based languages and that we discuss in more depth in Appendix B.

Graphene's second feature of interest is *logical thread groups*, which represent the currently active group of compute resources as a run-time object that can be manipulated and decomposed to model collectively executing hardware like Tensor Cores. Finally, inspired by FireIron [122], Graphene programs contain specifications that describe the required input and output shapes of their data tensors, as well as the configurations of threads required to run them. These language features allowed Graphene to express the tensor-level programming model of Tensor Cores; later work on Async Graphene [124] added support for Hopper and Blackwell's asynchronous hardware features by extending the language with concurrency primitives like futures and async references.

Prism [22], meanwhile, has attempted to improve SIMT's handling of collective operations by introducing *typed perspectives*, which reify the compute and memory hierarchy into the language's type system, allowing programmers to express the level of that hierarchy at which they are programming the GPU. Perspectives are similar to Graphene's logical thread groups, but exist statically instead of at run time and can thus be enforced by Prism's compiler. In a Prism kernel, each code point has an associated perspective that dictates which operations it can perform; writes to memory, for example, must happen from the perspective of a single thread, while a collective operation like a Tensor Core requires a larger perspective—a warp for a `wmma` on Ampere hardware or four warps for a Hopper `wgmma`. In Figure 24, the write to `y_thread` happens at a single-thread perspective, as indicated by the `group` operation preceding it.

To keep code perspectives coherent in the face of data-dependent control flow, Prism's perspectives also track how frequently data varies across threads. In the example kernel in Figure 24, the `tid` variable is annotated with a `thread[1]` perspective, indicating that it differs between every thread. The `m` parameter of the kernel, on the other hand, has a `grid[1]` perspective, meaning that every thread in the computation agrees on its value. Prism uses principles from the literature on dependency tracking [1] and information flow [86] to manage the interplay between the perspectives of code and of data; perspectives form a lattice, with broader perspectives (i.e., those representing larger groupings of compute resources or less frequently varying data) at the top and narrower perspectives at the bottom. Data and control are only allowed to flow down the lattice, from broad perspectives to narrow ones. Functions are annotated with the perspectives they require—allowing safe encapsulation and modular composition of collective operations—and Prism programs are guaranteed to have the necessary compute resources to invoke these operations safely.

```
1 @requires(grid[1], thread[1])
2 def add_m_thread(x: ptr(const(int)) @ grid[1],
3                  y: ptr(int) @ grid[1],
4                  m: int @ grid[1]):
5     tid: int @ thread[1] = id()
6     with partition(x, p=thread[1], offset=tid) as x_thread:
7         with partition(y, p=thread[1], offset=tid) as y_thread:
8             with group(thread[1]):
9                 y_thread[0] = x_thread[0] + m
```

Fig. 24. A Prism kernel adding `m` to a vector, from Bansal et al. [22].

3.4 Functional Array Languages

While Brook envisioned GPUs as stream processors, other approaches leaned into the idea of graphics programs as transformations of functions to develop a functional programming model for GPUs. Many of these languages operated over arrays as their primary data structure, borrowing time-tested techniques from APL [158] and building on the concept of *functional arrays* as set forth by Single Assignment C [116, 290] (SAC), which combined the efficient access patterns of arrays with high-level, compositional functional programming. The functional array paradigm gave SAC and its descendants a host of safety benefits, including memory safety and data-race freedom, by adding more structure to array accesses and enabling more expressive type systems.

The first GPU language in this vein was **Vertigo** [97], a high-level, domain-specific shading language embedded in Haskell [135]. Vertigo's embedding is shallow, resembling a library as much

as a language; programs are expressed as combinations of shading primitives, manipulated using typical higher-order functions. Due to its domain-specificity, Vertigo itself did not long survive the transition to GPGPU programming, but its ideas spawned a new line of research into functional array languages for GPUs, depicted in Figure 25.

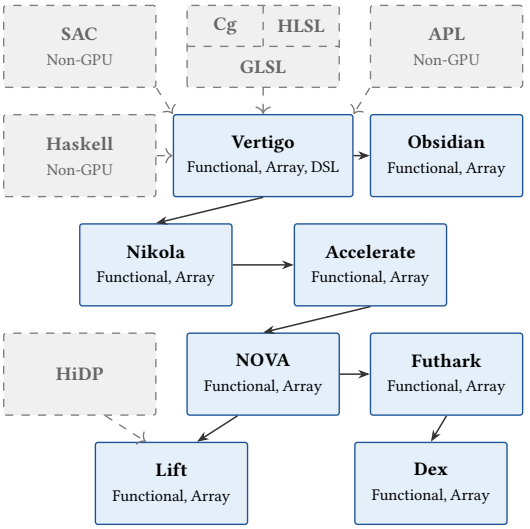


Fig. 25. Functional array languages for GPUs.

The next entry in that line was **Obsidian** [314], another Haskell-embedded language, but, unlike Vertigo, one designed for general-purpose computation. Also unlike Vertigo (and many of the functional languages that would follow), Obsidian exposes the structure of the GPU and gives programmers direct control over hardware resources, allowing them to choose whether to store data to shared or global memory, for example, and reflecting this choice in the type of the array containing the data. Obsidian programs, despite being embedded in Haskell, are imperative in spirit, making heavy use of Haskell's monadic do-notation to emulate stateful operations. The primary benefit of Obsidian is access to Haskell's type system, which has stronger guardrails than CUDA's, but the trade-off for this increased safety was performance; Obsidian kernels are slower than their CUDA counterparts. Subsequent work [313] attempted to bridge the gap by introducing a set of combinators for working with arrays and compiling these combinators to efficient CUDA, improving the situation for programs that could be expressed using compositions of these combinators, but not otherwise.

Other languages chose to obscure the hardware structure to pursue greater portability and simplify the programming model. **Nikola** [214] and **Accelerate** [58], for example, deepened the Haskell embedding and allowed programmers to write kernels that resemble Haskell but execute imperatively like CUDA without relying on do-notation to mimic stateful behavior. The performance of both languages still trailed behind that of CUDA, but the path forward was clear: more compiler optimization was necessary, in particular automatic kernel fusion.

These optimizations were implemented in **NOVA** [74], also the first language in this line of research not embedded in Haskell. While usable as a source language in its own right, it had the dual goal of being a useful IR for compiling DSLs to GPUs; certain design decisions, like explicit use of fixed-point combinators to implement recursion, reflected this goal. In either use case, NOVA achieved competitive performance via extensive optimization during compilation to CUDA, including combinator fusion and a pass to automatically flatten and unflatten nested lists; as differently sized sub-lists can lead to load imbalances when they are allocated to different blocks, this pass improves parallelism by resulting in a more uniform distribution of work.

```
1 dotp :: Vector Float -> Vector Float -> Acc (Scalar Float)
2 dotp xs ys = let xs' = use xs
3                  ys' = use ys
4              in
5              fold (+) 0 (zipWith (*) xs' ys')
```

Fig. 26. Vector dot product in Accelerate, from Chakravarty et al. [58].

Lift [306, 308], meanwhile, aimed to balance performance with portability, combining many of NOVA's ideas with the parallel patterns-based approach seen in languages like HiDP and libraries like SkelCL [307]. Lift is compiled to OpenCL via a series of rewrite rules, which are proven sound with respect to a dependently-typed core calculus. While Lift programs are architecture-agnostic, the rewrite rules compiling them are not; once given a particular hardware target, Lift's compiler selects appropriate rewrites for it, thus achieving portability. Moreover, the set of rewrite rules is extensible

in the compiler, allowing Lift to be forward-compatible with new hardware via new rewrite rules. This combination of hardware-agnostic source code with hardware-sensitive rewrites also appears in user-schedulable languages, but Lift is not such a language itself because users cannot directly interact with the rewriting process. Subsequent work on RISE/ELEVATE [125] and Exo [153, 154] evolved Lift's ideas by giving users control over application and extension of its rewrites; we discuss these further in Section 3.6.

The most influential recent work in the functional lineage is **Futhark** [142], which, like NOVA, makes use of compiler optimizations like combinator fusion and automatic coalescing to achieve CUDA-competitive performance while maintaining expressiveness. Its key innovation, however, was support for in-place array updates and Rust-inspired "uniqueness types" to ensure these updates have a pure semantics, further improving performance. Futhark's type system guarantees a number of additional properties beyond the memory safety and data-race freedom common to functional array languages, most notably freedom from run-time shape errors (i.e., a guarantee that inputs to array and tensor operations have the right dimensions). It has been a popular substrate for functional programming and type systems research on GPUs since its release; recent projects have added efficient array bounds checking [141], rank-polymorphic [151] type inference [289], a higher-order module system [98], and an Agda [321] front-end to execute formally verified tensor algebra on GPUs [373]. Other languages like **Dex** [262] have built on Futhark's ideas with dependent value types and a monadic effect system to support automatically differentiable array programming—important for deep learning.

```
1 fun main(m : [n][m]f32): ([n][m]f32, [n]f32) =
2   map (λrow : ([m]f32, f32) →
3     let row' = map (λx : f32 → x+1.0) row
4     let s = reduce (+) 0 row
5     in (row',s)) m
```

Fig. 27. A Futhark kernel adding 1 to a matrix and summing its rows into a vector, from Henriksen et al. [142].

3.5 Machine Learning DSLs

The functional languages discussed in the previous section were not the only GPU languages to borrow array-programming ideas from APL; **TensorFlow** [2] and **PyTorch** [261] are enormously influential deep learning DSLs that make up a significant fraction of current GPU programming and also feature an array-programming model. Unlike the languages in the previous section, however, they are not functional, and unlike the rest of the languages in this survey, they are used for programming entire machine learning pipelines, rather than individual kernels.

These two languages are similar in approach and design, and their gentle on-ramp and simple programming model make them popular choices for developers who do not need the full power of languages like CUDA or OpenCL to realize their goals. Prior to these languages, machine learning workloads were commonly handled using libraries like NumPy [257], Caffe [162], Theano [31], Chainer [332], and Torch [75], whose fundamental data structure is the tensor. Reflecting this, PyTorch and TensorFlow programs operate over tensors, which are handled similarly to arrays in APL; execution is implicitly parallelized over these tensors, which are reshaped or broadcasted as needed to guarantee they have the appropriate size. This behavior would be later generalized from arrays to arbitrarily dimensioned tensor tiles, as we will see in Section 3.7. Later DSLs like **JAX** [110] further developed PyTorch and TensorFlow's designs, abstracting away many of their machine-learning-specific details to focus on general differentiable programming and shifting to a more functional programming model with Just-In-Time compilation.

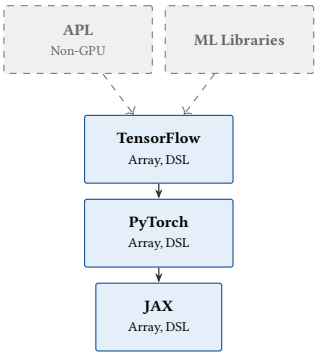


Fig. 28. Machine learning DSLs.

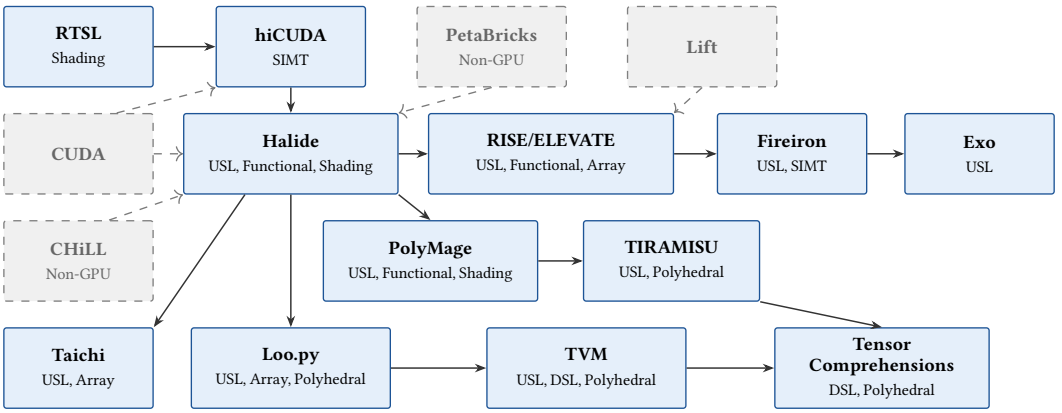


Fig. 29. Descendants of Halide [277, 278].

3.6 User-Schedulable Languages

High-level languages rely on extensive compiler optimization in order to achieve competitive performance. Their compilers can achieve equivalent or even better performance than hand-optimized low-level kernels in the best case, but, as we discussed in Theme II, the complexity of the GPU optimization space makes this approach brittle; when programmers deviate too far from the problems high-level languages are designed for, performance frequently suffers as a result. Performance engineers are thus often skeptical or mistrustful of optimizing compilers. The key idea of *user-schedulable languages* (USLs), whose genealogy on GPUs we depict in Figure 29, is to trust that these experts are more capable of optimizing their code than a compiler, while also recognizing that separating the concerns of the optimization process from the specification of program behavior results in a simpler programming model.

The insight that users should have some control over optimization appears early in the history of GPU languages in RenderMan's computation rates and the Real-Time Shading Language's computation frequencies. These annotations functioned as hints to the compiler about how often values should be computed or how frequently they changed across the different stages of the graphics pipeline. The ***hi*CUDA** [128] language—a high-level, sequential variant of CUDA—built upon these annotations with *directives*, which come in two forms and allow users to specify how to map code to hardware. Directives affecting the computation model specify which code should be executed on the GPU and how to parallelize it; a `loop_partition` directive, for example, informs the compiler how to distribute the iteration space of a sequential loop over the threads and blocks on the grid. Directives for the data model, on the other hand, dictate the movement of data between the levels of the memory hierarchy.

The **PetaBricks** [15, 268] framework, meanwhile, gave programmers control over *algorithmic choice*—the ability to specify multiple versions of their kernels and the conditions under which each version should be used. A simple motivating example for this idea is sorting an array; for small inputs, a quadratic algorithm like insertion sort outperforms merge sort despite the latter's superior asymptotic complexity. Accordingly, many languages' standard libraries use quadratic sorts for inputs below a certain hard-coded size. PetaBricks, however, argues that this size is hardware dependent and no single choice will perform well on every machine. Instead, the right size should be identified at compile time; PetaBricks extends this idea beyond list sorting by allowing users to specify general conditions—such as specific sizes, dimensions, or types of input data—under which different versions of their programs should be executed, filling in concrete values via autotuning.

```
permute ([3, 1, 2])
tile (0, 2, TJ)
tile (0, 2, TI)
tile (0, 5, TK)
datacopy (0, 3, 2, [1])
datacopy (0, 4, 3)
unroll (0, 4, UI)
unroll (0, 5, UJ)
```

Fig. 30. A transformation recipe in CHiLL, from Hall et al. [127].

PetaBricks's algorithmic choices and *hi*CUDA's directives, like computation rates and frequencies before them, were still just guidance to the compiler, however; users were still not able to directly control the optimization of their code. The first instance of a fully user-controlled optimizer appeared outside the GPU space in the form of **CHiLL** [59, 288], a compilation framework for sequential loop programs in C that allows programmers to directly control how their code is optimized and compiled via *transformation recipes*. These recipes are user-written scripts—an example of which can be seen in Figure 30—comprising many fine-grained transformation primitives handling operations like tiling, loop unrolling, and data movement; developers compile their programs by executing these scripts[5]. This approach is in some ways the inverse of PetaBricks's—whereas in PetaBricks, users specify multiple algorithms and rely on the compiler to choose one based on the hardware, CHiLL asks users to write one algorithm and then specify multiple possible ways to map it to hardware. There is much more to be said about CHiLL and its origins, but we lack space to cover the full history of USLs in general; we direct interested readers to Hall et al. [127]'s comprehensive survey for more details and focus here on their application to GPUs.

USLs achieved maturity with **Halide** [277, 278], a USL for graphics that has seen extensive industry adoption [5] and coined much of the terminology used in the literature. In particular, Halide codified the distinction between the *algorithm* (i.e., what a kernel is doing, sometimes called the "application logic") and the *schedule* (i.e., how the algorithm is being performed, or in particular how work is assigned to different compute and memory resources on hardware). Halide algorithms are pure functional transformations from inputs to outputs; this purity is necessary to guarantee the safety of scheduling operations like tiling, as otherwise it would not be safe to reorder parallel loops [127]. Schedules, like in CHiLL, are sequences of built-in primitives like `tile` and `vectorize`.

```
Func blur_3x3(Func input) {
    Func blur_x, blur_y;
    Var x, y, xi, yi;

    // The algorithm - no storage or order
    blur_x(x, y) = (input(x-1, y) + input(x, y) + input(x+1, y))/3;
    blur_y(x, y) = (blur_x(x, y-1) + blur_x(x, y) + blur_x(x, y+1))/3;

    // The schedule - defines order, locality; implies storage
    blur_y.tile(x, y, xi, yi, 256, 32)
          .vectorize(xi, 8).parallel(y);
    blur_x.compute_at(blur_y, x).vectorize(x, 8);

    return blur_y;
}
```

Fig. 31. A three-by-three image blur in Halide, from The Halide Team [324].

Halide also advanced the USL paradigm by unifying the languages for specifying the algorithm and the schedule. Unlike in CHiLL, for example, where transformation recipes were separate from the C code they compiled, Halide's schedules live in the same files as its algorithms, allowing its scheduling operators to directly inspect and modify values in the algorithm. In Figure 31 we can see a simple Halide kernel; the algorithm defines two functions, `blur_x` and `blur_y`, while the schedule tiles, parallelizes, and vectorizes these functions. We can also see that, like in Brook, images are functions from coordinates to data, and Halide algorithms do not specify the bounds of the images they operate over. Instead, Halide's compiler infers these by symbolically evaluating indices into functions and arrays to determine the intervals over which they must be computed. Later work on **TIRAMISU** [21] argues this is a limitation of Halide; in particular, Halide requires rectangular iteration spaces, but this is acceptable given its focus on graphics—images are rectangles, after all. TIRAMISU is similar to Halide in both design and purpose, but uses a *polyhedral* [156, 189, 204] representation for its optimizations (we discuss polyhedral compilation further in Appendix A),

[5] A note on terminology: a *user-schedulable language*, or USL, is one whose optimization (or schedule) can be controlled by the program author, while a *scheduling language* is a DSL for writing schedules for a USL. In the case of CHiLL, transformation recipes are the scheduling language and make C user-schedulable.

which has a different set of trade-offs. In particular, the polyhedral model supports arbitrarily shaped iteration spaces and can identify more opportunities for loop fusion than Halide's bounds inference, but requires loop bounds to be linear functions of constant values. Symbolic bounds inference, on the other hand, supports more complex, non-linear loop bounds and symbolically sized tiling transformations, which the polyhedral model cannot express [278].

Halide conceived of schedules as inhabiting a 2D space; programs are loop nests over the dimensions of images, and transformations on those images occur at a particular granularity. The deeper into the nest an operation appears, the finer its *compute granularity*, and if the operation's result is stored in memory, the level of the nest at which that occurs is its *storage granularity*. The space of schedules plots these two granularities against each other, with the fundamental constraint that an operation's compute granularity must be finer than its storage granularity—every computed value must have a location into which it can be stored. Each operation (or call) in an algorithm is given a call schedule along these two dimensions that places it within the loop nest. Finer compute granularity results in higher locality (data needs to persist for less time before it is used) but can cause more data recomputation. Coarser storage granularity, meanwhile, increases reuse but can introduce sequential dependencies, reducing parallelism.

There is a wide body of work building directly on Halide: formalizing its semantics [283], validating the correctness of its scheduling transformations [69], automatically deriving efficient schedules via autotuning [224, 299, 300], and using machine learning to improve that process [4]. Work has also been done to use Halide in domains beyond graphics like tensor algebra [366] and machine learning [200], or to extend its ideas to different programming models, as in the case of **Loo.py** [176], which used an imperative, Python-embedded approach.

Many DSLs built on Halide's ideas, using their domain specificity to simplify their scheduling search space and identify new optimizations. The **PolyMage** [160, 225] DSL, for example, leverages the doctrine of efficiency from specificity for graphics, employing a handful of domain-specific schedules, most notably *overlapped tiling*. Overlapped tiling differs from usual tiling strategies because tiles share some of their data with adjacent ones—necessary for certain graphics operations wherein pixels require data from the region surrounding them. However, an overlap that is too coarse can cause a large synchronization overhead between threads, and one too fine can result in redundant computation. PolyMage's approach is to perform overlapped tiling at the level of warps, rather than threads or blocks, and store part of the tile in the warp's registers instead of shared memory; it then employs *warp shuffling* [78] to exchange data between threads within the warp without going through shared memory, improving efficiency.

TVM [62], meanwhile, brought user-scheduling to the deep learning domain, comprising a high-level DSL for tensor operations and an optimizing compiler that builds on Halide's schedules and Loo.py's Pythonic front-end, using a learning-based search algorithm to identify the optimal schedule for each algorithm on different hardware. It also implements some additional schedules beyond those from Halide, such as automatic tensorization and explicit latency hiding, which are specialized to deep learning workloads and which allow it to target other new architectures like TPUs [120]. Other compilers for TVM target even more specific machine learning use cases, such as Cortex [103] (for recursive models) and Relax [185] (for models with dynamic tensor shapes), while IRs like Relay [286] and TensorIR [104] sit higher up the stack and present a common target for ML frameworks that wish to leverage TVM's compilers.

Tensor Comprehensions [341] (TC) is even higher level than TVM and was designed to present the most abstract possible model for tensor programming. Kernels are specified mathematically; the entire TC definition of

```
def sgemm(float a, float b, float(N, M) A, float(M, K) B) → (C) {
  C(i, j) = b * C(i, j)
  C(i, j) += a * A(i, k) * B(k, j)
}
```

Fig. 32. SGEMM using Tensor Comprehensions, from Vasilache et al. [341].

SGEMM, for example, is given in four lines in Figure 32. Despite its programming model, TC achieves competitive performance by leveraging its extreme domain-specificity; TC is lowered first to Halide's IR and then to a polyhedral representation, autotuning its parameters all the way down. The polyhedral optimization space—along with that of Halide—is well understood, and TC takes advantage of this fact to generate highly optimized kernels.

Taichi [150] is a DSL for computation over sparse data structures, particularly tensors, which require special data structures to avoid storing an excess of trivial data and do not (in general) support constant-time access; they can only be efficiently accessed along dimensions in which they are dense. Algorithms that operate over multiple tensors therefore differ depending on which of those tensors are sparse in which dimensions, and efficient implementations of tensor algebra operations have a combinatorial explosion of implementations—one for each combination of sparse and dense inputs. One influential solution to this problem was pioneered by the Tensor Algebra Compiler, or Taco [174], which automatically generates implementations of a given tensor operation for inputs of various sparsities. Users specify via a library API which computation to apply to which tensors, along with a storage format that describes the dimensions and sparsity characteristics of each input; Taco then synthesizes an *iteration graph* that describes how to co-iterate over the inputs and how iteration changes when one of their dimensions is empty of data. Later extensions to Taco added a scheduling API [173], allowing users more control over its code generation, and the ability to target both individual GPUs [66, 293] and multi-GPU systems [356].

Taichi, on the other hand, took a user-scheduling approach to sparse tensor programming, allowing developers to separate the specification of what data they want to store from how that data is actually arranged in memory. Said another way, Taichi lets programmers pretend that their data structures are dense, even when they are actually sparse. Taichi includes a language for defining and describing data structures via composition of sparse or dense components; based on this definition and guided by user-provided scheduling hints, Taichi's compiler transforms iterations over a data structure—written as if it were a dense array—into an iteration over the underlying representation of that structure by mapping the logical coordinates of the user-facing structure into real memory coordinates. However, it is worth noting that while Taichi originated as a USL, later development removed most of its user-scheduling features in the pursuit of portability [317].

The USLs we have discussed so far have been broadly successful, but suffer from a few drawbacks. By giving programmers precise control over compilation, USLs dramatically raise the complexity of their programming model and present a steep learning curve for non-expert users. Additionally, they are not extensible; users cannot add to or modify the set of scheduling operations that are available to them. This limitation has two dimensions: first, users cannot implement optimizations outside what the built-in operators allow [125], and second, users are dependent on the compiler to update the implementation of hardware-specific operators when new microarchitectures are released [153]. The latter problem is significantly more impactful on GPUs than CPUs due to the fast and visible cadence of hardware evolution discussed in Theme III; some work exists on updating user-scheduling languages to target Tensor Cores, such as **FireIron** [122], but these approaches only address a specific instance of a more general problem.

Work addressing the first of these dimensions made the set of scheduling operators user-extensible by reconceptualizing them as term rewrites. This approach drew on earlier work on Lift [306] (previously discussed in Section 3.4), and was first realized by a pair of programming languages: **RISE** and **ELEVATE** [125]. The algorithm language, RISE, was a functional array language, while ELEVATE was based on *strategy languages* [343]—a kind of language for specifying rules for term rewriting systems—allowing it to be fully programmable, unlike CHiLL or Halide's more limited set of primitives. Each strategy encodes a particular way of rewriting a program and can be composed using combinators; these can be as simple as sequencing (the ; operation in Figure 33) or can feature

complex logic like conditionally applying one strategy based on the result of another (the `match` expression in Figure 34).

One of the main risks of allowing users to specify their own rewrites is that they might write "optimizations" that do not actually preserve the semantics of the original program. RISE's semantics are formally modeled in Agda and ELEVATE's built-in rewrites and strategy combinators are proven correct against those semantics, but this does not guarantee the correctness of any new rewrites a user might implement. The ATL [207, 208] language has addressed this issue by embedding rewrites as theorems in the Rocq [320] proof assistant—guaranteeing their correctness by construction—but it has not yet been extended to target GPUs. Shoggoth [275], meanwhile, modeled the semantics of strategies in Isabelle [230] and formulated a calculus for proving properties of rewrites beyond correctness, such as termination or composability.

```
val dot = fun(as,
          fun(bs,
            zip(as)(bs) |>
            map(fun(ab, mult(fst(ab))(snd(ab)))) |>
            reduce(add)(0)))
val mm = fun(a : M.K.float,
          fun(b : K.N.float,
            a |> map(fun(arow,               // iterating over M
              transpose(b) |> map(fun(bcol, // iterating over N
                dot(arow)(bcol)))))))     // iterating over K
```

```
val tiledMM = (tile(32,32) `@` outermost(mapNest(2)) `;` lowerToC)(mm)
```

Fig. 33. Matrix multiplication in RISE/ELEVATE, from Hagedorn et al. [125]. The upper box is the algorithm in RISE, while the bottom is the optimization strategy in ELEVATE.

Strategic rewriting has a few limitations, however; it scales poorly to large search spaces, and later research sought to replace it in RISE's compiler with an approach based on *equality saturation* [178]. Additionally, it does not address the second of Halide's major challenges: ELEVATE's rewrites are source-to-source and the process of lowering optimized programs to executables is still opaque to the user. Exposing this process to the programmer is the key idea of *exocompilation*. Introduced by the **Exo** [153, 154] language, exocompilation identifies the boundary that every compiler implicitly draws between automation (i.e., what the compiler does on its own) and control (i.e., what users can affect directly) and shifts the process of hardware-specific code generation to the user side of that boundary. In languages like Halide, hardware-specific compilation or optimization steps like register allocation and instruction selection are performed automatically by the compiler—Ikarashi et al. [154] argue that this decision is why Halide has struggled to keep up with the release of new GPU hardware features like Tensor Cores. By contrast, in Exo, hardware-specific compilation steps can be specified as user-extensible libraries, allowing performance engineers to target whatever hardware they desire without needing to modify the compiler. Like ELEVATE, Exo's schedules are expressed as compositions of rewrite rules; Exo also features an effect system and formal semantics to argue the correctness of its rewrite system, although its metatheory is not yet formalized. Exo has seen some industry adoption [154] and has been used to generate portable GEMM kernels that outperform hand-optimized ones on various accelerators [51, 52], and recent work has extended its ideas to GPUs, with a focus on Tensor Cores [152].

```
def tileND(n: List[Int]): Strategy[Rise] = DFNF `;` (n.size match {
   case 1 => function(split(n.head)) // loop-blocking
   case i => fmap(tileND(d-1)(n.tail)) `;` // recurse
             function(split(n.head))   `;` // loop-blocking
             interchange(i) })             // loop-reorder
```

Fig. 34. A conditional strategy definition in ELEVATE, from Hagedorn et al. [125].

3.7 Tile-based Languages

As we discussed in Section 2.6, the advent of Tensor Cores gave rise to a new programming model for GPUs—one in which tiles, rather than scalars, were the new fundamental unit of computation. On first release, Tensor Cores were only fully programmable in PTX [250] bytecode; CUDA had little support for the new hardware. Programmers were limited to libraries like cuDNN [243] or cuBLAS [242], if those libraries supported their use case, or to the unstable `wmma` API [17, 216], if they did not. This situation was improved by **CUTLASS** [322], a library for recursive decomposition of linear algebra computations along the GPU's block-warp-thread hierarchy [166]; a GEMM, for example,

could be recursively decomposed into multiplication of block tiles, which could then be further decomposed into multiplication of warp tiles, and ultimately into thread tiles, each of which would be accumulated into a result. CUTLASS is embedded in C++'s template system and automatically generates kernels from *policies*, wherein users specify tensor dimensions and tiling strategies via templates. While not originally designed to support Tensor Cores, it proved a popular medium for programming them; CUTLASS's warp-tile-level GEMM could be implemented with Tensor Cores, with other parts of the API abstracting the process of massaging input data into their expected format.

However, while CUTLASS was a marked improvement over direct use of `wmma`, it lacked the structure and composability of a fully fledged programming language. **Triton** [331] soon filled this gap, offering the first language-level support for tile-based programming and reifying the previously implicit structure of tiles into the language via a new programming model. Instead of performing scalar operations per-thread, Triton programs are conceptually single-threaded, require no manual synchronization, and undertake computations a tile at a time, at the granularity of an entire block. Programmers can focus on the application logic of their kernels, leaving concerns like recursive decomposition and swizzling data for consumption by Tensor Cores to the compiler, while still achieving performance competitive with hand-optimized CUDA on common ML workloads like matrix multiplication and convolution. Meanwhile, Triton's type system statically tracks the shapes of tiles, enforcing that inputs to tile-level operations like tensor addition or multiplication have the correct dimensions; when tiles are incompatible, they can be automatically padded or replicated until they have the right shape similarly to broadcast semantics in languages like PyTorch and TensorFlow. Tile-based programming abstracts both SIMT (it is sometimes even called a Single-Instruction, Multiple-Block (SIMB) model [119]) and array-based programming (arrays are 1-dimensional tensors), and in the same way that SIMT is an effective model for programming CUDA Cores, the tile-based approach envisioned by Triton and its descendants (depicted in Figure 35) is a natural fit for the initial, synchronous generations of Tensor Cores on Volta and Ampere.

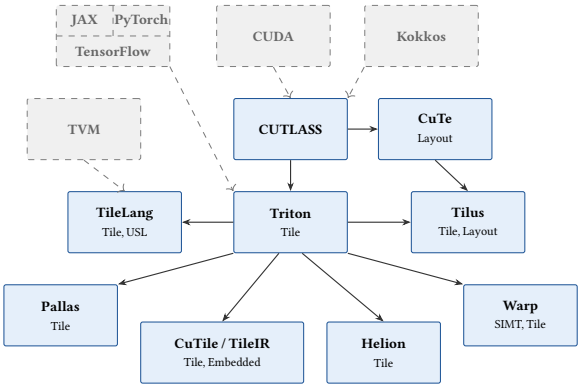


Fig. 35. Descendants of CUTLASS [322] and Triton [331].

In the example in Figure 36, the `x` and `y` variables are initialized with a (1-dimensional, in this case) tile of data, based on the `offsets` computed from the size of the block. The sum of these tiles is then stored in the `output`, with addition being performed at tile granularity. The `+` operation requires the types of its inputs to be compatible; if either `x` or `y` were longer than the other, the shorter tile would be reshaped to match.

```
def add(x_ptr, y_ptr, output_ptr, BLK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)

    block_start = pid * BLK_SIZE
    offsets = block_start + tl.arange(0, BLK_SIZE)

    x = tl.load(x_ptr + offsets)
    y = tl.load(y_ptr + offsets)

    output = x + y

    tl.store(output_ptr + offsets, output)
```

Fig. 36. Vector addition in Triton, from Gupta [121]

Triton developed the tile-based programming paradigm, but it did so very domain-specifically; its ecosystem and optimization strategies were primarily oriented towards deep learning workloads. As a result, some work—such as **Warp** [212]—aimed to bring tile-based programming to other areas by adapting Triton to differentiable computing more generally. Warp was initially designed with a SIMT programming model, but added tile-based programming support early in its lifespan [213].

Some languages built directly on Triton's foundation but further raised its abstraction level to be more portable or easier to program. **Helion** [274], for example, sought to be "higher-level Triton" [218] by abstracting details like tensor indexing and kernel launch dimensions and using autotuning to search for the best values for these factors at compile time. **TileLang** [344] built on ideas from TVM, asking developers to focus on specifying how data flowed through their kernels and leave scheduling to the compiler. TileLang programs comprise a number of tile-level operators that encapsulate common patterns like data copying or matrix multiplication, with a light set of scheduling operators and annotations available for when the compiler's automatic optimization passes fall short. The **cuTile** [96, 236] family of embedded languages, meanwhile, allows programmers to write Triton-like kernels in familiar languages like Python and Rust and compile to a common IR called TileIR [238].

Other languages went the opposite direction, lowering Triton's abstraction level to squeeze more performance out of the hardware. **Pallas** [325], for example, offers greater control, requiring explicit use of Tensor Cores and other hardware features, but offsets the increased complexity with source-level metaprogramming and the ability to target TPUs [120]—another kind of accelerator for tensor operations with similar (but, crucially, not identical) kernels. **CuTeDSL** [244], meanwhile, began as an extension of the CUTLASS library with support for algebraic layouts [55, 245]—discussed in Appendix B—but evolved into a fully fledged, Python-embedded language. Layouts complement the tile-based model by allowing programmers finer control over how tiles are arranged in memory; **Tilus** [90], a language for low-precision computation [109], makes use of them to expose more details of the GPU hierarchy and grant more control over data movement. CUDA libraries such as ThunderKittens [304] also emerged to support tile-based patterns while also allowing programmers to "drop down" into CUDA as needed for performance.

```
fn add<const B: i32>(
    z: &mut Tensor<f32, {[B]}>, // exclusive write
    x: &Tensor<f32, {[-1]}>, // shared read
    y: &Tensor<f32, {[-1]}>, // shared read
) {
    let tx = load_tile_like(x, z);
    let ty = load_tile_like(y, z);
    z.store(tx + ty);
}
```

Fig. 37. Matrix addition in cuTile Rust, from Elibol et al. [96].

However, while tile-based programming was an elegant abstraction for synchronous, warp-level Tensor Cores, offering simplicity without compromising on performance, it is less clear that it is the right model for programming asynchronous hardware units like TMAs and warpgroup- or block-level Tensor Cores on Hopper and Blackwell GPUs. Due to their single-threaded programming model, languages like Triton and cuTile must rely on their compilers to determine how to specialize warps and when to use Tensor Cores or the TMA [284], and the heuristic methods they employ may not always make the optimal choice. Benchmarks [355] indicate that Triton can underperform compared to handwritten kernels—including on benchmarks on which it was evaluated originally, like attention—while cuTile's performance varies wildly across machines, even within the same microarchitecture generation. Languages like Pallas and extensions to Triton like Gluon [334] and TLX [119] have attempted to address this problem by giving programmers more control, allowing them to program at the granularity of a warp or warpgroup rather than a whole block (Pallas), manually specialize warps (TLX), or explicitly specify when to use Tensor Cores and TMAs using inline IR (Gluon). Giving users this much control, however, sacrifices much of what made Triton an elegant abstraction in the first place; these languages cannot be conceptually single-threaded because they must handle resources like TMEM and cluster-level Tensor Cores that

```
def tlx_kernel(x_ptr, y_ptr, n_elements, BLOCK: tl.constexpr):
    ...
    with tlx.async_tasks():
      with tlx.async_task("default"):
        while tile_id != -1:
          tlx.barrier_wait(empty[0], prod_phase)
          x = tl.load(x_ptr+...)
          tlx.local_store(smem[0], x)
          tlx.barrier_arrive(full[0])
          tlx.clc_producer(...)
          tile_id = tlx.clc_consumer(...)
      with tlx.async_task(num_warps=1):
        while tile_id != -1:
          tlx.barrier_wait(full[0], cons_phase)
          x_local = tlx.local_load(smem[0])
          y = x_local * 2.0 + cta_rank
          tl.store(y_ptr+..., y)
          tlx.barrier_arrive(empty[0])
          tile_id = tlx.clc_consumer(...)
```

Fig. 38. Warp-specialization in Triton using TLX, from Guan et al. [119].

are shared across warpgroups and blocks. Thus, programmers are forced to grapple with the full complexity of communication on asynchronous hardware, making it easy to write buggy code.

Other work has attempted to preserve the simplicity of Triton's model by enhancing its compiler; in particular, Tawa [61] attempts to automatically warp-specialize Triton kernels, allowing them to make use of Hopper's TMAs, which Triton had previously struggled to wield effectively. Tawa's core idea is the asynchronous reference (`aref`), a communication primitive that allows warps to have a formal producer-consumer relationship. One warp can publish values to an `aref`, which has operations to get, set, and consume its value, while others can asynchronously wait for data to arrive. The `aref`'s semantics enforce the producer-consumer relationship between these warps: it cannot be read while empty or written while full, and each warp possesses a *credit* (effectively a capability) allowing producers to write to the `aref` while empty and consumers to read from it while full.

While Tawa focused on the problem of applying a given warp-specialization strategy to a Triton kernel, the Twill [302] framework tackled the related question of deriving the optimal such strategy, arguing that heuristic methods of warp specialization are not portable across GPU generations. Twill is based on the idea of software pipelining [186], which mimics at the software level the way that CPU hardware pipelines dynamically exploit instruction-level parallelism by statically reordering instructions to overlap independent computation and maximize utilization of hardware resources. While prior approaches to automatic warp specialization performed software pipelining independently, Twill argues that these two processes are intrinsically linked and must be considered together in order to derive optimal schedules for either. The usual algorithm to identify optimal software pipelines is called *modulo scheduling* [279, 280]; by parameterizing this algorithm over certain hardware details, Twill is able to derive and prove optimal warp-specialized schedules for a variety of deep learning kernels, including Flash Attention on Hopper [294] and Blackwell [363].

However, Chen et al. [61] note that while tools like Tawa have achieved competitive performance, the ergonomics of warp-specialization remain challenging, clashing with fundamental assumptions about SIMT execution baked into both GPU hardware and the software ecosystem surrounding it. Once again, the evolution of GPU hardware has broken the existing programming model; whether updated abstractions exist that can maintain Triton's ergonomics in the asynchronous setting without surrendering performance is an ongoing area of research.

3.8 Task-based Languages

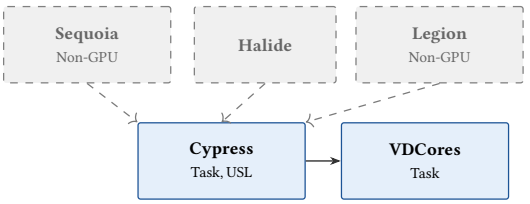


Fig. 39. Task-based languages for GPU programming.

Another approach to the evolution of asynchronous GPU hardware has been to abandon the tile-based model and move to a different, *task-based* programming model. Task-based programming has its origins in high-performance computing (HPC) languages—namely Sequoia [101] and Legion [29]—for large-scale multiprocessor machines. These languages have since been extended to target GPUs, but their original targets were more heterogeneous; in the task-based model programmers specify small "tasks" or "micro-operations", which describe a small part of a larger computation, and assign these tasks to the various components of their distributed or heterogeneous hardware. Yadav et al. [357] assert that this is the natural way to manage the complexity introduced by the various fixed-function components of modern GPUs, like Tensor Cores or TMAs, and scales as chips continue to evolve—Blackwell, for example, has even more specialized subcomponents than Hopper.

An early work in this area (depicted in Figure 39) is **Cypress** [357], which attempted to unify task-based programming with user-scheduling; programs comprise a *logical specification* and a *mapping specification*—essentially an algorithm-schedule distinction, except that Cypress's mapping specifications assign tasks to different groupings of threads. The logical specification for a GEMM,

for example, is initially mapped to the entire grid; this mapping is then decomposed into subtasks for blocks, warpgroups, warps, and eventually threads. Each level of the hierarchy has an associated mapping with its own particular properties; each mapping can specify how to move data between the levels of the hierarchy as the task is decomposed, for example, but the block-level mapping can also specify if and how to perform warp-specialization. Cypress's logical specifications elide the tricky details of synchronization or inter-thread communication, allowing the programmer to focus purely on their kernel's logic; these details instead appear in the mapping specification and are inserted by the compiler along with efficient data movement to maximize use of the GPU's resources.

Meanwhile, Virtual Decoupled Cores, or **VDCores** [140], is a task-based IR that models each execution unit of the GPU as an independent, isolated core; each core's behavior is specified in a "microservice" style, with each service connected to others via its data dependencies. So, for example, CUDA Cores and Tensor Cores would be coupled together into an Async Compute Unit and modeled as one core, while the TMA would be an Async Memory Unit modeled as a different core. Cores are abstract and extensible, allowing the system to easily adapt to further evolutions of GPUs' design. VDCores programs are dependency graphs of small operations (called *μops*), which are scheduled based on those dependencies. The *μ*ops are assigned to cores dynamically by the runtime, and can be reordered so long as their dependencies allow it, effectively performing software pipelining to hide latency. At time of writing, it is not yet clear whether this new line of research into task-based languages will be as effective for asynchronous GPUs as the tile-based model was for Tensor Cores, but it seems a promising avenue for future work.

```
1  @task ("gemm", Inner, reads=["A","B"], writes=[ "C" ])
2  def gemm_block (C: tensor[2, f16],
3                  A: tensor[2, f16],
4                  B: tensor[2, f16]) :
5    W = tunable(int)
6    M , N , K = C.shape[0], C.shape[1],  A.shape[1]
7    # Break A and B into tiles of width W.
8    Ap = partition_by_blocks(A, (M, W))
9    Bp = partition_by_blocks(B, (W, N))
10   Cacc : tensor[2, f16] = make_tensor(M, N)
11   launch("clear" ,Cacc)
12   # Launch a sequential group of tasks over each tile.
13   for k in srange (cdiv (K, W)):
14     launch("gemm", Cacc, Ap [0, k], Bp [k, 0])
15   launch("copy", Cacc, C)
16
17 # Mapping Specification
18 TaskMapping (
19   instance="gemm_block",
20   variant="gemm_block",
21   proc=BLOCK,
22   mems=[GLOBAL, GLOBAL, GLOBAL],
23   tunables={"W": 64},
24   calls=[ "clear_tile", "gemm_tile", "copy_tile"],
25   warpspecialize=True,
26   pipeline=3
27 )
```

Fig. 40. Block-level GEMM in Cypress with its associated mapping, from Yadav et al. [357]. The `launch` keyword invokes a new task; the version invoked depends on the mapping specification. The `gemm` on line 14, for example, is mapped to the tile-level `gemm_tile` task on line 24.

4 A Taxonomy of GPU Formal Methods

Having discussed their alternatives at length, we now return focus to CUDA and OpenCL. These languages' extensive ecosystems and canonization by chip manufacturers mean they are still widely used, both directly by programmers and as compilation targets for many of the other languages we have discussed. However, their low-level programming model and basis in C++ mean they lack many of the features and guardrails of modern languages, and so it falls to formal methods to fill the gap. In this section, we discuss Figure 41, which presents a taxonomy of existing GPU static analysis tools across various dimensions, including methodology, languages and issues targeted, purpose, and limitations. For space, a number of abbreviations are used: in the **Issues** column, *Bank* denotes bank conflicts, *Coal* denotes memory coalescing, *Corr* denotes functional correctness, *Det* denotes determinism, *Div* denotes thread or warp divergence, *Eq* denotes kernel equivalence checking, *Mem* denotes memory safety, *Race* denotes data races, *Sync* denotes barrier synchronization, and *Term* denotes termination. In the **Purpose** column, *verification* indicates the tool is proving the absence of bugs (or conformance to a specification), *bug-finding* indicates the tool is identifying the presence of bugs (or violations of a specification), and *cost* indicates the tool is quantifying the performance cost of bugs. We also identify gaps in the existing work and identify opportunities for future research.

Tool	Methodology	Target	Purpose	Issues	Limitations
PUG [193, 194]	Model Checking	CUDA	Bug-finding Verification	Bank Corr Race Sync	Only targets races in SHMEM; verifies functional correctness only, scaling poorly with increasing numbers of threads
KLEE-CL [73]	Symbolic Execution [172]	OpenCL	Bug-finding	Eq Race	Non-parametric in kernel configuration
Leung et al. [191]	Test Amplification	CUDA LLVM [188]	Bug-finding	Det Race	Requires test cases for properties to be exercised
GPUVerify [23, 33, 34]	Operational Semantics in Boogie [24]	CUDA OpenCL	Verification	Corr Div Race Sync	Non-parametric; unsound for certain CUDA features; requires blocks to have one warp
GKLEE [65, 195, 197]	Symbolic Execution	CUDA	Bug-finding	Bank Coal Div Race Sync	Makes unsound assumptions about the GPU's weak memory model
SESA [198]	Symbolic Execution	CUDA	Bug-finding	Race	Limited to data-races only
VerCors [14, 37]	Hoare Logic [144] Concurrent Sep. Logic [258]	OpenCL	Verification	Corr Race Sync	High annotation burden [340]; unsound assumptions about race-freedom
WEFT [295]	Symbolic Execution	PTX	Verification	Race Sync	Only handles warp-specialized code; predates PTX spec. [210]
CIVL [297]	Symbolic Execution Model Checking	CUDA	Verification	Corr Eq Mem Race Sync	No support for barriers or atomics [340]
GPUCSL [18]	Concurrent Sep. Logic Information Flow [86]	OpenCL	Verification	Corr Sync	Inherits high annotation burden from VerCors
Vericuda [179–181]	Hoare Logic	CUDA	Verification	Corr	Requires data race freedom as a precondition [340]
ESBMC-GPU [223, 266]	Model Checking	CUDA	Bug-finding Verification	Corr Mem Race	Can only verify correctness properties with user annotations
Ketema and Donaldson [167]	Term Rewriting	CUDA OpenCL	Verification	Term	Requires data-race freedom as a precondition
GPUDrano [11]	Abstract Interpretation [80]	CUDA	Bug-finding	Coal	Assumes warp-synchronicity
Alur et al. [12]	Abstract Interpretation Symbolic Execution	CUDA	Verification	Block-size indep.	Requires sync-free, race-free input (no shared memory); assumes warp-synchronicity
Xing et al. [354]	Model Checking	PTX	Verification	Race	Unsoundly unrolls loops [340]
CUDA au Coq [105]	Machine Validation	PTX	Verification	Corr Term	Requires manual proofs in Rocq
RaCUDA [226, 227]	Hoare Logic Quantitative Logic	CUDA	Cost	Bank Coal Div	Cannot handle thread-divergent loops; assumes warp-synchronicity
Faial [70]	Memory Access Protocols	CUDA	Bug-finding	Race	Reports benign races as errors; does not support atomics
FaialAA [203]	Memory Access Protocols	CUDA	Bug-finding Verification	Race	True Positives guarantee requires control-data independence
Volta [92]	Symbolic Execution	PTX	Verification	Eq Race Sync	Does not support async or warpgroup level operations
Dartagnan [333]	Model Checking	PTX SPIR-V [326]	Bug-finding	Mem	Only checks memory consistency
Pico [36]	Memory Access Protocols Relational Cost Analysis [67]	CUDA	Cost	Bank Coal Div	Certain loops and conditional patterns cause cost to be overestimated

Fig. 41. A taxonomy of formal methods techniques for GPU programs, ordered chronologically. The heavy line separates analyses developed before (above) and after (below) the release of the Volta architecture. The **Issues** column describes which common tasks or problems are targeted by each tool. The **Purpose** column describes what the tool is trying to achieve.

4.1 Early Bug-Finding Approaches

One of the earliest tools for analyzing GPU programs was **PUG** [193], which was released shortly after the Tesla architecture and used SMT-based model checking to find data races, bank conflicts, and synchronization errors in CUDA kernels, as well as verify user-provided assertions. PUG used standard techniques for model checking sequential programs, but extended them with new ways to encode thread interleavings and barrier semantics as SMT formulae; particularly important was the insight that detecting race conditions requires reasoning only about an arbitrary pair of threads, rather than every possible combination thereof.

PUG is an early exemplar of a number of common trends in the analysis of GPU kernels; many of its simplifying assumptions (such as the two-threaded approach to races), techniques, and limitations are endemic in later work. **ESBMC-GPU** [223, 266], for example, was a GPU-focused extension to the Efficient SMT-Based Context-Bounded Model Checker (ESBMC) [77] framework—originally designed for multi-threaded CPU programs—that built upon PUG's ideas and used many of its techniques, but applied them to new problem domains like memory safety.

A key challenge faced by static analysis tools for GPUs is managing the launch configuration of the kernel being analyzed: this configuration can vary across a number of factors—including number of blocks, threads per block, and dimensionality of both threads and blocks—all of which can affect the properties under analysis. PUG's initial solution was simply to require the user to specify the launch configuration at each invocation of the tool, a *non-parametric* approach that limited the class of bugs it could identify. If the tool reported no bank conflicts on a kernel launched with two threads, for example, this indicated nothing about whether bank conflicts might be present in the kernel if launched with three threads. The non-parametric approach also adversely impacted PUG's performance; it frequently timed out when analyzing kernels with as few as four threads [197], and thus did not scale to checking most functional correctness properties of interest to programmers. An alternative *parametric* approach was implemented in a later extension to PUG, called PUG$_{para}$ [194], allowing it to reason about kernels agnostically to their particular configuration and drawing from a parallel stream of contemporaneous work on symbolic execution.

The initial work in that stream was **KLEE-CL** [73], an extension of the KLEE [47] bug-finding tool, which used symbolic execution [172] techniques to find data races and behavioral mismatches between kernels and the C++ code from which they were generated. KLEE had previously been ported to the SIMD model in the form of KLEE-FP [72]; KLEE-CL further extended those ideas to SIMT by targeting OpenCL. Symbolic execution allows tools to explore all the possible execution paths through a program by evaluating code on *symbolic inputs*. Initially, these inputs are unconstrained, but as evaluation proceeds, assumptions and constraints on input values are collected and propagated through the analysis. Whenever input-dependent control is encountered, the analysis forks and checks all branches, recording assumptions about the input reflecting the control path taken.

Also building on KLEE was **GKLEE** [195], which used so-called "concolic" (concrete + symbolic) execution [48, 113, 292] to search for a wide variety of common kernel bugs. Concolic execution modifies the pure symbolic approach by holding some inputs concrete, its main advantage being that the process of searching for a bug also produces a test case demonstrating it. Unlike many other data-race detection tools, even including many released after Volta, GKLEE's analysis of data races is quite prescient, anticipating the arrival of Independent Thread Scheduling by handling both inter- and intra-warp data races. GKLEE does, however, assume the existence of a *canonical schedule* [65]—a sequential interleaving of threads that is equivalent in behavior to the parallel execution of the program—to order its symbolic evaluation; later work describing GPUs' weak memory models [10, 303] would show this assumption to be unfounded on most architectures.

Like PUG before it, GKLEE was also originally conceived as a non-parametric tool, although it was able to scale to a far higher number of threads—around 2000. GKLEE's approach to parametric analysis came in the form of GKLEE$_p$ [197], which developed the idea of *parametric flows*, also used later by PUG$_{para}$. Parametric flows rely on the assumption that most GPU kernels are behaviorally symmetric (i.e., most threads do similar things), a consequence of the control-data independence property we discussed in Theme II. Thus, instead of analyzing threads fully arbitrarily, analyses can sort threads into equivalence classes based on divergence behavior; the data-race detection problem can then be reduced to finding races between arbitrary threads within each class and between representatives of each class, as depicted in Figure 42. Whereas previous concolic execution tools needed to use concrete values for thread IDs, parametric flows allow IDs to be held symbolic during evaluation, further reducing the number of threads that must be considered. A subsequent tool called **SESA** [198] scaled the parametric approach for data-race analysis by using information flow to combine equivalence classes that only differ on non-control data. Its authors do note, however, that the parametric approach to symbolic execution does not scale well to complex functional correctness properties, as symbolic thread IDs require many manual annotations on the part of the programmer. Indeed, we can see in Figure 41 that few symbolic execution tools after GKLEE attempt to target functional correctness.

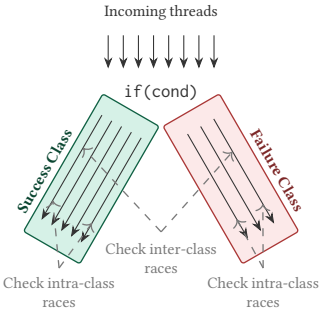


Fig. 42. Parametric flows handling an `if` statement. Races are checked between pairs of arbitrary threads within the classes of threads that succeed and fail the conditional check and between representatives of the two classes.

Even with a parametric approach, concolic execution still suffered another limitation: it was unable to handle programs that used *atomic operations* [239], which are guaranteed to execute without interference from other threads. The interaction between atomics and synchronization barriers had previously been dismissed as too complex for analysis, but an extension to GKLEE called GKLEE$_{atm}$ [65] pioneered *conflict-directed delay-bounding* as a technique to find canonical schedules even in the presence of atomics. Given a naïve schedule that does not consider atomics, the delay-bounding approach records instances of conflicts between the naïve schedule and any atomic operations encountered during evaluation. These conflicts are then used to delay threads to generate candidate schedules for the kernel; an error encountered by any such schedule is a potential bug in execution.

A separate stream of early research used information flow to perform *test amplification* [191]—a technique that expands the coverage of individual tests by extending their results to entire classes of program inputs—on CUDA kernels. The technique works like so: first, a user runs a test to dynamically check whether a given property of a program holds for some inputs. Then, information flow reasoning is used to determine the set of inputs to the program that can actually influence the property under test (also called the *property-integrity* inputs). The test result can then be extrapolated to any other execution with equal property-integrity inputs—any other inputs to the program can differ and the property will still hold or not hold as in the original test. Test amplification is effective on GPU kernels due to control-data independence; configuration data like thread IDs or tensor dimensions are typically property-integrity inputs, while tensor contents are not.

4.2 Early Verification Approaches

While tools like PUG and GKLEE focused on finding bugs in incorrect kernels, other work focused on verifying the absence of bugs in correct ones. An early such tool was **GPUVerify** [33, 34], which aimed to guarantee the absence of data races and thread and barrier divergence in CUDA and OpenCL kernels, while also checking some functional correctness properties. GPUVerify's approach is based on *synchronous, delayed visibility* (SDV) semantics, in which kernels are executed by a single,

synchronous warp whose threads are sequentially interleaved. To express the possibility of data races, writes to memory are delayed until the warp reaches a synchronization barrier, after which writes are populated to their respective locations in memory; in this semantics, reading a recently written value before synchronizing (i.e., a data race) causes a run-time error. Betts et al. [33] argued that SDV was a valid model of GPU evaluation for Tesla-generation microarchitectures, but the evolution of GPUs to include Independent Thread Scheduling (breaking the assumption of warp-synchronous execution) and warpgroup-level collective operations (making a single-warp semantics unviable) means that the semantics is less practical for modern hardware.

To analyze a kernel, GPUVerify first infers loop invariants for it, then passes the program and a description of its evaluation in SDV to Boogie [24], an intermediate verification language (IVL) using the Z3 [85] SMT solver, to determine whether the invariants hold. To avoid a combinatorial explosion of SMT formulae, GPUVerify employs a similar technique to PUG: reducing the number of threads under consideration to just two. This two-thread reduction is sound relative to the SDV semantics, but is subject to the same caveats about its usability after Volta. Among all the static verifiers considered in their survey, van den Haak et al. [340] evaluate GPUVerify as the most practical, citing its ability to automatically infer loop invariants in many cases. They also note, however, that its analysis is not sound on all CUDA features, likely due to its reliance on SDV and the two-thread reduction. Later work by Bardsley and Donaldson [23] extended GPUVerify to handle atomics, but unlike GKLEE$_{atm}$, which could search the space of canonical schedules heuristically, GPUVerify must reason about thread interleavings abstractly to remain sound, leaving the results of synchronized reads to shared memory arbitrary and yielding many false positives.

VerCors [37] was another approach to verification that used Hoare logic [144] and *permission-based separation logic* [43] to prove a variety of correctness properties about OpenCL kernels. Permission-based separation logic extends concurrent separation logic [258] by adding permissions into the annotation language; functional correctness properties must have the appropriate read or write permissions to be verified. Because the logic requires that only one thread can hold the write permission for a memory location and that threads can only hold read permissions if none can write, any program that verifies is inherently data-race free. To use VerCors, users describe the initial permissions associated with a kernel when it is launched, and at each barrier synchronization specify pre- and post-conditions about program state and the distribution of permissions across threads. VerCors is identified by van den Haak et al. [340] as the most versatile verification tool for functional correctness, but they note that it imposes a heavy annotation burden on users.

Like GPUVerify, VerCors was also subsequently extended with support for atomics [14], this time by adding a distinction between two kinds of permissions: kernel resource invariants, which are properties that hold for entire kernels, and group resource invariants, which hold only for each work-group (block). Each barrier's kernel and group specifications can be derived by the universal separating conjunction over the specifications of all the threads in the kernel and in the group, respectively. This extension allowed VerCors to verify programs using barriers and atomics, but later work by Asakura et al. [18] identified a critical issue: VerCors assumes that thread ID independence (the condition that control flow and the values of variables are independent of thread ID) is a sufficient condition for the absence of barrier divergence. However, this is only true in the absence of data races; one needs both thread ID independence and data race freedom to get barrier divergence freedom. Asakura et al. [18] also note that VerCors's formalization of its separation logic is nonstandard, as it lacks a frame rule, making it difficult to use standard techniques to prove its soundness. They rectify both these issues with **GPUCSL**, which builds upon VerCors, proves the corrected logic's soundness in Rocq, and uses an information flow type system to identify when data is thread-ID independent.

Another program logic for CUDA was developed by Kojima and Igarashi [179, 180] and later implemented in the **Vericuda** [181] verification tool. Rather than building on separation logic,

Vericuda's approach extends Hoare logic directly, adding a new *mask* element specifying the set of active threads to the usual Hoare triple. Users decorate their programs with assertions and loop invariants, and Vericuda encodes these as input to an SMT solver; for the class of programs it can handle, Vericuda can efficiently and parametrically verify functional correctness properties. However, its underlying Hoare logic makes a number of simplifying assumptions that limit its applicability. In particular, the logic requires that variables used in the guards of `while` loops never appear in the bodies of those loops—if they did, one thread could impact the success of the guard for another. Additionally, Vericuda's semantics assumes that all threads execute in lockstep and thus cannot handle Independent Thread Scheduling. Kojima and Igarashi [180] prove their lockstep semantics equivalent to a more general interleaved semantics in the absence of data races, and so Vericuda can only soundly verify post-Volta kernels when they are entirely devoid of races [340].

While verifiers like GPUVerify and VerCors chose to focus on a wide class of common GPU bugs, others elected to either broaden or narrow their focus. The Concurrency Intermediate Verification Language, or **CIVL** [297], aimed to be a common intermediate representation for verifying concurrency properties across multiple different parallel architectures, rather than just the GPU. Languages like CUDA, Chapel, and OpenMP are compiled to CIVL's IR, and model checking and symbolic execution are used to verify the absence of many concurrency bugs.

WEFT [295], on the other hand, was a symbolic execution tool that focused solely on warp-specialized code. WEFT was unusual among early tools in that it performed its analysis on PTX—the ISA to which CUDA compiles—rather than on CUDA itself, which allowed it to support operations that did not exist in CUDA. In particular, WEFT analyzes programs that use *named barriers*, PTX primitives that synchronize at finer granularity than the full-block synchronization of `__syncthreads` and that are necessary for warp specialization. WEFT posits a formal operational semantics for PTX and evaluates programs symbolically, leaving abstract any actual data values but computing control flow and array accesses, and hence requires kernels to be control-data independent. The choice to analyze PTX would influence later tools that also wished to support lower-level operations than could be expressed in CUDA, but it is worth noting that WEFT predates the official PTX memory consistency model, which was released alongside Volta in 2017, and thus its model of PTX's semantics may not reflect the reality of how the ISA behaved at the time.

Few early formal methods approaches considered the termination of kernels—particularly important on GPUs because the result of a kernel cannot be accessed by its host unless the kernel has terminated. Ketema and Donaldson [167]'s tool was the first to do so, building on KITTeL [100], a termination analyzer that translates single-threaded C programs into a term rewriting system from which it can automatically synthesize a termination proof. Their approach deals with the multi-threaded nature of GPU programs by only considering the behavior of a single arbitrary thread, assuming control-data independence to avoid considering kernels where control flow depends on values in shared memory. Ketema and Donaldson [167] prove that if an arbitrarily chosen thread can be shown to terminate, then so must every thread, although like other methods such as Vericuda that use an interleaving semantics, they require data-race freedom as a prerequisite to analysis.

4.3 Formal Methods After Volta

Much as it had for programming languages, Volta changed the landscape for formal methods on the GPU, both invalidating old assumptions and laying the groundwork for new approaches. The advent of Independent Thread Scheduling meant that tools could no longer assume warp-synchronicity, and supporting Tensor Cores would require them to analyze more than one or two threads at a time. On the other hand, the release of an official consistency model for PTX [210, 250]—spurred by research into GPUs' weak memory models [10, 303]—opened the door to analyzing NVIDIA's ISA directly.

Despite the arrival of ITS, however, much of the early effort following the release of Volta still assumed warp-synchronous execution. Examples of such work included **GPUDrano** [11], an abstract-interpretation [80]-based tool that identifies uncoalesced memory accesses, and an analysis by Alur et al. [12] to verify that the behavior of CUDA kernels is independent of the block size with which they are launched. As Kojima and Igarashi [180] argued earlier, assuming warp-synchronicity restricts sound analysis to kernels without data races, and the latter of these two tools is further limited to kernels that make no use of synchronization or shared memory.

A particularly notable new tool for identifying data races in GPU kernels was **Faial** [70], which built upon the SDV semantics of GPUVerify to present *memory access protocols*, a new abstraction of array accesses that eschews information about what is being read or written to memory, focusing instead on where the reads and writes are happening. For example, `A[tid + i] = B[tid * X + j]` would translate to the protocol `read[tid * X + j]; write[tid + i]`. Faial encodes these protocols as SMT formulae, which it can use to identify whether a race is possible in a given protocol; like many other tools before it, by abstracting away the details of what data is being read from memory, Faial implicitly assumes control-data independence. One consequence of this, however, is that Faial is unable to distinguish between data races that attempt to write different values to the same location (i.e., harmful ones), and races that write the same value to a location (i.e., benign ones). Faial was later extended by **FaialAA** [203], which addressed this issue, added support for atomics, and reduced its rate of false alarms. In particular, FaialAA is able to prove a True Positives theorem (i.e., that all reported data races are real) for control-data independent kernels.

Separately, a strand of research arose to measure the *cost* of kernels, in particular any efficiency bugs they may contain. The first such tool was **RaCUDA** [226, 227], which developed a program logic based on *amortized resource analysis* [146] to quantify the cost of CUDA kernels. RaCUDA infers symbolic resource bounds by assigning a "potential" to states of computation and models how this potential changes during evaluation—each operation in a program consumes a certain amount of potential, with inefficient events like bank conflicts and thread divergence requiring more. RaCUDA then upper-bounds the total execution time of a kernel by computing its *work* (the total cost of all operations of a kernel across all threads) and *span* (the maximum cost of any single warp); the total cost of a kernel is its work divided by the maximum warps the GPU can schedule at a time, plus its span. Later work on **Pico** [36] addressed a few of RaCUDA's limitations; in particular, it could not handle thread-divergent loops and overestimated the cost of thread-divergent conditionals. Pico's approach is primarily based on Faial's memory access protocols but also uses a *relational cost analysis* [67], which compares the relative costs of two programs. In Pico's case, it relates each input to a sequential translation of itself to upper-bound its cost. RaCUDA's and Pico's implementations also both assume control-data independence to make their analyses more tractable.

Spurred by its new memory model, a number of other projects began exploring the potential of analyzing PTX directly. Work by Xing et al. [354] used model checking to identify data races; like WEFT, their work supported inline PTX instructions, which were most commonly used by advanced GPU developers but were becoming more and more essential with the release of new hardware features without direct CUDA support. It is worth noting, however, that Xing et al. [354]'s tool unsoundly unrolls loops as part of its analysis [340]. **CUDA au Coq** [105] was another PTX-level approach, embedding the ISA's semantics in Rocq and providing a framework for proving functional correctness properties and termination.

Another PTX analysis tool of interest is **Volta** [92]—not to be confused with the microarchitecture—a symbolic-execution-based verifier for machine learning kernels. Volta aims to prove the functional correctness of such kernels, but it has a particular focus on checking equivalence between optimized ML kernels and their unoptimized forms, motivated by a recent rise in LLM-driven program optimization [25, 346]; to make this problem decidable, Volta limits itself to control-data independent kernels

and analyzes only one thread block for correctness. Notably, Volta differs from other work in this area by supporting Tensor Core operations—in fact, it is the only tool among all those we surveyed that does so. Even then, though, its support only extends to the Ampere generation of Cores. As we have seen, the vast majority of GPUs' computational power now lives in their Tensor Cores and equivalents [61], so tools that do not support them are incapable of analyzing performant kernels. Moreover, as discussed in Theme I, correctness is of little use without efficiency, so a large gap in the current formal methods landscape, and therefore a large opportunity for future work, lies in tools that can handle modern features like Tensor Cores. This is especially true of Hopper and Blackwell's asynchronous hardware units, as no tools yet exist at time of writing that can reason about those.

Some tools were also developed to reason about general properties of GPU hardware and languages, rather than about individual programs. Rather than kernels, **Dartagnan** [333] analyzes the memory consistency models of PTX and SPIR-V [326] (OpenCL's low-level bytecode) and has so far identified a handful of existing bugs in both. Work by Valpey et al. [339] developed an SMT-based formal model for Volta- and Ampere-generation Tensor Cores with which they were able to identify ways in which the Cores differ between architectures and correct mistaken assumptions about their behavior.

SIMT-Step execution [64], meanwhile, proposed a new operational semantics for GPUs, one where the granularity below which thread execution is synchronous is configurable during analysis. This flexibility allows the semantics to be portable across different GPUs, where the sizes and synchronization behavior of thread groups (e.g., a warp or a wavefront) may differ. SIMT-Step is also able to track which threads enter each basic block in a program's control flow graph, allowing it to describe the collective execution of Tensor Cores.

4.4 Dynamic Approaches

A number of dynamic analyses for GPU programs also exist; full enumeration and in-depth discussion of these is out of scope for this survey, but we list a representative sampling as a starting point for interested readers. BARRACUDA [95], CURD [264], HAccRG [147], HiRace [159], GRace [368], GMRace [369], LD [196], NVIDIA's Compute Sanitizer [241], Oclgrind [271], iGuard [164], ScoRD [165], and the work of Boyer et al. [44] detect data races. Simulee [353], CURD, BARRACUDA, and the Compute Sanitizer can also be used to find sources of barrier divergence, and the Compute Sanitizer, cuCatch [319], and Boyer et al. [44]'s work also identify inefficient or invalid memory accesses. Hayes et al. [138]'s work finds information leaks. NVIDIA's Nsight [247] suite is commonly used to profile kernels and analyze their performance. Guardian [263] enforces safe memory sharing between multiple kernels running on the same GPU. GPU-FPX [202] identifies floating-point bugs in kernels that use Tensor Cores. Some dynamic tools like GRace [368] use static techniques to prune cases from their analysis when they can verify that they cannot cause a data race. We also direct readers towards van den Haak et al. [340]'s survey of formal methods for GPU programming for further discussion of both static and dynamic analyses.

5 Conclusion

We have surveyed the research on language design and formal methods for GPUs, tracing the genealogy of the former, tracking how ideas spread between languages and influence the development of new paradigms, and taxonomizing the latter, describing the various approaches to kernel analysis and identifying gaps in the literature. Along the way, we discussed major themes of GPU programming, identifying three of particular salience to programming languages and formal methods researchers.

The first theme, that performance is critically important on the GPU, manifested throughout our discussion of its programming languages; efficiency is a key factor in language design, and subpar performance frequently motivates further research and evolution. This theme also points to a major gap in current formal methods research: the lack of static analysis tools that support modern

hardware features like Tensor Cores, TMAs, and TMEM. Careful use of these features is required for peak performance, and tools that cannot handle them are thus limited in their applicability.

The second theme, that kernel logic is often simple while its optimization is complex, has spawned an ecosystem of programming languages that expose more control over hardware than is typical. GPUs have a vast and complex optimization space, and searching it is hard; many languages opt to target specialized domains to narrow the search space, while others (most often, USLs) prefer to shift more of the burden of optimization onto the programmer. Another consequence of this theme is the widespread property of control-data independence, which is commonly used as a simplifying assumption by formal methods tools. While this property does not hold in all domains, it is common in particularly important ones like graphics and machine learning.

The third theme, that GPU hardware is diverse and constantly evolving, results in a high rate of kernel churn, as developers implement and reimplement their code across various architectures to improve performance. The pursuit of performance portability has thus been a common factor in language design, although some argue that such efforts are futile and that languages would be better served by giving programmers more tools to control the optimization and compilation of their code.

We have also seen this last theme affect the design and usability of both programming languages and formal methods tools, as new hardware features and backwards-incompatible changes have violated many old assumptions and required new approaches to handle them. The move from warp-synchronous execution to Independent Thread Scheduling has posed a challenge for formal methods, for example, while the evolution of GPUs from bulk-synchronous scalar processors to an asynchronous tensor model has upended language design. In particular, Triton and the tile-based programming model were an elegant design for the synchronous, warp-level generation of Tensor Cores, but we are still searching for the right solution for asynchronous Cores and whatever future heterogeneous units come afterward—maybe this will be smarter compilation for Triton or more powerful types for SIMT, maybe it will be an adaptation of other high-performance computing paradigms like task-based programming, or maybe it will be something completely new.

A Polyhedral Compilation

Polyhedral compilation originated outside the GPU space in the compilation of *loop nest* [102] programs, whose primary structure is a series of nested `for` loops. The polyhedral model envisions the iteration domain of a loop nest of depth n as an n-dimensional polyhedron, or *polytope*. Each statement within a loop nest corresponds to a polytope, and the coordinates of points within that polytope correspond to the values of the index variables of the loop nest. A program can be perfectly nested (i.e., all its statements appear in the body of the innermost loop) or imperfectly nested (i.e., some statements appear at higher levels of the nest or multiple loops appear sequentially within an outer loop). So, for example, the assignment on line 3 of Figure 43 can be envisioned as a triangle in 2D space, with base and height equal to $N-1$. Each pair of i and j that are valid loop indices (that is, when $i < N$ and $i \leq j$) corresponds to a point within this triangle, while the iteration bounds of the loop nest define the edges of the statement's *iteration space polytope*.

```
1 for (int i = 0; i < N; i++) {
2   for (int j = 0; i <= j; j++) {
3     arr[i][j] = arr[i][j] * 2;
4   }
5 }
```

Fig. 43. A simple loop nest.

The polyhedral model is a convenient way of reasoning about program optimizations that restructure loop nests, such as tiling, because such optimizations can be represented as affine functions on the polytopes of a loop nest that preserve key geometric properties and dependencies. Polyhedral compilation has a long history that we cannot cover in detail; we focus here on its applications to GPUs and direct readers to Thangamani et al. [323]'s survey for further information, noting particularly Courant and Leroy [79]'s work on formally verifying the correctness of a simple polyhedral compiler that may be of interest to a PL audience.

```
// a length 2 vector, encoded as a 2x1 matrix
local(2, 1)
// a 2x3 matrix, distributed over the same
// address in 6 adjacent threads instead of 6
// adjacent memory addresses
spatial(2, 3)
// a 2x2 matrix via layout composition
local(2, 1).local(1, 2)
// A Tensor Core input layout
local(2, 1).spatial(8, 4).local(1, 2)
```

Fig. 44. Example layouts in Tilus, from Ding et al. [90].

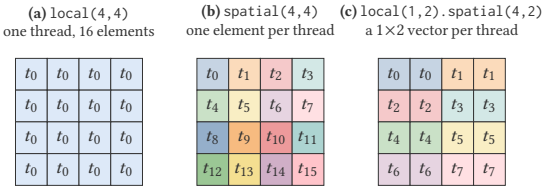


Fig. 45. Three layouts assigning the elements of a 4×4 tile to threads, in the notation of Figure 44; each cell is labeled by the thread that holds it in its registers. Composition (c) yields the distributed fragments: each thread holds a small contiguous vector, and consecutive threads hold adjacent vectors.

The polyhedral model is a natural fit for programs that iterate deeply over multidimensional structures like images and tensors; consider, for instance, how the sequential matrix multiplication algorithm we saw in Figure 12 is a doubly nested loop, implicitly wrapped in a third parallel loop following the SIMT intuition that kernels execute inside a "parallel `for` loop" whose indices are thread IDs. In addition, the polyhedral model requires the control flow of its loop nests to be statically determinable; that is, loop bounds and conditional tests must be affine functions of compile-time constants and the iteration variables of surrounding loops. This is essentially the same requirement as the control-data independence property already common in GPU kernels, so it is no surprise that the ideas behind polyhedral compilation were quickly adapted to the SIMT model. One of the first such adaptations was that of Baskaran et al. [26], which built on the existing PLuTo [41, 42] polyhedral compiler to automatically unroll and tile loops to improve factors like data locality and parallelism. However, one cannot achieve optimal performance naïvely applying parallel CPU techniques to GPUs; as discussed in Theme II, few optimizations are uniformly beneficial in the GPU space, so Baskaran et al. [26]'s compiler must search for the best parameters for tiling dimensions and loop unrolling depth to balance concerns like parallelism with register pressure. Their compiler also attempts to automatically coalesce global memory accesses and eliminate bank conflicts by reordering iteration, rearranging independent accesses, or inserting shared memory indirection as needed.

Many of the languages we discuss in this survey rely on the polyhedral model for their compilation, whether for parallelizing sequential code or for loop transformations specific to particular domains.

B Algebraic Layouts

CuTeDSL, Graphene, and Tilus feature systems of algebraic tensor layouts designed to streamline tile programming by abstracting tile shapes and giving operations over them an elegant compositional theory. The *layout* of a tensor is a bijective map between the logical indices of its elements and the physical addresses in memory where they are stored; the most basic example of a layout represents an arbitrarily dimensioned tensor as an array in memory by implicitly multiplying by the tensor's dimensions during access. More complex layouts, however, have their origins in research on sparse tensor algebra libraries like Kokkos [93, 94] and Taco [174] and were later reified as programming languages features by PyTorch [261] and TensorFlow [296]. By giving up on constant-time access, one can represent tensors compactly and express operations like scaling or transposition as a modification of the layout (quick to compute) rather than a transformation on the data (slow, scaling with tensor size). The CUTLASS library incorporated a system of layouts called CuTe [245] to support Tensor Core programming, allowing programmers to express programmatically how their tensors are distributed across both memory and compute resources. In Tilus, these two methods of distribution are reflected by two primitive kinds of layouts, local (i.e., the tensor exists within a single thread) and spatial (i.e., the tensor is distributed across threads), and Ding et al. [90] show that many common layouts can

be constructed via composition of these primitives. Some examples of Tilus layouts are shown in Figure 44; the layout on line 10 exactly describes the shape expected by 16×8 Tensor Core instructions. Figure 45 visualizes how local and spatial layouts, and their compositions, assign the elements of a tile to threads. CUTLASS's and CuTeDSL's layouts [55] likewise possess a suite of operators to compose them—later work gives a full category-theoretic treatment of their theory [50]—and later influenced an extension adding linear algebraic layouts to Triton [371].

References

[1] Martín Abadi, Anindya Banerjee, Nevin Heintze, and Jon G. Riecke. 1999. A Core Calculus of Dependency. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Antonio Texas USA, 1999-01). ACM, 147–160. doi:10.1145/292540.292555

[2] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: A System for Large-scale Machine Learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation* (Savannah, GA, USA) *(OSDI'16)*. USENIX Association, USA, 265–283.

[3] Hamdy Abdelkhalik, Yehia Arafa, Nandakishore Santhi, and Abdel-Hameed Badawy. 2022. Demystifying the Nvidia Ampere Architecture through Microbenchmarking and Instruction-level Analysis. (2022). https://arxiv.org/abs/2208.11174v1

[4] Andrew Adams, Karima Ma, Luke Anderson, Riyadh Baghdadi, Tzu-Mao Li, Michaël Gharbi, Benoit Steiner, Steven Johnson, Kayvon Fatahalian, Frédo Durand, and Jonathan Ragan-Kelley. 2019. Learning to Optimize Halide with Tree Search and Random Programs. *ACM Transactions on Graphics* 38, 4 (Aug. 2019), 1–12. doi:10.1145/3306346.3322967

[5] Adobe. 2025. Inside Halide, the open source language engineers use to make imaging tools faster. https://research.adobe.com/news/inside-halide-the-open-source-language/ Accessed: 2026-06-20.

[6] Advanced Micro Devices, Inc. 2025. Introducing AMD CDNA 4 Architecture: Breakthrough AI and HPC Acceleration with Enhanced AI Capabilities, Advanced Precisions, and High Efficiency. (2025). https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/white-papers/amd-cdna-4-architecture-whitepaper.pdf Accessed: 2026-06-01.

[7] Advanced Micro Devices, Inc. 2025. RDNA4 Instruction Set Architecture Reference Guide. https://docs.amd.com/v/u/en-US/rdna4-instruction-set-architecture Accessed: 2026-07-25.

[8] Advanced Micro Devices, Inc. 2026. Introduction to the HIP programming model. https://rocm.docs.amd.com/projects/HIP/en/latest/understand/programming_model.html Accessed: 2026-05-29.

[9] Alex Aiken, Michael Bauer, and Sean Treichler. 2014. Realm: An Event-Based Low-level Runtime for Distributed Memory Architectures. In *2014 23rd International Conference on Parallel Architecture and Compilation Techniques (PACT)*. 263–275. doi:10.1145/2628071.2628084

[10] Jade Alglave, Mark Batty, Alastair F. Donaldson, Ganesh Gopalakrishnan, Jeroen Ketema, Daniel Poetzl, Tyler Sorensen, and John Wickerson. 2015. GPU Concurrency: Weak Behaviours and Programming Assumptions. *ACM SIGARCH Computer Architecture News* 43, 1 (May 2015), 577–591. doi:10.1145/2786763.2694391

[11] Rajeev Alur, Joseph Devietti, Omar S. Navarro Leija, and Nimit Singhania. 2017. GPUDrano: Detecting Uncoalesced Accesses in GPU Programs. In *Computer Aided Verification*. Springer International Publishing, Cham, 507–525. doi:10.1007/978-3-319-63387-9_25

[12] Rajeev Alur, Joseph Devietti, and Nimit Singhania. 2018. Block-Size Independence for GPU Programs. In *Static Analysis* (Cham, 2018). Springer International Publishing, 107–126. doi:10.1007/978-3-319-99725-4_9

[13] Amazon Web Services. 2026. Amazon EC2 G4 Instances. https://aws.amazon.com/ec2/instance-types/g4/ Accessed: 2026-07-16.

[14] Afshin Amighi, Saeed Darabi, Stefan Blom, and Marieke Huisman. 2015. Specification and Verification of Atomic Operations in GPGPU Programs. In *Software Engineering and Formal Methods* (Cham, 2015). Springer International Publishing, 69–83. doi:10.1007/978-3-319-22969-0_5

[15] Jason Ansel, Cy Chan, Yee Lok Wong, Marek Olszewski, Qin Zhao, Alan Edelman, and Saman Amarasinghe. 2009. PetaBricks: A Language and Compiler for Algorithmic Choice. In *Proceedings of the 30th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, Dublin Ireland, 38–49. doi:10.1145/1542476.1542481

[16] Apple, Inc. 2026. Metal Shading Language Specification. https://developer.apple.com/metal/Metal-Shading-Language-Specification.pdf Accessed: 2026-08-27.

[17] Jeremy Appleyard and Scott Yokim. 2017. Programming Tensor Cores in CUDA 9. (2017). https://developer.nvidia.com/blog/programming-tensor-cores-cuda-9/ Accessed: 2026-06-03.

[18] Izumi Asakura, Hidehiko Masuhara, and Tomoyuki Aotani. 2016. Proof of Soundness of Concurrent Separation Logic for GPGPU in Coq. *Journal of Information Processing* 24, 1 (2016), 132–140. doi:10.2197/ipsjjip.24.132

[19] Joshua Auerbach, David F. Bacon, Ioana Burcea, Perry Cheng, Stephen J. Fink, Rodric Rabbah, and Sunil Shukla. 2012. A Compiler and Runtime for Heterogeneous Computing. In *Proceedings of the 49th Annual Design Automation Conference* (San Francisco, California) *(DAC '12)*. Association for Computing Machinery, New York, NY, USA, 271–276. doi:10.1145/2228360.2228411

[20] Joshua Auerbach, David F. Bacon, Perry Cheng, and Rodric Rabbah. 2010. Lime: A Java-Compatible and Synthesizable Language for Heterogeneous Architectures. *ACM SIGPLAN Notices* 45, 10 (Oct. 2010), 89–108. doi:10.1145/1932682.1869469

[21] Riyadh Baghdadi, Jessica Ray, Malek Ben Romdhane, Emanuele Del Sozzo, Abdurrahman Akkas, Yunming Zhang, Patricia Suriana, Shoaib Kamil, and Saman Amarasinghe. 2019. Tiramisu: A Polyhedral Compiler for Expressing Fast and Portable Code. In *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 193–205. doi:10.1109/CGO.2019.8661197

[22] Manya Bansal, Daniel Sainati, Joseph W. Cutler, Saman Amarasinghe, and Jonathan Ragan-Kelley. 2026. Modular GPU Programming with Typed Perspectives. *Proc. ACM Program. Lang.* 10, PLDI, Article 212 (June 2026), 25 pages. doi:10.1145/3808290

[23] Ethel Bardsley and Alastair F. Donaldson. 2014. Warps and Atomics: Beyond Barrier Synchronization in the Verification of GPU Kernels. In *NASA Formal Methods* (Cham, 2014). Springer International Publishing, 230–245. doi:10.1007/978-3-319-06200-6_18

[24] Mike Barnett, Bor-Yuh Evan Chang, Robert DeLine, Bart Jacobs, and K. Rustan M. Leino. 2005. Boogie: A Modular Reusable Verifier for Object-Oriented Programs. In *Proceedings of the 4th International Conference on Formal Methods for Components and Objects* (Amsterdam, The Netherlands) *(FMCO'05)*. Springer-Verlag, Berlin, Heidelberg, 364–387. doi:10.1007/11804192_17

[25] Carlo Baronio, Pietro Marsella, Ben Pan, Simon Guo, and Silas Alberti. 2025. Kevin: Multi-Turn RL for Generating CUDA Kernels. arXiv:2507.11948 doi:10.48550/arXiv.2507.11948

[26] Muthu Manikandan Baskaran, Uday Bondhugula, Sriram Krishnamoorthy, J. Ramanujam, Atanas Rountev, and P. Sadayappan. 2008. A Compiler Framework for Optimization of Affine Loop Nests for GPGPUs. In *Proceedings of the 22nd Annual International Conference on Supercomputing*. ACM, Island of Kos, Greece, 225–234. doi:10.1145/1375527.1375562

[27] Michael Bauer, Henry Cook, and Brucek Khailany. 2011. CudaDMA: Optimizing GPU Memory Bandwidth via Warp Specialization. In *SC '11: Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–11. doi:10.1145/2063384.2063400

[28] Michael Bauer, Sean Treichler, and Alex Aiken. 2014. Singe: Leveraging Warp Specialization for High Performance on GPUs. In *Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Orlando, Florida, USA) *(PPoPP '14)*. Association for Computing Machinery, New York, NY, USA, 119–130. doi:10.1145/2555243.2555258

[29] Michael Bauer, Sean Treichler, Elliott Slaughter, and Alex Aiken. 2012. Legion: Expressing Locality and Independence with Logical Regions. In *SC '12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. 1–11. doi:10.1109/SC.2012.71

[30] Eli Ben-Sasson, Matan Hamilis, Mark Silberstein, and Eran Tromer. 2016. Fast Multiplication in Binary Fields on GPUs via Register Cache. In *Proceedings of the 2016 International Conference on Supercomputing* (Istanbul, Turkey) *(ICS '16)*. Association for Computing Machinery, New York, NY, USA, Article 35, 12 pages. doi:10.1145/2925426.2926259

[31] James Bergstra, Olivier Breuleux, Frédéric Bastien, Pascal Lamblin, Razvan Pascanu, Guillaume Desjardins, Joseph Turian, David Warde-Farley, and Yoshua Bengio. 2010. Theano: A CPU and GPU Math Compiler in Python. *SciPy 2010* (2010). doi:10.25080/Majora-92bf1922-003

[32] Gilbert Louis Bernstein, Chinmayee Shah, Crystal Lemire, Zachary Devito, Matthew Fisher, Philip Levis, and Pat Hanrahan. 2016. Ebb: A DSL for Physical Simulation on CPUs and GPUs. *ACM Transactions on Graphics* 35, 2 (May 2016), 1–12. doi:10.1145/2892632

[33] Adam Betts, Nathan Chong, Alastair Donaldson, Shaz Qadeer, and Paul Thomson. 2012. GPUVerify: A Verifier for GPU Kernels. In *Proceedings of the ACM International Conference on Object-Oriented Programming Systems Languages and Applications*. ACM, Tucson Arizona USA, 113–132. doi:10.1145/2384616.2384625

[34] Adam Betts, Nathan Chong, Alastair F. Donaldson, Jeroen Ketema, Shaz Qadeer, Paul Thomson, and John Wickerson. 2015. The Design and Implementation of a Verification Technique for GPU Kernels. *ACM Transactions on Programming Languages and Systems* 37, 3 (June 2015), 1–49. doi:10.1145/2743017

[35] Somashekaracharya G. Bhaskaracharya, Julien Demouth, and Vinod Grover. 2020. Automatic Kernel Generation for Volta Tensor Cores. arXiv:2006.12645 doi:10.48550/arXiv.2006.12645

[36] Gregory Blike, Hannah Zicarelli, Udaya Sathiyamoorthy, Julien Lange, and Tiago Cogumbreiro. 2026. A Modular Static Cost Analysis for GPU Warp-Level Parallelism. *Proceedings of the ACM on Programming Languages* 10, POPL (Jan. 2026), 1471–1499. doi:10.1145/3776693

[37] Stefan Blom, Marieke Huisman, and Matej Mihelčić. 2014. Specification and Verification of GPGPU Programs. *Science of Computer Programming* 95 (Dec. 2014), 376–388. doi:10.1016/j.scico.2014.03.013

[38] François Bodin, Toru Kisuki, Peter Knijnenburg, Mike O'Boyle, and Erven Rohou. 1998. Iterative Compilation in a Non-Linear Optimisation Space. *Workshop on Profile and Feedback-Directed Compilation* (03 1998).

[39] Simon Boehm. 2022. How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance: a Worklog. https://siboehm.com/articles/22/CUDA-MMM Accessed: 2026-05-29.

[40] Jeff Bolz, Ian Farmer, Eitan Grinspun, and Peter Schröder. 2003. The GPU as Numerical Simulation Engine. (03 2003). https://www.researchgate.net/publication/2551205_ÆŠThe_GPU_as_Numerical_Simulation_Engine

[41] Uday Bondhugula, Muthu Baskaran, Sriram Krishnamoorthy, J. Ramanujam, Atanas Rountev, and P. Sadayappan. 2008. Automatic Transformations for Communication-Minimized Parallelization and Locality Optimization in the Polyhedral Model. In *Compiler Construction* (Berlin, Heidelberg, 2008). Springer, 132–146. doi:10.1007/978-3-540-78791-4_9

[42] Uday Bondhugula, Albert Hartono, J. Ramanujam, and P. Sadayappan. 2008. A Practical Automatic Polyhedral Parallelizer and Locality Optimizer. *SIGPLAN Not.* 43, 6 (June 2008), 101–113. doi:10.1145/1379022.1375595

[43] Richard Bornat, Cristiano Calcagno, Peter O'Hearn, and Matthew Parkinson. 2005. Permission Accounting in Separation Logic. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Long Beach, California, USA) *(POPL '05)*. Association for Computing Machinery, New York, NY, USA, 259–270. doi:10.1145/1040305.1040327

[44] Michael Boyer, Kevin Skadron, and Westley Weimer. 2008. Automated Dynamic Analysis of CUDA Programs. (March 2008). https://www.nvidia.com/docs/IO/67190/stmcs08.pdf

[45] Ian Buck. 2009. From Brook to CUDA. https://www.nvidia.com/content/GTC/documents/1408_GTC09.pdf Accessed: 2026-05-29.

[46] Ian Buck, Theresa Foley, Daniel Horn, Jeremy Sugerman, Kayvon Fatahalian, Mike Houston, and Pat Hanrahan. 2004. Brook for GPUs: Stream Computing on Graphics Hardware. *ACM Transactions on Graphics* 23, 3 (Aug. 2004), 777–786. doi:10.1145/1015706.1015800

[47] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-coverage tests for complex Systems Programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation* (San Diego, California) *(OSDI'08)*. USENIX Association, USA, 209–224.

[48] Cristian Cadar, Vijay Ganesh, Peter M. Pawlowski, David L. Dill, and Dawson R. Engler. 2008. EXE: Automatically Generating Inputs of Death. *ACM Trans. Inf. Syst. Secur.* 12, 2, Article 10 (Dec. 2008), 38 pages. doi:10.1145/1455518.1455522

[49] D. Callahan, B.L. Chamberlain, and H.P. Zima. 2004. The Cascade High Productivity Language. In *Ninth International Workshop on High-level Parallel Programming Models and Supportive Environments, 2004. Proceedings.* 52–60. doi:10.1109/HIPS.2004.1299190

[50] Jack Carlisle, Jay Shah, Reuben Stern, and Paul VanKoughnett. 2026. Categorical Foundations for CuTe Layouts. https://arxiv.org/abs/2601.05972v1

[51] Adrián Castelló, Julian Bellavita, Grace Dinh, Yuka Ikarashi, and Héctor Martínez. 2024. Tackling the Matrix Multiplication Micro-kernel Generation with Exo. In *Proceedings of the 2024 IEEE/ACM International Symposium on Code Generation and Optimization* (Edinburgh, United Kingdom) *(CGO '24)*. IEEE Press, 182–192. doi:10.1109/CGO57630.2024.10444883

[52] Adrián Castelló, Hèctor Martínez, Sandra Catalán, Jie Lei, Yuka Ikarashi, Grace Dinh, Francisco D. Igual, and Enrique S. Quintana-Ortí. 2025. Portable, High Performance Matrix Multiplication Micro-Kernels for RISC-V with ExO. In *2025 33rd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing (PDP)*. 25–32. doi:10.1109/PDP66500.2025.00013 ISSN: 2377-5750.

[53] Bryan Catanzaro, Michael Garland, and Kurt Keutzer. 2011. Copperhead: Compiling an Embedded Data Parallel Language. In *Proceedings of the 16th ACM symposium on Principles and Practice of Parallel Programming*. ACM, San Antonio TX USA, 47–56. doi:10.1145/1941553.1941562

[54] Bryan Catanzaro, Alexander Keller, and Michael Garland. 2014. A Decomposition for In-Place Matrix Transposition. *SIGPLAN Not.* 49, 8 (Feb. 2014), 193–206. doi:10.1145/2692916.2555253

[55] Cris Cecka. 2026. CuTe Layout Representation and Algebra. arXiv:2603.02298 doi:10.48550/arXiv.2603.02298

[56] Hassan Chafi, Zach DeVito, Adriaan Moors, Tiark Rompf, Arvind K. Sujeeth, Pat Hanrahan, Martin Odersky, and Kunle Olukotun. 2010. Language Virtualization for Heterogeneous Parallel Computing. *SIGPLAN Not.* 45, 10 (Oct. 2010), 835–847. doi:10.1145/1932682.1869527

[57] Hassan Chafi, Arvind K. Sujeeth, Kevin J. Brown, HyoukJoong Lee, Anand R. Atreya, and Kunle Olukotun. 2011. A Domain-Specific Approach to Heterogeneous Parallelism. *ACM SIGPLAN Notices* 46, 8 (Sept. 2011), 35–46. doi:10.1145/2038037.1941561

[58] Manuel M.T. Chakravarty, Gabriele Keller, Sean Lee, Trevor L. McDonell, and Vinod Grover. 2011. Accelerating Haskell Array Codes with Multicore GPUs. In *Proceedings of the Sixth Workshop on Declarative Aspects of Multicore Programming*. ACM, Austin Texas USA, 3–14. doi:10.1145/1926354.1926358

[59] Chun Chen, Jacqueline Chame, and Mary W. Hall. 2008. CHiLL : A Framework for Composing High-level Loop Transformations. https://api.semanticscholar.org/CorpusID:15619135

[60] Ethan Chen, Jiwon Chang, and Yuhao Zhu. 2024. CoolerSpace: A Language for Physically Correct and Computationally Efficient Color Programming. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (Oct. 2024), 846–875. doi:10.1145/3689741

[61] Hongzheng Chen, Bin Fan, Alexander Collins, Bastian Hagedorn, Evghenii Gaburov, Masahiro Masuda, Matthew Brookhart, Chris Sullivan, Jason Knight, Zhiru Zhang, and Vinod Grover. 2026. Tawa: Automatic Warp Specialization for Modern GPUs with Asynchronous References. In *2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 255–267. doi:10.1109/CGO68049.2026.11394849

[62] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation* (Carlsbad, CA, USA) *(OSDI'18)*. USENIX Association, Carlsbad, CA, USA, 579–594.

[63] Wentao Chen, Jiace Zhu, Qi Fan, Yehan Ma, and An Zou. 2025. CUDA-LLM: LLMs Can Write Efficient CUDA Kernels. arXiv:2506.09092 doi:10.48550/arXiv.2506.09092

[64] Zheyuan Chen, Naomi Rehman, Guido Martínez, and Tyler Sorensen. 2026. SIMT-Step Execution: A Flexible Operational Semantics for GPU Subgroup Behavior. *Proc. ACM Program. Lang.* 10, PLDI, Article 219 (June 2026), 25 pages. doi:10.1145/3808297

[65] Wei-Fan Chiang, Ganesh Gopalakrishnan, Guodong Li, and Zvonimir Rakamarić. 2013. Formal Analysis of GPU Programs with Atomics via Conflict-Directed Delay-Bounding. In *NASA Formal Methods* (Berlin, Heidelberg, 2013). Springer, 213–228. doi:10.1007/978-3-642-38088-4_15

[66] Atharva Chougule, Alexander J. Root, Rubens Lacouture, Bobby Yan, Rohan Yadav, and Fredrik Kjolstad. 2026. Partitioning Unstructured Sparse Tensor Algebra for Load-Balanced Parallel Execution. arXiv:2604.17198 doi:10.48550/arXiv.2604.17198

[67] Ezgi Çiçek, Gilles Barthe, Marco Gaboardi, Deepak Garg, and Jan Hoffmann. 2017. Relational Cost Analysis. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages* (Paris, France) *(POPL '17)*. Association for Computing Machinery, New York, NY, USA, 316–329. doi:10.1145/3009837.3009858

[68] R. Clint Whaley, Antoine Petitet, and Jack J. Dongarra. 2001. Automated Empirical Optimizations of Software and the ATLAS Project. *Parallel Comput.* 27, 1 (2001), 3–35. doi:10.1016/S0167-8191(00)00087-9 New Trends in High Performance Computing.

[69] Basile Clément and Albert Cohen. 2022. End-to-End Translation Validation for the Halide Language. *Proceedings of the ACM on Programming Languages* 6, OOPSLA1 (April 2022), 1–30. doi:10.1145/3527328

[70] Tiago Cogumbreiro, Julien Lange, Dennis Liew Zhen Rong, and Hannah Zicarelli. 2021. Checking Data-Race Freedom of GPU Kernels, Compositionally. In *Computer Aided Verification*. Springer International Publishing, Cham, 403–426. doi:10.1007/978-3-030-81685-8_19

[71] Murray Cole. 1991. *Algorithmic Skeletons: Structured Management of Parallel Computation*. MIT Press, Cambridge, MA, USA.

[72] Peter Collingbourne, Cristian Cadar, and Paul H.J. Kelly. 2011. Symbolic Crosschecking of Floating-Point and SIMD Code. In *Proceedings of the Sixth Conference on Computer Systems* (Salzburg, Austria) *(EuroSys '11)*. Association for Computing Machinery, New York, NY, USA, 315–328. doi:10.1145/1966445.1966475

[73] Peter Collingbourne, Cristian Cadar, and Paul H. J. Kelly. 2012. Symbolic Testing of OpenCL Code. In *Hardware and Software: Verification and Testing*. Vol. 7261. Springer Berlin Heidelberg, Berlin, Heidelberg, 203–218. doi:10.1007/978-3-642-34188-5_18

[74] Alexander Collins, Dominik Grewe, Vinod Grover, Sean Lee, and Adriana Susnea. 2014. NOVA: A Functional Language for Data Parallelism. In *Proceedings of ACM SIGPLAN International Workshop on Libraries, Languages, and Compilers for Array Programming*. ACM, Edinburgh United Kingdom, 8–13. doi:10.1145/2627373.2627375

[75] Ronan Collobert, Koray Kavukcuoglu, and Clément Farabet. 2011. Torch7: A Matlab-like Environment for Machine Learning. In *NIPS Workshop BigLearn*. https://api.semanticscholar.org/CorpusID:14365368

[76] Robert L. Cook. 1984. Shade trees. *ACM SIGGRAPH Computer Graphics* 18, 3 (July 1984), 223–231. doi:10.1145/964965.808602

[77] Lucas Cordeiro and Bernd Fischer. 2011. Verifying Multi-threaded Software using SMT-based Context-Bounded Model Checking. In *Proceedings of the 33rd International Conference on Software Engineering* (Waikiki, Honolulu, HI, USA) *(ICSE '11)*. Association for Computing Machinery, New York, NY, USA, 331–340. doi:10.1145/1985793.1985839

[78] NVIDIA Corporation. 2026. Warp Shuffle Functions. https://docs.nvidia.com/cuda/cuda-c-programming-guide/#warp-shuffle-functions Accessed: 2025-07-01.

[79] Nathanaël Courant and Xavier Leroy. 2021. Verified Code Generation for the Polyhedral Model. *Proceedings of the ACM on Programming Languages* 5, POPL (Jan. 2021), 1–24. doi:10.1145/3434321

[80] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages* (Los Angeles, California) *(POPL '77)*. Association for Computing Machinery, New York, NY, USA, 238–252. doi:10.1145/512950.512973

[81] Duncan Coutts, Roman Leshchinskiy, and Don Stewart. 2007. Stream Fusion: from Lists to Streams to Nothing at All. *SIGPLAN Not.* 42, 9 (Oct. 2007), 315–326. doi:10.1145/1291220.1291199

[82] L. Dagum and R. Menon. 1998. OpenMP: An Industry Standard API for Shared-Memory Programming. *IEEE Computational Science and Engineering* 5, 1 (1998), 46–55. doi:10.1109/99.660313

[83] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691 doi:10.48550/arXiv.2307.08691

[84] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. arXiv:2205.14135 doi:10.48550/arXiv.2205.14135

[85] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems* (Budapest, Hungary) *(TACAS'08/ETAPS'08)*. Springer-Verlag, Berlin, Heidelberg, 337–340.

[86] Dorothy E. Denning and Peter J. Denning. 1977. Certification of Programs for Secure Information Flow. *Commun. ACM* 20, 7 (July 1977), 504–513. doi:10.1145/359636.359712

[87] Rajeshwari Devaramani, Jonathan Bentz, and Tony Scudiero. 2025. Unlock GPU Performance: Global Memory Access in CUDA. https://developer.nvidia.com/blog/unlock-gpu-performance-global-memory-access-in-cuda/ Accessed: 2026-06-05.

[88] Zachary DeVito, Niels Joubert, Francisco Palacios, Stephen Oakley, Montserrat Medina, Mike Barrientos, Erich Elsen, Frank Ham, Alex Aiken, Karthik Duraisamy, Eric Darve, Juan Alonso, and Pat Hanrahan. 2011. Liszt: A Domain Specific Language for Building Portable Mesh-based PDE Solvers. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, Seattle Washington, 1–12. doi:10.1145/2063384.2063396

[89] Zachary Devito, Michael Mara, Michael Zollhöfer, Gilbert Bernstein, Jonathan Ragan-Kelley, Christian Theobalt, Pat Hanrahan, Matthew Fisher, and Matthias Niessner. 2017. Opt: A Domain Specific Language for Non-Linear Least Squares Optimization in Graphics and Imaging. *ACM Transactions on Graphics* 36, 5 (Oct. 2017), 1–27. doi:10.1145/3132188

[90] Yaoyao Ding, Bohan Hou, Xiao Zhang, Allan Lin, Tianqi Chen, Cody Hao Yu, Yida Wang, and Gennady Pekhimenko. 2026. Tilus: A Tile-Level GPGPU Programming Language for Low-Precision Computation. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. ACM, Pittsburgh PA USA, 281–297. doi:10.1145/3760250.3762219

[91] Christophe Dubach, Perry Cheng, Rodric Rabbah, David F. Bacon, and Stephen J. Fink. 2012. Compiling a High-level Language for GPUs: (via Language Support for Architectures and Compilers). In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, Beijing China, 1–12. doi:10.1145/2254064.2254066

[92] Kshitij Dubey, Benjamin Driscoll, Anjiang Wei, Neeraj Kayal, Rahul Sharma, and Alex Aiken. 2025. Equivalence Checking of ML GPU Kernels. arXiv:2511.12638 doi:10.48550/arXiv.2511.12638

[93] H. Carter Edwards, Daniel Sunderland, Chris Amsler, and Sam Mish. 2011. Multicore/GPGPU Portable Computational Kernels via Multidimensional Arrays. In *2011 IEEE International Conference on Cluster Computing*. 363–370. doi:10.1109/CLUSTER.2011.47

[94] H. Carter Edwards and Christian R. Trott. 2013. Kokkos: Enabling Performance Portability Across Manycore Architectures. In *2013 Extreme Scaling Workshop (xsw 2013)*. 18–24. doi:10.1109/XSW.2013.7

[95] Ariel Eizenberg, Yuanfeng Peng, Toma Pigli, William Mansky, and Joseph Devietti. 2017. BARRACUDA: Binary-level Analysis of Runtime RAces in CUDA Programs. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, Barcelona Spain, 126–140. doi:10.1145/3062341.3062342

[96] Melih Elibol, Jared Roesch, Isaac Gelado, Eric Buehler, and Michael Garland. 2026. Fearless Concurrency on the GPU. arXiv:2606.15991 doi:10.48550/arXiv.2606.15991

[97] Conal Elliott. 2004. Programming Graphics Processors Functionally. In *Proceedings of the 2004 ACM SIGPLAN Workshop on Haskell*. ACM, Snowbird, Utah USA, 45–56. doi:10.1145/1017472.1017482

[98] Martin Elsman, Troels Henriksen, Danil Annenkov, and Cosmin E. Oancea. 2018. Static Interpretation of Higher-Order Modules in Futhark: Functional GPU Programming in the Large. *Proceedings of the ACM on Programming Languages* 2, ICFP (July 2018), 1–30. doi:10.1145/3236792

[99] Eunomia Labs. 2026. A Taxonomy of GPU Bugs: 19 Defect Classes for CUDA Verification. https://eunomia.dev/blog/2026/01/29/a-taxonomy-of-gpu-bugs-19-defect-classes-for-cuda-verification/ Accessed: 2026-07-22.

[100] Stephan Falke, Deepak Kapur, and Carsten Sinz. 2011. Termination Analysis of C Programs using Compiler Intermediate Languages. In *22nd International Conference on Rewriting Techniques and Applications (RTA'11)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.

[101] Kayvon Fatahalian, Timothy J. Knight, Mike Houston, Mattan Erez, Daniel Reiter Horn, Larkhoon Leem, Ji Young Park, Manman Ren, Alex Aiken, William J. Dally, and Pat Hanrahan. 2006. Sequoia: Programming the Memory Hierarchy. In *SC '06: Proceedings of the 2006 ACM/IEEE Conference on Supercomputing*. 4–4. doi:10.1109/SC.2006.55

[102] Paul Feautrier. 1992. Some Efficient Solutions to the Affine Scheduling Problem: I. One-Dimensional Time. *Int. J. Parallel Program.* 21, 5 (Oct. 1992), 313–348. doi:10.1007/BF01407835

[103] Pratik Fegade, Tianqi Chen, Phillip B. Gibbons, and Todd C. Mowry. 2021. Cortex: A Compiler for Recursive Deep Learning Models. arXiv:2011.01383 doi:10.48550/arXiv.2011.01383

[104] Siyuan Feng, Bohan Hou, Hongyi Jin, Wuwei Lin, Junru Shao, Ruihang Lai, Zihao Ye, Lianmin Zheng, Cody Hao Yu, Yong Yu, and Tianqi Chen. 2023. TensorIR: An Abstraction for Automatic Tensorized Program Optimization. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 804–817. doi:10.1145/3575693.3576933

[105] Benjamin Ferrell, Jun Duan, and Kevin W. Hamlen. 2019. CUDA au Coq: A Framework for Machine-validating GPU Assembly Programs. In *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 474–479. doi:10.23919/DATE.2019.8715160 ISSN: 1558-1101.

[106] Jiří Filipovič, Matúš Madzin, Jan Fousek, and Ludundefinedk Matyska. 2015. Optimizing CUDA Code by Kernel Fusion: Application on BLAS. *J. Supercomput.* 71, 10 (Oct. 2015), 3934–3957. doi:10.1007/s11227-015-1483-z

[107] Michael J. Flynn. 1972. Some Computer Organizations and Their Effectiveness. *IEEE Trans. Comput.* C-21, 9 (1972), 948–960. doi:10.1109/TC.1972.5009071

[108] Theresa Foley and Pat Hanrahan. 2011. Spark: Modular, Composable Shaders for Graphics Hardware. *ACM Transactions on Graphics* 30, 4 (July 2011), 1–12. doi:10.1145/2010324.1965002

[109] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. arXiv:2210.17323 doi:10.48550/arXiv.2210.17323

[110] Roy Frostig, Matthew Johnson, and Chris Leary. 2018. Compiling machine learning programs via high-level tracing. In *SysML Conference 2018*. https://mlsys.org/Conferences/doc/2018/146.pdf

[111] Yoshihiko Futamura. 1983. Partial Computation of Programs. In *RIMS Symposia on Software Science and Engineering* (Berlin, Heidelberg, 1983). Springer, 1–35. doi:10.1007/3-540-11980-9_13

[112] Dietrich Geisler, Irene Yoon, Aditi Kabra, Horace He, Yinnon Sanders, and Adrian Sampson. 2020. Geometry Types for Graphics Programming. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (Nov. 2020), 1–25. doi:10.1145/3428241

[113] Patrice Godefroid, Nils Klarlund, and Koushik Sen. 2005. DART: Directed Automated Random Testing. *SIGPLAN Not.* 40, 6 (June 2005), 213–223. doi:10.1145/1064978.1065036

[114] Google Cloud. 2026. GPU pricing. https://cloud.google.com/products/compute/gpus-pricing Accessed: 2026-07-16.

[115] Ganesh Gopalakrishnan, Ignacio Laguna, Ang Li, Pavel Panchekha, Cindy Rubio-González, and Zachary Tatlock. 2021. Guarding Numerics Amidst Rising Heterogeneity. In *2021 IEEE/ACM 5th International Workshop on Software Correctness for HPC Applications (Correctness)*. 9–15. doi:10.1109/Correctness54621.2021.00007

[116] Clemens Grelck and Sven-Bodo Scholz. 2006. SAC—A Functional Array Language for Efficient Multi-threaded Execution. *International Journal of Parallel Programming* 34, 4 (2006), 383–427. doi:10.1007/s10766-006-0018-x

[117] Larry Gritz, Clifford Stein, Chris Kulla, and Alejandro Conty. 2010. Open Shading Language. In *ACM SIGGRAPH 2010 Talks* (Los Angeles, California) *(SIGGRAPH '10)*. Association for Computing Machinery, New York, NY, USA, Article 33, 1 pages. doi:10.1145/1837026.1837070

[118] Max Grossman, Mauricio Breternitz, and Vivek Sarkar. 2013. HadoopCL: MapReduce on Distributed Heterogeneous Platforms through Seamless Integration of Hadoop and OpenCL. In *2013 IEEE International Symposium on Parallel & Distributed Processing, Workshops and Phd Forum*. 1918–1927. doi:10.1109/IPDPSW.2013.246

[119] Yue Guan, Hongtao Yu, Peng Chen, Daohang Shi, Karthik Manivannan, Nicholas J. Riasanovsky, Manman Ren, Lei Wang, Shane Nay, Partha Kanuparthy, Zaifeng Pan, Zhengding Hu, and Yufei Ding. 2026. TLX: Hardware-Native, Evolvable MIMW GPU Compiler for Large-scale Production Environments. arXiv:2605.10905 doi:10.48550/arXiv.2605.10905

[120] Diwakar Gupta and Sabastian Mugazambi. 2026. Inside the Eighth-Generation TPU: An Architecture Deep Dive. https://cloud.google.com/blog/products/compute/tpu-8t-and-tpu-8i-technical-deep-dive Accessed: 2026-06-24.

[121] Prateek Gupta. 2025. Understanding GPU Programming with Triton: From Hardware to Code. https://www.pgupta.info/blog/2025/12/triton/ Accessed: 2026-06-24.

[122] Bastian Hagedorn, Archibald Samuel Elliott, Henrik Barthels, Rastislav Bodik, and Vinod Grover. 2020. Fireiron: A Data-Movement-Aware Scheduling Language for GPUs. In *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques*. ACM, Virtual Event GA USA, 71–82. doi:10.1145/3410463.3414632

[123] Bastian Hagedorn, Bin Fan, Hanfeng Chen, Cris Cecka, Michael Garland, and Vinod Grover. 2023. Graphene: An IR for Optimized Tensor Computations on GPUs. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. ACM, Vancouver BC Canada, 302–313. doi:10.1145/3582016.3582018

[124] Bastian Hagedorn and Vinod Grover. 2026. It's about Time: Temporal Abstractions for Asynchronous GPU Tensor Computations. In *Proceedings of the 35th ACM SIGPLAN International Conference on Compiler Construction* (Sydney, NSW, Australia) *(CC '26)*. Association for Computing Machinery, New York, NY, USA, 14–26. doi:10.1145/3771775.3786277

[125] Bastian Hagedorn, Johannes Lenfers, Thomas Kœhler, Xueying Qin, Sergei Gorlatch, and Michel Steuwer. 2020. Achieving High-Performance the Functional Way: A Functional Pearl on Expressing High-Performance Optimizations as Rewrite Strategies. *Proceedings of the ACM on Programming Languages* 4, ICFP (Aug. 2020), 1–29. doi:10.1145/3408974

[126] Mary Hall, Jacqueline Chame, Chun Chen, Jaewook Shin, Gabe Rudy, and Malik Murtaza Khan. 2010. Loop Transformation Recipes for Code Generation and Auto-Tuning. In *Languages and Compilers for Parallel Computing*. Springer, Berlin, Heidelberg, 50–64. doi:10.1007/978-3-642-13374-9_4

[127] Mary Hall, Cosmin E. Oancea, Anne C. Elster, Ari Rasch, Sameeran Joshi, Amir Mohammad Tavakkoli, and Richard Schulze. 2025. Scheduling Language Chronology: Past, Present, and Future. *ACM Trans. Archit. Code Optim.* 22, 3, Article 100 (Sept. 2025), 31 pages. doi:10.1145/3743135

[128] Tianyi David Han and Tarek S. Abdelrahman. 2009. *hi*CUDA: A High-level Directive-based Language for GPU Programming. In *Proceedings of 2nd Workshop on General Purpose Processing on Graphics Processing Units*. ACM, Washington D.C. USA, 52–61. doi:10.1145/1513895.1513902

[129] Pat Hanrahan and Jim Lawson. 1990. A Language for Shading and Lighting Calculuations. In *Proceedings of the 17th Annual Conference on Computer Graphics and Interactive Techniques*. ACM, Dallas TX USA, 289–298. doi:10.1145/97879.97911

[130] Mark Harris. 2012. How to Optimize Data Transfers in CUDA C/C++. https://developer.nvidia.com/blog/how-optimize-data-transfers-cuda-cc/ Accessed: 2026-06-05.

[131] Mark Harris. 2012. How to Query Device Properties and Handle Errors in CUDA C/C++. https://developer.nvidia.com/blog/how-query-device-properties-and-handle-errors-cuda-cc/ Accessed: 2026-06-09.

[132] Mark Harris. 2013. An Efficient Matrix Transpose in CUDA C/C++. https://developer.nvidia.com/blog/efficient-matrix-transpose-cuda-cc/ Accessed: 2026-05-29.

[133] Mark Harris. 2013. Using Shared Memory in CUDA C/C++. https://developer.nvidia.com/blog/using-shared-memory-cuda-cc/ Accessed: 2026-06-05.

[134] Mark J Harris, Greg Coombe, Thorsten Scheuermann, and Anselmo Lastra. 2002. Physically-based Visual Simulation on Graphics Hardware. In *Graphics Hardware*.

[135] Haskell.org. 2026. Haskell Language. https://www.haskell.org/ Accessed: 2026-06-01.

[136] Akihiro Hayashi, Max Grossman, Jisheng Zhao, Jun Shirako, and Vivek Sarkar. 2014. Speculative Execution of Parallel Programs with Precise Exception Semantics on GPUs. In *Languages and Compilers for Parallel Computing* (Cham, 2014). Springer International Publishing, 342–356. doi:10.1007/978-3-319-09967-5_20

[137] Akihiro Hayashi, Max Grossman, Jisheng Zhao, Jun Shirako, and Vivek Sarkar. 2013. Accelerating Habanero-Java Programs with OpenCL Generation. In *Proceedings of the 2013 International Conference on Principles and Practices of Programming on the Java Platform: Virtual Machines, Languages, and Tools* (Stuttgart, Germany) *(PPPJ '13)*. Association for Computing Machinery, New York, NY, USA, 124–134. doi:10.1145/2500828.2500840

[138] Ari B. Hayes, Lingda Li, Mohammad Hedayati, Jiahuan He, Eddy Z. Zhang, and Kai Shen. 2017. GPU Taint Tracking. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. USENIX Association, Santa Clara, CA, 209–220. https://www.usenix.org/conference/atc17/technical-sessions/presentation/hayes

[139] Yong He, Kayvon Fatahalian, and Theresa Foley. 2018. Slang: Language Mechanisms for Extensible Real-time Shading Systems. *ACM Trans. Graph.* 37, 4, Article 141 (July 2018), 13 pages. doi:10.1145/3197517.3201380

[140] Zijian He, Adrian Sampson, Yiying Zhang, and Zhiyuan Guo. 2026. VDCores: Resource Decoupled Programming and Execution for Asynchronous GPU. arXiv:2605.03190 doi:10.48550/arXiv.2605.03190

[141] Troels Henriksen. 2021. Bounds Checking on GPU. *International Journal of Parallel Programming* 49, 6 (Dec. 2021), 761–775. doi:10.1007/s10766-021-00703-4

[142] Troels Henriksen, Niels G. W. Serup, Martin Elsman, Fritz Henglein, and Cosmin E. Oancea. 2017. Futhark: Purely Functional GPU-Programming with Nested Parallelism and In-Place Array Updates. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, Barcelona Spain, 556–571. doi:10.1145/3062341.3062354

[143] Pieter Hijma, Stijn Heldens, Alessio Sclocco, Ben Van Werkhoven, and Henri E. Bal. 2023. Optimization Techniques for GPU Programming. *Comput. Surveys* 55, 11 (Nov. 2023), 1–81. doi:10.1145/3570638

[144] Charles Antony Richard Hoare. 1969. An Axiomatic Basis for Computer Programming. *Commun. ACM* 12, 10 (1969), 576–580.

[145] Kenneth E. Hoff, John Keyser, Ming Lin, Dinesh Manocha, and Tim Culver. 1999. Fast Computation of Generalized Voronoi Diagrams using Graphics Hardware. In *Proceedings of the 26th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '99)*. ACM Press/Addison-Wesley Publishing Co., USA, 277–286. doi:10.1145/311535.311567

[146] Martin Hofmann and Steffen Jost. 2003. Static Prediction of Heap Space Usage for First-Order Functional Programs. *SIGPLAN Not.* 38, 1 (Jan. 2003), 185–197. doi:10.1145/640128.604148

[147] Anup Holey, Vineeth Mekkat, and Antonia Zhai. 2013. HAccRG: Hardware-Accelerated Data Race Detection in GPUs. In *2013 42nd International Conference on Parallel Processing*. 60–69. doi:10.1109/ICPP.2013.15 ISSN: 2332-5690.

[148] Mike Houston. 2008. Anatomy of AMD's TeraScale Graphics Engine. https://web.archive.org/web/20100613174446/http://s08.idav.ucdavis.edu/houston-amd-terascale.pdf Accessed: 2026-06-01.

[149] P. Y T Hsu and E. S. Davidson. 1986. Highly Concurrent Scalar Processing. In *Proceedings of the 13th Annual International Symposium on Computer Architecture* (Tokyo, Japan) *(ISCA '86)*. IEEE Computer Society Press, Washington, DC, USA, 386–395.

[150] Yuanming Hu, Tzu-Mao Li, Luke Anderson, Jonathan Ragan-Kelley, and Frédo Durand. 2019. Taichi: A Language for High-Performance Computation on Spatially Sparse Data Structures. *ACM Transactions on Graphics* 38, 6 (Dec. 2019), 1–16. doi:10.1145/3355089.3356506

[151] Roger K. W. Hui and Morten J. Kromberg. 2020. APL since 1978. *Proc. ACM Program. Lang.* 4, HOPL, Article 69 (June 2020), 108 pages. doi:10.1145/3386319

[152] Yuka Ikarashi. 2026. *Uncompromising Performance with Exocompilation.* Ph. D. Dissertation. Massachusetts Institute of Technology. https://www.cs.cornell.edu/~yuka/pdf/dissertation.pdf Accessed: 2025-08-27.

[153] Yuka Ikarashi, Gilbert Louis Bernstein, Alex Reinking, Hasan Genc, and Jonathan Ragan-Kelley. 2022. Exocompilation for Productive Programming of Hardware Accelerators. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. ACM, San Diego CA USA, 703–718. doi:10.1145/3519939.3523446

[154] Yuka Ikarashi, Kevin Qian, Samir Droubi, Alex Reinking, Gilbert Louis Bernstein, and Jonathan Ragan-Kelley. 2025. Exo 2: Growing a Scheduling Language. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. ACM, Rotterdam Netherlands, 426–444. doi:10.1145/3669940.3707218

[155] Intel Corporation. 2022. Introduction to the Xe-HPG Architecture. https://www.intel.com/content/www/us/en/developer/articles/technical/introduction-to-the-xe-hpg-architecture.html Accessed: 2026-07-25.

[156] F. Irigoin and R. Triolet. 1988. Supernode Partitioning. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages - POPL '88*. ACM Press, San Diego, California, United States, 319–329. doi:10.1145/73560.73588

[157] Kazuaki Ishizaki, Akihiro Hayashi, Gita Koblents, and Vivek Sarkar. 2015. Compiling and Optimizing Java 8 Programs for GPU Execution. In *2015 International Conference on Parallel Architecture and Compilation (PACT)*. 419–431. doi:10.1109/PACT.2015.46 ISSN: 1089-795X.

[158] K.E. Iverson. 1962. *A Programming Language.* Wiley.

[159] John Jacobson, Martin Burtscher, and Ganesh Gopalakrishnan. 2024. HiRace: Accurate and Fast Data Race Checking for GPU Programs. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14. doi:10.1109/SC41406.2024.00042

[160] Abhinav Jangda and Arjun Guha. 2020. Model-Based Warp Overlapped Tiling for Image Processing Programs on GPUs. In *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques* (Virtual Event, GA, USA) *(PACT '20)*. Association for Computing Machinery, New York, NY, USA, 317–328. doi:10.1145/3410463.3414649

[161] Aaron Jarmusch and Sunita Chandrasekaran. 2025. Microbenchmarking NVIDIA's Blackwell Architecture: An in-depth Architectural Analysis. (2025). https://arxiv.org/abs/2512.02189v3

[162] Yangqing Jia, Evan Shelhamer, Jeff Donahue, Sergey Karayev, Jonathan Long, Ross Girshick, Sergio Guadarrama, and Trevor Darrell. 2014. Caffe: Convolutional Architecture for Fast Feature Embedding. (2014). doi:10.48550/arXiv.1408.5093

[163] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P. Scarpazza. 2018. Dissecting the NVIDIA Volta GPU Architecture via Microbenchmarking. (2018). https://arxiv.org/abs/1804.06826v1

[164] Aditya K. Kamath and Arkaprava Basu. 2021. iGUARD: In-GPU Advanced Race Detection. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles* (Virtual Event, Germany) *(SOSP '21)*. Association for Computing Machinery, New York, NY, USA, 49–65. doi:10.1145/3477132.3483545

[165] Aditya K. Kamath, Alvin A. George, and Arkaprava Basu. 2020. ScoRD: A Scoped Race Detector for GPUs. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 1036–1049. doi:10.1109/ISCA45697.2020.00088

[166] Andrew Kerr, Duane Merrill, Julien Demouth, and John Tran. 2017. CUTLASS: Fast Linear Algebra in CUDA C++. https://developer.nvidia.com/blog/cutlass-linear-algebra-cuda/ Accessed: 2026-06-23.

[167] Jeroen Ketema and Alastair F. Donaldson. 2017. Termination Analysis for GPU Kernels. *Science of Computer Programming* 148 (2017), 107–122. doi:10.1016/j.scico.2017.04.009 Special Issue on Automated Verification of Critical Systems (AVoCS 2015).

[168] L. Kettner, M. Raab, D. Seibert, J. Jordan, and A. Keller. 2015. The Material Definition Language. In *Proceedings of the Third Workshop on Material Appearance Modeling: Issues and Acquisition* (Darmstadt, Germany) *(MAM '15)*. Eurographics Association, Goslar, DEU, 1–4.

[169] Malik Khan, Protonu Basu, Gabe Rudy, Mary Hall, Chun Chen, and Jacqueline Chame. 2013. A Script-based Autotuning Compiler System to Generate High-Performance CUDA Code. *ACM Transactions on Architecture and Code Optimization* 9, 4 (Jan. 2013), 1–25. doi:10.1145/2400682.2400690

[170] Emmett Kilgariff, Henry Moreton, Nick Stam, and Brandon Bell. 2018. NVIDIA Turing Architecture In-Depth. https://developer.nvidia.com/blog/nvidia-turing-architecture-in-depth/ Accessed: 2026-08-27.

[171] Hansung Kim, Ruohan Richard Yan, Joshua You, Tieliang Vamber Yang, and Yakun Sophia Shao. 2025. Virgo: Cluster-Level Matrix Unit Integration in GPUs for Scalability and Energy Efficiency. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Rotterdam, Netherlands) *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 1382–1399. doi:10.1145/3676641.3716281

[172] James C. King. 1975. A New Approach to Program Testing. *SIGPLAN Not.* 10, 6 (April 1975), 228–233. doi:10.1145/390016.808444

[173] Fredrik Kjolstad, Willow Ahrens, Shoaib Kamil, and Saman Amarasinghe. 2019. Tensor Algebra Compilation with Workspaces. In *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 180–192. doi:10.1109/CGO.2019.8661185

[174] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. 2017. The Tensor Algebra Compiler. *Proceedings of the ACM on Programming Languages* 1, OOPSLA (Oct. 2017), 1–29. doi:10.1145/3133901

[175] Fredrik Kjolstad, Shoaib Kamil, Jonathan Ragan-Kelley, David I. W. Levin, Shinjiro Sueda, Desai Chen, Etienne Vouga, Danny M. Kaufman, Gurtej Kanwar, Wojciech Matusik, and Saman Amarasinghe. 2016. Simit: A Language for Physical Simulation. *ACM Transactions on Graphics* 35, 2 (May 2016), 1–21. doi:10.1145/2866569

[176] Andreas Klöckner. 2014. Loo.py: Transformation-based Code Generation for GPUs and CPUs. In *Proceedings of ACM SIGPLAN International Workshop on Libraries, Languages, and Compilers for Array Programming*. ACM, Edinburgh United Kingdom, 82–87. doi:10.1145/2627373.2627387

[177] Marcin Knap and Paweł Czarnul. 2019. Performance Evaluation of Unified Memory with Prefetching and Oversubscription for Selected Parallel CUDA Applications on NVIDIA Pascal and Volta GPUs. *The Journal of Supercomputing* 75, 11 (2019), 7625–7645. doi:10.1007/s11227-019-02966-8

[178] Thomas Kœhler, Andrés Goens, Siddharth Bhat, Tobias Grosser, Phil Trinder, and Michel Steuwer. 2024. Guided Equality Saturation. *Proc. ACM Program. Lang.* 8, POPL, Article 58 (Jan. 2024), 32 pages. doi:10.1145/3632900

[179] Kensuke Kojima and Atsushi Igarashi. 2013. A Hoare Logic for SIMT Programs. In *Programming Languages and Systems*. Springer International Publishing, Cham, 58–73. doi:10.1007/978-3-319-03542-0_5

[180] Kensuke Kojima and Atsushi Igarashi. 2017. A Hoare Logic for GPU Kernels. *ACM Transactions on Computational Logic* 18, 1 (Jan. 2017), 1–43. doi:10.1145/3001834

[181] Kensuke Kojima, Akifumi Imanishi, and Atsushi Igarashi. 2018. Automated Verification of Functional Correctness of Race-Free GPU Programs. *Journal of Automated Reasoning* 60, 3 (March 2018), 279–298. doi:10.1007/s10817-017-9428-2

[182] Hsiang Tsung Kung, Charles E Leiserson, et al. 1979. Systolic Arrays (for VLSI). In *Sparse Matrix Proceedings 1978*, Vol. 1. SIAM Philadelphia, PA, USA, 256–282.

[183] A. Kunimatsu, N. Ide, T. Sato, Y. Endo, H. Murakami, T. Kamei, M. Hirano, F. Ishihara, H. Tago, M. Oka, A. Ohba, T. Yutaka, T. Okada, and M. Suzuoki. 2000. Vector Unit Architecture for Emotion Synthesis. *IEEE Micro* 20, 2 (2000), 40–47. doi:10.1109/40.848471

[184] Bastian Köpcke, Sergei Gorlatch, and Michel Steuwer. 2024. Descend: A Safe GPU Systems Programming Language. *Proceedings of the ACM on Programming Languages* 8, PLDI (June 2024), 841–864. doi:10.1145/3656411

[185] Ruihang Lai, Junru Shao, Siyuan Feng, Steven Lyubomirsky, Bohan Hou, Wuwei Lin, Zihao Ye, Hongyi Jin, Yuchen Jin, Jiawei Liu, Lesheng Jin, Yaxing Cai, Ziheng Jiang, Yong Wu, Sunghyun Park, Prakalp Srivastava, Jared Roesch, Todd C. Mowry, and Tianqi Chen. 2025. Relax: Composable Abstractions for End-to-End Dynamic Machine Learning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Rotterdam, Netherlands) *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 998–1013. doi:10.1145/3676641.3716249

[186] Monica S. Lam. 2004. Software Pipelining: An Effective Scheduling Technique for VLIW Machines. *SIGPLAN Not.* 39, 4 (April 2004), 244–256. doi:10.1145/989393.989420

[187] E. Scott Larsen and David McAllister. 2001. Fast Matrix Multiplies using Graphics Hardware. In *Proceedings of the 2001 ACM/IEEE Conference on Supercomputing*. ACM, Denver Colorado, 55–55. doi:10.1145/582034.582089

[188] C. Lattner and V. Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004.* 75–86. doi:10.1109/CGO.2004.1281665

[189] Christian Lengauer. 1993. Loop Parallelization in the Polytope Model. In *CONCUR'93*. Springer, Berlin, Heidelberg, 398–416. doi:10.1007/3-540-57208-2_28

[190] Jed Lengyel, Mark Reichert, Bruce R. Donald, and Donald P. Greenberg. 1990. Real-time Robot Motion Planning using Rasterizing Computer Graphics Hardware. In *Proceedings of the 17th Annual Conference on Computer Graphics and Interactive Techniques* (Dallas, TX, USA) *(SIGGRAPH '90)*. Association for Computing Machinery, New York, NY, USA, 327–335. doi:10.1145/97879.97915

[191] Alan Leung, Manish Gupta, Yuvraj Agarwal, Rajesh Gupta, Ranjit Jhala, and Sorin Lerner. 2012. Verifying GPU Kernels by Test Amplification. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation* (Beijing, China) *(PLDI '12)*. Association for Computing Machinery, New York, NY, USA, 383–394. doi:10.1145/2254064.2254110

[192] Alan Leung, Ondřej Lhoták, and Ghulam Lashari. 2009. Automatic Parallelization for Graphics Processing Units. In *Proceedings of the 7th International Conference on Principles and Practice of Programming in Java* (Calgary, Alberta, Canada) *(PPPJ '09)*. Association for Computing Machinery, New York, NY, USA, 91–100. doi:10.1145/1596655.1596670

[193] Guodong Li and Ganesh Gopalakrishnan. 2010. Scalable SMT-based Verification of GPU Kernel Functions. In *Proceedings of the Eighteenth ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Santa Fe, New Mexico, USA) *(FSE '10)*. Association for Computing Machinery, New York, NY, USA, 187–196. doi:10.1145/1882291.1882320

[194] Guodong Li and Ganesh Gopalakrishnan. 2012. Parameterized Verification of GPU Kernel Programs. In *2012 IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum*. 2450–2459. doi:10.1109/IPDPSW.2012.302

[195] Guodong Li, Peng Li, Geof Sawaya, Ganesh Gopalakrishnan, Indradeep Ghosh, and Sreeranga P. Rajan. 2012. GKLEE: Concolic Verification and Test Generation for GPUs. *ACM SIGPLAN Notices* 47, 8 (Sept. 2012), 215–224. doi:10.1145/2370036.2145844

[196] Pengcheng Li, Xiaoyu Hu, Dong Chen, Jacob Brock, Hao Luo, Eddy Z. Zhang, and Chen Ding. 2017. LD: Low-Overhead GPU Race Detection Without Access Monitoring. *ACM Trans. Archit. Code Optim.* 14, 1, Article 9 (March 2017), 25 pages. doi:10.1145/3046678

[197] Peng Li, Guodong Li, and Ganesh Gopalakrishnan. 2012. Parametric Flows: Automated Behavior Equivalencing for Symbolic Analysis of Races in CUDA Programs. In *SC '12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. 1–10. doi:10.1109/SC.2012.94 ISSN: 2167-4337.

[198] Peng Li, Guodong Li, and Ganesh Gopalakrishnan. 2014. Practical Symbolic Race Checking of GPU Programs. In *SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 179–190. doi:10.1109/SC.2014.20 ISSN: 2167-4337.

[199] Ruihao Li, Sanjana Yadav, Qinzhe Wu, Krishna Kavi, Gayatri Mehta, Neeraja J. Yadwadkar, and Lizy K. John. 2023. Performance Implications of Async Memcpy and UVM: A Tale of Two Data Transfer Modes. In *2023 IEEE International Symposium on Workload Characterization (IISWC)*. 115–127. doi:10.1109/IISWC59245.2023.00024

[200] Tzu-Mao Li, Michaël Gharbi, Andrew Adams, Frédo Durand, and Jonathan Ragan-Kelley. 2018. Differentiable Programming for Image Processing and Deep Learning in Halide. *ACM Transactions on Graphics* 37, 4 (Aug. 2018), 1–13. doi:10.1145/3197517.3201383

[201] Wei Li, Xiaoming Wei, and Arie Kaufman. 2003. Implementing Lattice Boltzmann Computation on Graphics Hardware. *The Visual Computer* 19, 7 (2003), 444–456. doi:10.1007/s00371-003-0210-6

[202] Xinyi Li, Mark Baranowski, Harvey Dam, and Ganesh Gopalakrishnan. 2025. Array Programming on GPUs: Challenges and Opportunities. In *Proceedings of the 11th ACM SIGPLAN International Workshop on Libraries, Languages and Compilers for Array Programming* (Seoul, Republic of Korea) *(ARRAY '25)*. Association for Computing Machinery, New York, NY, USA, 41–52. doi:10.1145/3736112.3736144

[203] Dennis Liew, Tiago Cogumbreiro, and Julien Lange. 2024. Sound and Partially-Complete Static Analysis of Data-Races in GPU Programs. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (Oct. 2024), 2434–2461. doi:10.1145/3689797

[204] Amy W. Lim and Monica S. Lam. 1997. Maximizing Parallelism and Minimizing Synchronization with Affine Transforms. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Paris, France) *(POPL '97)*. Association for Computing Machinery, New York, NY, USA, 201–214. doi:10.1145/263699.263719

[205] Erik Lindholm, Mark J. Kilgard, and Henry Moreton. 2001. A User-Programmable Vertex Engine. In *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques*. ACM, 149–158. doi:10.1145/383259.383274

[206] Erik Lindholm, John Nickolls, Stuart Oberman, and John Montrym. 2008. NVIDIA Tesla: A Unified Graphics and Computing Architecture. *IEEE Micro* 28, 2 (March 2008), 39–55. doi:10.1109/MM.2008.31

[207] Amanda Liu, Gilbert Bernstein, Adam Chlipala, and Jonathan Ragan-Kelley. 2024. A Verified Compiler for a Functional Tensor Language. *Proceedings of the ACM on Programming Languages* 8, PLDI (June 2024), 320–342. doi:10.1145/3656390

[208] Amanda Liu, Gilbert Louis Bernstein, Adam Chlipala, and Jonathan Ragan-Kelley. 2022. Verified Tensor-Program Optimization via High-level Scheduling Rewrites. *Proceedings of the ACM on Programming Languages* 6, POPL (Jan. 2022), 1–28. doi:10.1145/3498717

[209] Weile Luo, Ruibo Fan, Zeyu Li, Dayou Du, Hongyuan Liu, Qiang Wang, and Xiaowen Chu. 2025. Dissecting the NVIDIA Hopper Architecture through Microbenchmarking and Multiple Level Analysis. (2025). https://arxiv.org/abs/2501.12084v2

[210] Daniel Lustig, Sameer Sahasrabuddhe, and Olivier Giroux. 2019. A Formal Analysis of the NVIDIA PTX Memory Consistency Model. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, Providence RI USA, 257–270. doi:10.1145/3297858.3304043

[211] Weiliang Ma, Qian Xiong, Xuanhua Shi, Xiaosong Ma, Hai Jin, Haozhao Kuang, Mingyu Gao, Ye Zhang, Haichen Shen, and Weifang Hu. 2023. GZKP: A GPU Accelerated Zero-Knowledge Proof System. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 340–353. doi:10.1145/3575693.3575711

[212] Miles Macklin. 2024. Warp: Differentiable Spatial Computing for Python. In *ACM SIGGRAPH 2024 Courses*. ACM, Denver CO USA, 1–147. doi:10.1145/3664475.3664543

[213] Miles Macklin, Leopold Cambier, and Eric Shi. 2024. Introducing Tile-Based Programming in Warp 1.5.0. https://developer.nvidia.com/blog/introducing-tile-based-programming-in-warp-1-5-0/ Accessed: 2026-06-23.

[214] Geoffrey Mainland and Greg Morrisett. 2010. Nikola: Embedding Compiled GPU Functions in Haskell. In *Proceedings of the third ACM Haskell symposium on Haskell*. ACM, Baltimore Maryland USA, 67–78. doi:10.1145/1863523.1863533

[215] William R. Mark, R. Steven Glanville, Kurt Akeley, and Mark J. Kilgard. 2003. Cg: A System for Programming Graphics Hardware in a C-like Language. *ACM Transactions on Graphics* 22, 3 (July 2003), 896–907. doi:10.1145/882262.882362

[216] Stefano Markidis, Steven Wei Der Chien, Erwin Laure, Ivy Bo Peng, and Jeffrey S. Vetter. 2018. NVIDIA Tensor Core Programmability, Performance & Precision. arXiv:1803.04014 doi:10.48550/arXiv.1803.04014

[217] Guido Martínez, Bastian Köpcke, Jonáš Fiala, Gabriel Ebner, Tahina Ramananandro, Michel Steuwer, Tyler Sorensen, and Nikhil Swamy. 2026. Kuiper: Correct and Efficient GPU Programming with Dependent Types and Separation Logic. *Proc. ACM Program. Lang.* 10, PLDI, Article 202 (June 2026), 25 pages. doi:10.1145/3808280

[218] Meta Platforms, Inc . 2025. Helion Documentation. https://helionlang.com/ Accessed: 2026-06-24.

[219] Microsoft. 2021. Direct3D 12 Raytracing HLSL Shaders. https://learn.microsoft.com/en-us/windows/win32/direct3d12/direct3d-12-raytracing-hlsl-shaders Accessed: 2026-08-27.

[220] Microsoft. 2021. High-level Shader Language (HLSL). https://learn.microsoft.com/en-us/windows/win32/direct3dhlsl/dx-graphics-hlsl Accessed: 2026-05-29.

[221] Sparsh Mittal, S. B. Abhinaya, Manish Reddy, and Irfan Ali. 2018. A Survey of Techniques for Improving Security of GPUs. *Journal of Hardware and Systems Security* 2, 3 (2018), 266–285. doi:10.1007/s41635-018-0039-0

[222] Modular Inc. 2023. Mojo. https://mojolang.org/ Accessed: 2026-06-22.

[223] Felipe R. Monteiro, Erickson H. da S. Alves, Isabela S. Silva, Hussama I. Ismail, Lucas C. Cordeiro, and Eddie B. de Lima Filho. 2018. ESBMC-GPU A Context-Bounded Model Checking Tool to Verify CUDA Programs. *Science of Computer Programming* 152 (2018), 63–69. doi:10.1016/j.scico.2017.09.005

[224] Ravi Teja Mullapudi, Andrew Adams, Dillon Sharlet, Jonathan Ragan-Kelley, and Kayvon Fatahalian. 2016. Automatically Scheduling Halide Image Processing Pipelines. *ACM Transactions on Graphics* 35, 4 (July 2016), 1–11. doi:10.1145/2897824.2925952

[225] Ravi Teja Mullapudi, Vinay Vasista, and Uday Bondhugula. 2015. PolyMage: Automatic Optimization for Image Processing Pipelines. *SIGARCH Comput. Archit. News* 43, 1 (March 2015), 429–443. doi:10.1145/2786763.2694364

[226] Stefan K. Muller and Jan Hoffmann. 2021. Modeling and Analyzing Evaluation Cost of CUDA Kernels. *Proceedings of the ACM on Programming Languages* 5, POPL (Jan. 2021), 1–31. doi:10.1145/3434306

[227] Stefan K. Muller and Jan Hoffmann. 2024. Modeling and Analyzing Evaluation Cost of CUDA Kernels. *ACM Trans. Parallel Comput.* 11, 1, Article 5 (March 2024), 53 pages. doi:10.1145/3639403

[228] Brad Nemire. 2015. Inside the Programming Evolution of GPU Computing. https://developer.nvidia.com/blog/inside-the-programming-evolution-of-gpu-computing/ Accessed: 2026-06-24.

[229] John Nickolls, Ian Buck, Michael Garland, and Kevin Skadron. 2008. Scalable Parallel Programming with CUDA. In *ACM SIGGRAPH 2008 classes*. ACM, Los Angeles California, 1–14. doi:10.1145/1401132.1401152

[230] Tobias Nipkow, Markus Wenzel, and Lawrence C. Paulson. 2002. *Isabelle/HOL*. Lecture Notes in Computer Science, Vol. 2283. Springer Berlin Heidelberg. doi:10.1007/3-540-45949-9

[231] NVIDIA Corporation. 2016. NVIDIA Tesla P100 Whitepaper. (2016). https://images.nvidia.com/content/pdf/tesla/whitepaper/pascal-architecture-whitepaper.pdf Accessed: 2026-06-01.

[232] NVIDIA Corporation. 2017. Independent Thread Scheduling. https://docs.nvidia.com/cuda/volta-tuning-guide/index.html#independent-thread-scheduling Accessed: 2026-07-21.

[233] NVIDIA Corporation. 2017. NVIDIA Tesla V100 GPU Architecture. (2017). https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf Accessed: 2026-06-01.
[234] NVIDIA Corporation. 2020. NVIDIA A100 Tensor Core GPU Architecture. (2020). https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf Accessed: 2026-06-02.
[235] NVIDIA Corporation. 2022. NVIDIA H100 Tensor Core GPU Architecture. (2022). https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c Accessed: 2026-06-01.
[236] NVIDIA Corporation. 2025. cuTile Python. https://docs.nvidia.com/cuda/cutile-python/ Accessed: 2026-05-29.
[237] NVIDIA Corporation. 2025. NVIDIA Blackwell Architecture Technical Brief. (2025). https://resources.nvidia.com/en-us-blackwell-architecture Accessed: 2026-06-01.
[238] NVIDIA Corporation. 2025. Tile IR. https://docs.nvidia.com/cuda/tile-ir/latest/ Accessed: 2026-05-29.
[239] NVIDIA Corporation. 2026. Atomic Functions. https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html#atomic-functions Accessed: 2026-06-26.
[240] NVIDIA Corporation. 2026. Coalesced Access to Global Memory. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/#coalesced-access-to-global-memory Accessed: 2026-06-01.
[241] NVIDIA Corporation. 2026. Compute Sanitizer. https://docs.nvidia.com/compute-sanitizer/ComputeSanitizer/index.html Accessed: 2026-06-09.
[242] NVIDIA Corporation. 2026. cuBLAS. https://docs.nvidia.com/cuda/cublas/index.html Accessed: 2025-06-22.
[243] NVIDIA Corporation. 2026. cuDNN. https://docs.nvidia.com/deeplearning/cudnn/latest/ Accessed: 2025-06-23.
[244] NVIDIA Corporation. 2026. CuTe DSL. https://docs.nvidia.com/cutlass/latest/media/docs/pythonDSL/cute_dsl.html Accessed: 2026-06-23.
[245] NVIDIA Corporation. 2026. CuTe Layouts. https://docs.nvidia.com/cutlass/latest/media/docs/cpp/cute/01_layout.html Accessed: 2026-05-29.
[246] NVIDIA Corporation. 2026. Independent Thread Scheduling. https://docs.nvidia.com/cuda/cuda-programming-guide/03-advanced/advanced-kernel-programming.html#advanced-kernel-programming Accessed: 2026-06-02.
[247] NVIDIA Corporation. 2026. NVIDIA Nsight Compute. https://developer.nvidia.com/nsight-compute Accessed: 2026-09-02.
[248] NVIDIA Corporation. 2026. NVIDIA Tensor Cores. https://www.nvidia.com/en-us/data-center/tensor-cores/ Accessed: 2026-06-02.
[249] NVIDIA Corporation. 2026. Occupancy. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/#occupancy Accessed: 2026-06-01.
[250] NVIDIA Corporation. 2026. Parallel Thread Execution ISA Version 9.3. https://docs.nvidia.com/cuda/parallel-thread-execution/ Accessed: 2026-09-02.
[251] NVIDIA Corporation. 2026. Parallel Thread Execution (PTX) ISA — Asynchronous Warpgroup Level Matrix Multiply-Accumulate Instructions. https://docs.nvidia.com/cuda/parallel-thread-execution/index.html?highlight=tcgen05%2520cp#asynchronous-warpgroup-level-matrix-instructions. Accessed: 2026-06-01.
[252] NVIDIA Corporation. 2026. Parallel Thread Execution (PTX) ISA — TensorCore 5th Generation Instructions (tcgen05). https://docs.nvidia.com/cuda/parallel-thread-execution/index.html?highlight=tcgen05%2520cp#tensorcore-5th-generation-instructions Accessed: 2026-06-01.
[253] NVIDIA Corporation. 2026. Parallel Thread Execution (PTX) ISA — Warp Level Matrix Multiply-Accumulate Instructions. https://docs.nvidia.com/cuda/parallel-thread-execution/index.html?highlight=tcgen05%2520cp#warp-level-matrix-instructions. Accessed: 2026-06-01.
[254] NVIDIA Corporation. 2026. Shared Memory and Memory Banks. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/#shared-memory-and-memory-banks Accessed: 2026-06-01.
[255] NVIDIA Corporation. 2026. Tensor Memory Accelerator. (2026). https://docs.nvidia.com/cuda/hopper-tuning-guide/index.html#tensor-memory-accelerator Accessed: 2026-06-01.
[256] NVIDIA Corporation. 2026. Unified Memory. https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/unified-memory.html Accessed: 2026-06-10.
[257] Travis E. Oliphant. 2006. *Guide to Numpy*.
[258] Peter W. O'Hearn. 2007. Resources, Concurrency, and Local Reasoning. *Theoretical Computer Science* 375, 1 (2007), 271–307. doi:10.1016/j.tcs.2006.12.035 Festschrift for John C. Reynolds's 70th birthday.
[259] Steven G. Parker, James Bigler, Andreas Dietrich, Heiko Friedrich, Jared Hoberock, David Luebke, David McAllister, Morgan McGuire, Keith Morley, Austin Robison, and Martin Stich. 2010. OptiX: A General Purpose Ray Tracing Engine. *ACM Trans. Graph.* 29, 4, Article 66 (July 2010), 13 pages. doi:10.1145/1778765.1778803
[260] Steven G. Parker, Solomon Boulos, James Bigler, and Austin Robison. 2007. RTSL: A Ray Tracing Shading Language. In *2007 IEEE Symposium on Interactive Ray Tracing*. 149–160. doi:10.1109/RT.2007.4342603
[261] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison,

Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. arXiv:1912.01703 doi:10.48550/arXiv.1912.01703

[262] Adam Paszke, Daniel D. Johnson, David Duvenaud, Dimitrios Vytiniotis, Alexey Radul, Matthew J. Johnson, Jonathan Ragan-Kelley, and Dougal Maclaurin. 2021. Getting to the Point: Index Sets and Parallelism-Preserving Autodiff for Pointful Array Programming. *Proc. ACM Program. Lang.* 5, ICFP, Article 88 (Aug. 2021), 29 pages. doi:10.1145/3473593

[263] Manos Pavlidakis, Giorgos Vasiliadis, Stelios Mavridis, Anargyros Argyros, Antony Chazapis, and Angelos Bilas. 2024. Guardian: Safe GPU Sharing in Multi-Tenant Environments. (2024). https://arxiv.org/abs/2401.09290v2

[264] Yuanfeng Peng, Vinod Grover, and Joseph Devietti. 2018. CURD: A Dynamic CUDA Race Detector. *ACM SIGPLAN Notices* 53, 4 (Dec. 2018), 390–403. doi:10.1145/3296979.3192368

[265] Arsène Pérard-Gayot, Richard Membarth, Roland Leißa, Sebastian Hack, and Philipp Slusallek. 2019. Rodent: Generating Renderers Without Writing a Generator. *ACM Trans. Graph.* 38, 4, Article 40 (July 2019), 12 pages. doi:10.1145/3306346.3322955

[266] Phillipe Pereira, Higo Albuquerque, Hendrio Marques, Isabela Silva, Celso Carvalho, Lucas Cordeiro, Vanessa Santos, and Ricardo Ferreira. 2016. Verifying CUDA Programs using SMT-based Context-Bounded Model Checking. In *Proceedings of the 31st Annual ACM Symposium on Applied Computing*. ACM, Pisa Italy, 1648–1653. doi:10.1145/2851613.2851830

[267] Ken Perlin. 1985. An Image Synthesizer. *ACM SIGGRAPH Computer Graphics* 19, 3 (July 1985), 287–296. doi:10.1145/325165.325247

[268] Phitchaya Mangpo Phothilimthana, Jason Ansel, Jonathan Ragan-Kelley, and Saman Amarasinghe. 2013. Portable Performance on Heterogeneous Architectures. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, Houston Texas USA, 431–444. doi:10.1145/2451116.2451162

[269] Roberto Di Pietro, Flavio Lombardi, and Antonio Villani. 2016. CUDA Leaks: A Detailed Hack for CUDA and a (Partial) Fix. *ACM Trans. Embed. Comput. Syst.* 15, 1, Article 15 (Jan. 2016), 25 pages. doi:10.1145/2801153

[270] Philip C. Pratt-Szeliga, James W. Fawcett, and Roy D. Welch. 2012. Rootbeer: Seamlessly Using GPUs from Java. In *2012 IEEE 14th International Conference on High Performance Computing and Communication & 2012 IEEE 9th International Conference on Embedded Software and Systems*. 375–380. doi:10.1109/HPCC.2012.57

[271] James Price and Simon McIntosh-Smith. 2015. Oclgrind: An Extensible OpenCL Device Simulator. In *Proceedings of the 3rd International Workshop on OpenCL* (Palo Alto, California) *(IWOCL '15)*. Association for Computing Machinery, New York, NY, USA, Article 12, 7 pages. doi:10.1145/2791321.2791333

[272] Kekoa Proudfoot, William R. Mark, Svetoslav Tzvetkov, and Pat Hanrahan. 2001. A Real-time Procedural Shading System for Programmable Graphics Hardware. In *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques*. ACM, 159–170. doi:10.1145/383259.383275

[273] Python Software Foundation. 2026. Python. https://www.python.org/ Accessed: 2026-06-20.

[274] PyTorch Team. 2025. Helion: A High-level DSL for Performant and Portable ML Kernels. https://pytorch.org/blog/helion/ Accessed: 2026-05-29.

[275] Xueying Qin, Liam O'Connor, Rob van Glabbeek, Peter Höfner, Ohad Kammar, and Michel Steuwer. 2024. Shoggoth: A Formal Foundation for Strategic Rewriting. *Proc. ACM Program. Lang.* 8, POPL, Article 3 (Jan. 2024), 29 pages. doi:10.1145/3633211

[276] Jonathan Ragan-Kelley. 2024. The Future of Fast Code: Giving Hardware What It Wants. https://www.youtube.com/watch?v=vU3ryvZYlkk Accessed: 2026-06-01.

[277] Jonathan Ragan-Kelley, Andrew Adams, Sylvain Paris, Marc Levoy, Saman Amarasinghe, and Frédo Durand. 2012. Decoupling Algorithms from Schedules for Easy Optimization of Image Processing Pipelines. *ACM Transactions on Graphics* 31, 4 (Aug. 2012), 1–12. doi:10.1145/2185520.2185528

[278] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines. *SIGPLAN Not.* 48, 6 (June 2013), 519–530. doi:10.1145/2499370.2462176

[279] B. Ramakrishna Rau. 1994. Iterative Modulo Scheduling: An Algorithm for Software Pipelining Loops. In *Proceedings of the 27th Annual International Symposium on Microarchitecture* (San Jose, California, USA) *(MICRO 27)*. Association for Computing Machinery, New York, NY, USA, 63–74. doi:10.1145/192724.192731

[280] B. R. Rau and C. D. Glaeser. 1981. Some Scheduling Techniques and an Easily Schedulable Horizontal Architecture for High Performance Scientific Computing. In *Proceedings of the 14th Annual Workshop on Microprogramming* (Chatham, Massachusetts, USA) *(MICRO 14)*. IEEE Press, 183–198.

[281] Mahesh Ravishankar, Justin Holewinski, and Vinod Grover. 2015. Forma: A DSL for Image Processing Applications to Target GPUs and Multi-core CPUs. In *Proceedings of the 8th Workshop on General Purpose Processing Using GPUs* (San Francisco, CA, USA) *(GPGPU-8)*. Association for Computing Machinery, New York, NY, USA, 109–120. doi:10.1145/2716282.2716290

[282] Prashant Singh Rawat, Miheer Vaidya, Aravind Sukumaran-Rajam, Mahesh Ravishankar, Vinod Grover, Atanas Rountev, Louis-Noël Pouchet, and P. Sadayappan. 2018. Domain-Specific Optimization and Generation of High-Performance GPU Code for Stencil Computations. *Proc. IEEE* 106, 11 (2018), 1902–1920. doi:10.1109/JPROC.2018.2862896

[283] Alex Reinking, Gilbert Louis Bernstein, and Jonathan Ragan-Kelley. 2022. Formal Semantics for the Halide Language. doi:10.48550/arXiv.2210.15740 arXiv:2210.15740.

[284] Manman Ren, Nick Riasanovsky, Neil Dhar, Hongtao Yu, Jie Liu, Partha Kanuparthy, and Shane Nay. 2026. Warp Specialization in Triton: Design and Roadmap. https://pytorch.org/blog/warp-specialization-in-triton-design-and-roadmap/ Accessed: 2026-06-24.

[285] John C. Reynolds. 2002. Separation Logic: A Logic for Shared Mutable Data Structures. In *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science (LICS '02)*. IEEE Computer Society, USA, 55–74.

[286] Jared Roesch, Steven Lyubomirsky, Logan Weber, Josh Pollock, Marisa Kirisame, Tianqi Chen, and Zachary Tatlock. 2018. Relay: A New IR for Machine Learning Frameworks. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (Philadelphia, PA, USA) *(MAPL 2018)*. Association for Computing Machinery, New York, NY, USA, 58–68. doi:10.1145/3211346.3211348

[287] Randi J. Rost. 2004. *OpenGL Shading Language*. Addison-Wesley Professional.

[288] Gabe Rudy, Malik Murtaza Khan, Mary Hall, Chun Chen, and Jacqueline Chame. 2011. A Programming Language Interface to Describe Transformations and Code Generation. In *Languages and Compilers for Parallel Computing*. Springer, Berlin, Heidelberg, 136–150. doi:10.1007/978-3-642-19595-2_10

[289] Robert Schenck, Nikolaj Hey Hinnerskov, Troels Henriksen, Magnus Madsen, and Martin Elsman. 2024. AUTOMAP: Inferring Rank-Polymorphic Function Applications with Integer Linear Programming. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (Oct. 2024), 1787–1813. doi:10.1145/3689774

[290] Sven-Bodo Scholz. 2003. Single Assignment C: Efficient Support for High-level Array Operations in a Functional Setting. *J. Funct. Program.* 13, 6 (Nov. 2003), 1005–1059. doi:10.1017/S0956796802004458

[291] Fabian Schuetze. 2024. Reverse-Engineering cuBLAS. https://accu.org/journals/overload/32/181/schuetze/ Accessed: 2025-06-22.

[292] Koushik Sen, Darko Marinov, and Gul Agha. 2005. CUTE: A Concolic Unit Testing Engine for C. *SIGSOFT Softw. Eng. Notes* 30, 5 (Sept. 2005), 263–272. doi:10.1145/1095430.1081750

[293] Ryan Senanayake, Changwan Hong, Ziheng Wang, Amalee Wilson, Stephen Chou, Shoaib Kamil, Saman Amarasinghe, and Fredrik Kjolstad. 2020. A Sparse Iteration Space Transformation Framework for Sparse Tensor Algebra. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 158 (Nov. 2020), 30 pages. doi:10.1145/3428226

[294] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision. arXiv:2407.08608 doi:10.48550/arXiv.2407.08608

[295] Rahul Sharma, Michael Bauer, and Alex Aiken. 2015. Verification of Producer-Consumer Synchronization in GPU Programs. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Portland, OR, USA) *(PLDI '15)*. Association for Computing Machinery, New York, NY, USA, 88–98. doi:10.1145/2737924.2737962

[296] Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Penporn Koanantakool, Peter Hawkins, HyoukJoong Lee, Mingsheng Hong, Cliff Young, Ryan Sepassi, and Blake Hechtman. 2018. Mesh-TensorFlow: Deep Learning for Supercomputers. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems* (Montréal, Canada) *(NIPS'18)*. Curran Associates Inc., Red Hook, NY, USA, 10435–10444.

[297] Stephen F. Siegel, Manchun Zheng, Ziqing Luo, Timothy K. Zirkel, Andre V. Marianiello, John G. Edenhofner, Matthew B. Dwyer, and Michael S. Rogers. 2015. CIVL: The Concurrency Intermediate Verification Language. In *SC '15: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–12. doi:10.1145/2807591.2807635

[298] Vivek Singh. 2024. Game-Changer: How the World's First GPU Leveled Up Gaming and Ignited the AI Era. https://blogs.nvidia.com/blog/first-gpu-gaming-ai/ Accessed: 2026-07-07.

[299] Savvas Sioutas, Sander Stuijk, Twan Basten, Henk Corporaal, and Lou Somers. 2020. Schedule Synthesis for Halide Pipelines on GPUs. *ACM Trans. Archit. Code Optim.* 17, 3, Article 23 (Aug. 2020), 25 pages. doi:10.1145/3406117

[300] Savvas Sioutas, Sander Stuijk, Luc Waeijen, Twan Basten, Henk Corporaal, and Lou Somers. 2019. Schedule Synthesis for Halide Pipelines through Reuse Analysis. *ACM Trans. Archit. Code Optim.* 16, 2, Article 10 (April 2019), 22 pages. doi:10.1145/3310248

[301] Gina Sitaraman, Noel Chalmers, Nicholas Malaya, Damon McDougall, Ossan O'Reilly, Rene Van Oostrum, and Joseph Greathouse. 2023. AMD matrix cores. https://gpuopen.com/learn/amd-lab-notes/amd-lab-notes-matrix-cores-readme/ Accessed: 2026-07-25.

[302] Rupanshu Soi, Rohan Yadav, Fredrik Kjolstad, Alex Aiken, Maryam Mehri Dehnavi, Michael Garland, and Michael Bauer. 2026. Optimal Software Pipelining and Warp Specialization for Tensor Core GPUs. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, Seattle, WA, 1875–1890. https:

//www.usenix.org/conference/osdi26/presentation/soi

[303] Tyler Sorensen and Alastair F. Donaldson. 2016. Exposing Errors Related to Weak Memory in GPU Applications. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, Santa Barbara CA USA, 100–113. doi:10.1145/2908080.2908114

[304] Benjamin F. Spector, Simran Arora, Aaryan Singhal, Daniel Y. Fu, and Christopher Ré. 2024. ThunderKittens: Simple, Fast, and Adorable AI Kernels. doi:10.48550/arXiv.2410.20399 arXiv:2410.20399.

[305] Stack Exchange Inc. 2026. Stack Overflow. https://stackoverflow.co/ Accessed: 2026-06-08.

[306] Michel Steuwer, Christian Fensch, Sam Lindley, and Christophe Dubach. 2015. Generating Performance Portable Code using Rewrite Rules: From High-level Functional Expressions to High-Performance OpenCL Code. In *Proceedings of the 20th ACM SIGPLAN International Conference on Functional Programming*. ACM, Vancouver BC Canada, 205–217. doi:10.1145/2784731.2784754

[307] Michel Steuwer, Philipp Kegel, and Sergei Gorlatch. 2011. SkelCL - A Portable Skeleton Library for High-Level GPU Programming. In *2011 IEEE International Symposium on Parallel and Distributed Processing Workshops and PhD Forum*. 1176–1182. doi:10.1109/IPDPS.2011.269

[308] Michel Steuwer, Toomas Remmelg, and Christophe Dubach. 2017. LIFT: A Functional Data-Parallel IR for High-Performance GPU Code Generation. In *2017 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 74–85. doi:10.1109/CGO.2017.7863730

[309] John E. Stone, David Gohara, and Guochun Shi. 2010. OpenCL: A Parallel Programming Standard for Heterogeneous Computing Systems. *Computing in Science & Engineering* 12, 3 (May 2010), 66–73. doi:10.1109/MCSE.2010.69

[310] Arvind K. Sujeeth, Kevin J. Brown, Hyoukjoong Lee, Tiark Rompf, Hassan Chafi, Martin Odersky, and Kunle Olukotun. 2014. Delite: A Compiler Architecture for Performance-Oriented Embedded Domain-Specific Languages. *ACM Transactions on Embedded Computing Systems* 13, 4s (July 2014), 1–25. doi:10.1145/2584665

[311] Arvind K. Sujeeth, HyoukJoong Lee, Kevin J. Brown, Hassan Chafi, Michael Wu, Anand R. Atreya, Kunle Olukotun, Tiark Rompf, and Martin Odersky. 2011. OptiML: An Implicitly Parallel Domain-Specific Language for Machine Learning. In *Proceedings of the 28th International Conference on International Conference on Machine Learning* (Bellevue, Washington, USA) *(ICML'11)*. Omnipress, Madison, WI, USA, 609–616.

[312] Shiv Sundram, Akhilesh Balasingam, Thierry Tambe, Kunle Olukotun, and Fredrik Kjolstad. 2026. Cyclotron: The Streaming Multiprocessor Abstraction is Broken. Pittsburgh, Pennsylvania, 3. https://capra.cs.cornell.edu/latte26/paper/latte26-final28.pdf

[313] Joel Svensson, Koen Claessen, and Mary Sheeran. 2010. GPGPU Kernel Implementation and Refinement using Obsidian. *Procedia Computer Science* 1, 1 (2010), 2065–2074. doi:10.1016/j.procs.2010.04.231 ICCS 2010.

[314] Joel Svensson, Mary Sheeran, and Koen Claessen. 2011. Obsidian: A Domain Specific Embedded Language for Parallel Programming of Graphics Processors. In *Implementation and Application of Functional Languages*. Springer, Berlin, Heidelberg, 156–173. doi:10.1007/978-3-642-24452-0_9

[315] Nikhil Swamy, Juan Chen, Cédric Fournet, Pierre-Yves Strub, Karthikeyan Bhargavan, and Jean Yang. 2011. Secure Distributed Programming with Value-Dependent Types. In *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming* (Tokyo, Japan) *(ICFP '11)*. Association for Computing Machinery, New York, NY, USA, 266–278. doi:10.1145/2034773.2034811

[316] Ali Taha, Jiexiang Liu, and Hengjie Wang. 2025. Matrix Multiplication on Blackwell. https://www.modular.com/matrix-multiplication-on-blackwell. Accessed: 2026-06-03.

[317] Taichi Lang. 2022. Taichi: High-Performance Parallel Programming in Python. https://www.taichi-lang.org/ Accessed: 2026-07-28.

[318] David Tarditi, Sidd Puri, and Jose Oglesby. 2006. Accelerator: Using Data Parallelism to Program GPUs for General-Purpose Uses. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, San Jose California USA, 325–335. doi:10.1145/1168857.1168898

[319] Mohamed Tarek Ibn Ziad, Sana Damani, Aamer Jaleel, Stephen W. Keckler, and Mark Stephenson. 2023. cuCatch: A Debugging Tool for Efficiently Catching Memory Safety Violations in CUDA Applications. *Proc. ACM Program. Lang.* 7, PLDI, Article 111 (June 2023), 24 pages. doi:10.1145/3591225

[320] Rocq Team. 2026. The Rocq Prover. https://rocq-prover.org/ Accessed: 2026-05-29.

[321] The Agda Team. 2026. Agda. https://agda.readthedocs.io/en/latest/index.html Accessed: 2026-06-01.

[322] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, Matt Nicely, Duane Merrill, Dustyn Blasig, Aditya Atluri, Fengqi Qiao, Piotr Majcher, Paul Springer, Markus Hohnerbach, Jin Wang, and Manish Gupta. 2023. *CUTLASS*. https://github.com/NVIDIA/cutlass

[323] Arun Thangamani, Vincent Loechner, and Stéphane Genaud. 2024. A Survey of General-Purpose Polyhedral Compilers. *ACM Trans. Archit. Code Optim.* 21, 4, Article 72 (Nov. 2024), 26 pages. doi:10.1145/3674735

[324] The Halide Team. 2026. Halide: A Language for Fast, Portable Computation on Images and Tensors. https://halide-lang.org/#gettingstarted Accessed: 2026-06-20.

[325] The JAX Authors. 2024. Pallas: a JAX kernel language. https://docs.jax.dev/en/latest/pallas/index.html. Accessed: 2026-09-02.

[326] The Khronos Group. 2026. SPIR: The Standard IR for Parallel Compute and Graphics. https://www.khronos.org/spirv/ Accessed: 2026-06-09.

[327] The MathWorks, Inc. 2026. MATLAB. https://www.mathworks.com/products/matlab.html Accessed: 2026-06-24.

[328] The Rust Team. 2026. Rust. https://rust-lang.org/ Accessed: 2026-06-01.

[329] William Thies, Michal Karczmarek, and Saman P. Amarasinghe. 2002. StreamIt: A Language for Streaming Applications. In *Proceedings of the 11th International Conference on Compiler Construction (CC '02)*. Springer-Verlag, Berlin, Heidelberg, 179–196.

[330] C.J. Thompson, Sahngyun Hahn, and M. Oskin. 2002. Using Modern Graphics Architectures for General-Purpose Computing: A Framework and Analysis. In *35th Annual IEEE/ACM International Symposium on Microarchitecture, 2002. (MICRO-35). Proceedings.* 306–317. doi:10.1109/MICRO.2002.1176259

[331] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. ACM, Phoenix AZ USA, 10–19. doi:10.1145/3315508.3329973

[332] Seiya Tokui, Ryosuke Okuta, Takuya Akiba, Yusuke Niitani, Toru Ogawa, Shunta Saito, Shuji Suzuki, Kota Uenishi, Brian Vogel, and Hiroyuki Yamazaki Vincent. 2019. Chainer: A Deep Learning Framework for Accelerating the Research Cycle. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (Anchorage, AK, USA) *(KDD '19)*. Association for Computing Machinery, New York, NY, USA, 2002–2011. doi:10.1145/3292500.3330756

[333] Haining Tong, Natalia Gavrilenko, Hernan Ponce De Leon, and Keijo Heljanko. 2024. Towards Unified Analysis of GPU Consistency. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*. ACM, Hilton La Jolla Torrey Pines La Jolla CA USA, 329–344. doi:10.1145/3622781.3674174

[334] triton-lang. 2025. "01-intro.py" – Introduction to Gluon tutorial in Triton. https://github.com/triton-lang/triton/blob/main/python/tutorials/gluon/01-intro.py. Accessed: 2025-06-23.

[335] Leonard Truong, Rajkishore Barik, Ehsan Totoni, Hai Liu, Chick Markley, Armando Fox, and Tatiana Shpeisman. 2016. Latte: A Language, Compiler, and Runtime for Elegant and Efficient Deep Neural Networks. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, Santa Barbara CA USA, 209–223. doi:10.1145/2908080.2908105

[336] Abhishek Udupa, R. Govindarajan, and Matthew J. Thazhuthaveetil. 2009. Software Pipelined Execution of Stream Programs on GPUs. In *2009 International Symposium on Code Generation and Optimization*. 200–209. doi:10.1109/CGO.2009.20

[337] Sain-Zee Ueng, Melvin Lathara, Sara S. Baghsorkhi, and Wen-mei W. Hwu. 2008. CUDA-Lite: Reducing GPU Programming Complexity. In *Languages and Compilers for Parallel Computing*. Springer, Berlin, Heidelberg, 1–15. doi:10.1007/978-3-540-89740-8_1

[338] Swapneela Unkule, Christopher Shaltz, and Apan Qasem. 2012. Automatic Restructuring of GPU Kernels for Exploiting Inter-thread Data Locality. In *Compiler Construction*. Springer, Berlin, Heidelberg, 21–40. doi:10.1007/978-3-642-28652-0_2

[339] Benjamin Valpey, Xinyi Li, Sreepathi Pai, and Ganesh Gopalakrishnan. 2025. An SMT Formalization of Mixed-Precision Matrix Multiplication. In *NASA Formal Methods* (Cham, 2025). Springer Nature Switzerland, 360–379. doi:10.1007/978-3-031-93706-4_21

[340] Lars B. van den Haak, Anton Wijs, Mark van den Brand, and Marieke Huisman. 2020. Formal Methods for GPGPU Programming: Is the Demand Met?. In *Integrated Formal Methods* (Cham). Springer International Publishing, 160–177. doi:10.1007/978-3-030-63461-2_9

[341] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary Devito, William S. Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. 2019. The Next 700 Accelerated Layers: From Mathematical Expressions of Network Computation Graphs to Accelerated GPU Kernels, Automatically. *ACM Transactions on Architecture and Code Optimization* 16, 4 (Dec. 2019), 1–26. doi:10.1145/3355606

[342] Giorgos Vasiliadis, Elias Athanasopoulos, Michalis Polychronakis, and Sotiris Ioannidis. 2014. PixelVault: Using GPUs for Securing Cryptographic Operations. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security* (Scottsdale, Arizona, USA) *(CCS '14)*. Association for Computing Machinery, New York, NY, USA, 1131–1142. doi:10.1145/2660267.2660316

[343] Eelco Visser, Zine-el-Abidine Benaissa, and Andrew Tolmach. 1998. Building Program Optimizers with Rewriting Strategies. *SIGPLAN Not.* 34, 1 (Sept. 1998), 13–26. doi:10.1145/291251.289425

[344] Lei Wang, Yu Cheng, Yining Shi, Zhengju Tang, Zhiwen Mo, Wenhao Xie, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, and Zhi Yang. 2025. TileLang: A Composable Tiled Programming Model for AI Systems. arXiv:2504.17577

doi:10.48550/arXiv.2504.17577

[345] Qifan Wang and David Oswald. 2026. Confidential Computing on Heterogeneous CPU-GPU Systems: Survey and Future Directions. *ACM Comput. Surv.* 58, 9, Article 230 (Feb. 2026), 35 pages. doi:10.1145/3793532

[346] Anjiang Wei, Tianran Sun, Yogesh Seenichamy, Hang Song, Anne Ouyang, Azalia Mirhoseini, Ke Wang, and Alex Aiken. 2025. Astra: A Multi-Agent System for GPU Kernel Performance Optimization. arXiv:2509.07506 doi:10.48550/arXiv.2509.07506

[347] R. Clint Whaley and Jack J. Dongarra. 1998. Automatically Tuned Linear Algebra Software. In *Proceedings of the 1998 ACM/IEEE Conference on Supercomputing* (San Jose, CA) *(SC '98)*. IEEE Computer Society, USA, 1–27.

[348] Turner Whitted. 1980. An Improved Illumination Model for Shaded Display. *Commun. ACM* 23, 6 (June 1980), 343–349. doi:10.1145/358876.358882

[349] Sandra Wienke, Paul Springer, Christian Terboven, and Dieter an Mey. 2012. OpenACC — First Experiences with Real-World Applications. In *Euro-Par 2012 Parallel Processing* (Berlin, Heidelberg, 2012). Springer, 859–870. doi:10.1007/978-3-642-32820-6_85

[350] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Commun. ACM* 52, 4 (April 2009), 65–76. doi:10.1145/1498765.1498785

[351] Nicholas Wilt. 2013. Independent Thread Scheduling. In *The CUDA Handbook*. https://www.cudahandbook.com/book/ch7/independent-thread-scheduling Accessed: 2026-07-16.

[352] World Wide Web Consortium. 2026. WebGPU Shading Language. https://www.w3.org/TR/WGSL/ Accessed: 2026-08-27.

[353] Mingyuan Wu, Yicheng Ouyang, Husheng Zhou, Lingming Zhang, Cong Liu, and Yuqun Zhang. 2020. Simulee: Detecting CUDA Synchronization Bugs via Memory-Access Modeling. In *2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)*. 937–948. doi:10.1145/3377811.3380358

[354] Yue Xing, Bo-Yuan Huang, Aarti Gupta, and Sharad Malik. 2018. A Formal Instruction-level GPU Model for Scalable Verification. In *Proceedings of the International Conference on Computer-Aided Design* (San Diego, California) *(ICCAD '18)*. Association for Computing Machinery, New York, NY, USA, Article 130, 8 pages. doi:10.1145/3240765.3240771

[355] Divakar Kumar Yadav, Tian Zhao, and Deepak Kumar. 2026. Evaluating CUDA Tile for AI Workloads on Hopper and Blackwell GPUs. arXiv:2604.23466 doi:10.48550/arXiv.2604.23466

[356] Rohan Yadav, Alex Aiken, and Fredrik Kjolstad. 2022. DISTAL: The Distributed Tensor Algebra Compiler. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. ACM, San Diego CA USA, 286–300. doi:10.1145/3519939.3523437

[357] Rohan Yadav, Michael Garland, Alex Aiken, and Michael Bauer. 2025. Task-Based Tensor Computations on Modern GPUs. *Proceedings of the ACM on Programming Languages* 9, PLDI (June 2025), 396–420. doi:10.1145/3729262

[358] Yonghong Yan, Max Grossman, and Vivek Sarkar. 2009. JCUDA: A Programmer-Friendly Interface for Accelerating Java Programs with CUDA. In *Euro-Par 2009 Parallel Processing* (Berlin, Heidelberg, 2009). Springer, 887–899. doi:10.1007/978-3-642-03869-3_82

[359] Tyler Yandrofski, Jingyuan Chen, Nathan Otterness, James H. Anderson, and F. Donelson Smith. 2022. Making Powerful Enemies on NVIDIA GPUs. In *2022 IEEE Real-Time Systems Symposium (RTSS)*. 383–395. doi:10.1109/RTSS55097.2022.00040

[360] Wenhua Yang, Chong Zhang, and Minxue Pan. 2023. Understanding the Topics and Challenges of GPU Programming by Classifying and Analyzing Stack Overflow Posts. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) *(ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 1444–1456. doi:10.1145/3611643.3616365

[361] Yi Yang, Ping Xiang, Jingfei Kong, and Huiyang Zhou. 2010. A GPGPU Compiler for Memory Optimization and Parallelism Management. In *Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, Toronto Ontario Canada, 86–97. doi:10.1145/1806596.1806606

[362] Chang Yu, Yi Xu, Ye Kuang, Yuanming Hu, and Tiantian Liu. 2022. MeshTaichi: A Compiler for Efficient Mesh-Based Operations. *ACM Transactions on Graphics* 41, 6 (Dec. 2022), 1–17. doi:10.1145/3550454.3555430

[363] Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. 2026. FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling. arXiv:2603.05451 doi:10.48550/arXiv.2603.05451

[364] Wojciech Zaremba, Yuan Lin, and Vinod Grover. 2012. JaBEE: Framework for Object-Oriented Java Bytecode Compilation and Execution on Graphics Processor Units. In *Proceedings of the 5th Annual Workshop on General Purpose Processing with Graphics Processing Units* (London, United Kingdom) *(GPGPU-5)*. Association for Computing Machinery, New York, NY, USA, 74–83. doi:10.1145/2159430.2159439

[365] An Qi Zhang, Andrés Goens, Daniel Sorin, and Vijay Nagarajan. 2026. A Formally Verified Foundation for Compositional Heterogeneous Coherence. *Proc. ACM Program. Lang.* 10, PLDI, Article 272 (June 2026), 25 pages. doi:10.1145/3808350

[366] Yihong Zhang, Derek Gerstmann, Andrew Adams, and Maaz Bin Safeer Ahmad. 2026. Pushing Tensor Accelerators Beyond MatMul in a User-Schedulable Language. In *2026 IEEE/ACM International Symposium on Code Generation and*

Optimization (CGO). 242–254. doi:10.1109/CGO68049.2026.11395214

[367] Yongpeng Zhang and Frank Mueller. 2013. Hidp: A Hierarchical Data Parallel Language. In *Proceedings of the 2013 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 1–11. doi:10.1109/CGO.2013.6494994

[368] Mai Zheng, Vignesh T. Ravi, Feng Qin, and Gagan Agrawal. 2011. GRace: A Low-Overhead Mechanism for Detecting Data Races in GPU Programs. In *Proceedings of the 16th ACM symposium on Principles and Practice of Parallel Programming*. ACM, San Antonio TX USA, 135–146. doi:10.1145/1941553.1941574

[369] Mai Zheng, Vignesh T. Ravi, Feng Qin, and Gagan Agrawal. 2014. GMRace: Detecting Data Races in GPU Programs via a Low-Overhead Scheme. *IEEE Transactions on Parallel and Distributed Systems* 25, 1 (Jan. 2014), 104–115. doi:10.1109/TPDS.2013.44

[370] Shaokun Zheng, Zhiqian Zhou, Xin Chen, Difei Yan, Chuyan Zhang, Yuefeng Geng, Yan Gu, and Kun Xu. 2022. LuisaRender: A High-Performance Rendering Framework with Layered and Unified Interfaces on Stream Architectures. *ACM Trans. Graph.* 41, 6, Article 232 (Nov. 2022), 19 pages. doi:10.1145/3550454.3555463

[371] Keren Zhou, Mario Lezcano, Adam Goucher, Akhmed Rakhmati, Jeff Niu, Justin Lebar, Pawel Szczerbuk, Peter Bell, Phil Tillet, Thomas Raoux, and Zahi Moudallal. 2026. Linear Layouts: Robust Code Generation of Efficient Tensor Computation Using $\mathbb{F}_2$. arXiv:2505.23819 doi:10.48550/arXiv.2505.23819

[372] École Polytechnique Fédérale . 2026. Scala. https://www.scala-lang.org/ Accessed: 2026-06-24.

[373] Artjoms Šinkarovs and Troels Henriksen. 2025. Correctness Meets Performance: From Agda to Futhark. *Proceedings of the ACM on Programming Languages* 9, ICFP (Aug. 2025), 567–596. doi:10.1145/3747524